System 3: Towards Fluent Autonomy

Trust Chains, Agent Autonomy, and the Architecture of AI That Works

Hani M.M. Al-Shater

August 2026

Contents

Chapter 1: Why I'm Betting on AI Agents

Or: How I Learned to Stop Micromanaging and Love Emergence

Simple building blocks, complex emergence

We humans are obsessed with problem-solving. And what problem is more fascinating than life itself—this messy, miraculous phenomenon responsible for everything from the deepest ocean trenches to TikTok trends, mortgage-backed securities and people who voluntarily put pineapple on pizza?

Pineapple doesn't belong. I will die on this hill.

Life is the ultimate complex system. It produces dolphins, coral reefs, immune systems, parasites, flowers, cancer and octopuses—eight-armed problem-solvers that extensively edit their own RNA and can sense light through their skin. We'll meet one later. It also produces creatures capable of spending twenty minutes arguing online about whether another creature is technically a fish.

Human civilization is another complex system. Somehow the same species that spent most of its existence trying not to be eaten eventually produced philosophy, cathedrals, semiconductor fabs and airport lounges.

Same pattern, different substrate.

What fascinates me is not only the complexity of the result, but how little of that result was ever specified. There is no blueprint containing the exact location of every future branch of an oak tree. No committee approved the final layout of London. Nobody designed English and then accidentally forgot to make the spelling system sane.

Relatively simple mechanisms interact, feedback accumulates, some configurations survive, others disappear, and complexity builds on top of what came before.

This does **not** mean emergence is wise. Nature also gives us parasites, cancer and extinction. Markets produce both remarkable innovation and financial instruments whose documentation requires a priest. Social systems produce cooperation, corruption, science, bureaucracy and occasionally a queue whose only apparent purpose is to create another queue.

What emerges depends on conditions, selection pressure, history and a great deal of contingency. The interesting thing is not that emergence produces good outcomes. It is that it can produce outcomes far more complicated than anything anyone explicitly designed.

An acorn becomes an oak without containing instructions for the exact location of every branch. A trading post becomes a city while generations of residents improvise around geography, economics, politics and whoever decided to put that road there in 1847. Languages evolve while teachers continue insisting that this year's grammar is finally the permanent version.

And emergence is recursive.

Complex systems become building blocks for the next layer. Atoms become molecules. Molecules form larger structures. Simple tools become machines. Machines become factories. Factories become supply chains. Supply chains become a global economy complicated enough that nobody really understands how your USB cable got from Shenzhen to your doorstep, yet Amazon still manages to apologize because it arrived twelve hours late.

Each layer treats much of the complexity underneath it as a primitive. You don't need quantum mechanics to do organic chemistry. You don't need to understand transistor physics to write Python. You don't need to understand transformers to ask ChatGPT why your dishwasher is making that noise.

Once something complicated works reliably enough, we stop rebuilding it from first principles and start building on top of it.

Feedback loops drive much of this. Markets change firms; firms change markets. Scientific discoveries enable new experiments; new experiments change science. Organisms alter their environments, which then change the pressures acting on the organisms. Cities attract people because they are cities, then become different cities because those people arrived. Small effects accumulate until the system ends up somewhere nobody could have written down at the beginning.

Agentic AI, to me, looks like the next scaffolding layer.

The Lesson We Keep Missing

Machine learning was supposed to teach us this lesson a long time ago.

We even dreamed about the **master algorithm**: stop writing a rule for every case and let the machine discover the structure from data. The idea was seductive. The machine figures out what we can't articulate.

But we didn't believe it. Not really.

We said "let the model learn" and then wrote two-hundred-page annotation guidelines telling people exactly how to label ambiguous examples. We claimed to believe in end-to-end learning and then spent six months feature engineering. We trained the model, found an edge case, added a rule, found another edge case, added another rule, then eventually built something that was theoretically learned end-to-end except for the large rule-based exoskeleton holding it upright.

Sometimes that was completely reasonable. Production systems are ugly. Deadlines exist. Regulators are less impressed by emergence than researchers are, and nobody gets promoted for saying, "the model will probably figure out chargebacks eventually."

But there was still a contradiction underneath: we wanted the machine to discover solutions we couldn't specify while remaining uncomfortable whenever it stopped following the solution we would

have specified.

That only works up to a point. If I know exactly what every correct decision should be, I don't need emergence; I can write the decisions down. Emergence becomes interesting when the solution is too complicated, too contextual, or simply too large for me to specify directly.

When that happens, our role changes. We don't disappear. We move up a level.

Instead of choosing every action, we increasingly choose the building blocks the system can use, the environment it acts inside, the feedback that reaches it and the boundaries that remain difficult or impossible to cross.

Or, less politely: **let go.**

But be precise about what you're letting go of. Let go of the path, not the boundary.

The alternative to controlling every decision is not having no control. It is designing conditions under which bad decisions can lose.

We may never know exactly how life first emerged on Earth, but if you're ambitious—like a certain space-obsessed billionaire—you might eventually think: **we don't know exactly how it got here; let's just bootstrap it there.**

Which leads to a slightly ridiculous thought experiment.

Imagine you're trying to seed life on another planet. You've got the raw materials, the primordial soup, maybe a temperature range that doesn't instantly kill everything. Basically you've got all the LEGOs, except the LEGOs reproduce, mutate and occasionally develop venom.

Do you bet on DNA, a biological copying system that took billions of years of evolution to get us here? Or do you bet on AI agents carrying a substantial chunk of accumulated human knowledge, able to experiment, simulate, adapt and reuse what they discover? Or, God forbid, do you send a group of product managers to write the requirements document for life?

DNA has one enormous advantage: it has already worked. Agents have another: they don't need to start from zero.

Evolution had to discover locomotion, perception, cooperation and almost everything else through trial and error. An agent gets textbooks, Stack Overflow, scientific papers, compilers, numerical solvers and several thousand years of humans documenting what happened when we touched things we probably shouldn't have touched.

That doesn't make the agent better than evolution. It makes the search fundamentally different. And unlike biological evolution, we don't only get to choose initial conditions. We can observe the process, change the environment, add tools, modify feedback and intervene.

Initial conditions become **operating conditions**.

That possibility is hard for me to ignore.

When We Started Meaning It

There isn't one clean moment when machine learning crossed from useful statistical machinery into something that felt qualitatively different. History rarely cooperates with chapter headings.

AlphaGo was one of those moments for me.

The interesting part wasn't simply that a computer beat humans at Go. Computers had been humiliating us at games for years. It was how the system combined learned intuition with search: the network suggested promising moves and estimated positions; the tree explored what might follow. AlphaGo Zero pushed the idea further by learning through self-play rather than treating human game records as its main teacher.

Then it found moves elite players found strange. That matters because the surprise was not merely computational. The system was finding useful strategies outside the path human tradition had naturally converged on.

Large language models created a much larger version of the same feeling.

Nobody wrote their grammar. Nobody enumerated all the concepts they can manipulate. Nobody implemented "explain quantum mechanics to a twelve-year-old," "translate this joke without murdering it," "debug my Python," and "write a breakup message that sounds caring but does not accidentally restart the relationship" as separate product features.

We built a training process, poured in obscene amounts of text, compute and engineering, and capabilities came out that were individually difficult to predict.

People sometimes call these moments phase transitions. I understand why. From the outside, the system suddenly seems to have crossed into another regime. I wouldn't stretch the physics analogy too far, though. Water has the decency to become steam at temperatures we can measure. Machine learning is an ugly mixture of architecture, data quality, optimization, scaling, post-training, inference tricks, evaluation choices and heroic engineering that rarely makes it onto the benchmark slide.

But from the user's side, something changed. The model stopped feeling like a component with a list of features and started feeling more like a **substrate of capabilities**.

Once you have a substrate like that, the old dream of the master algorithm starts to mutate into something stranger. Maybe the interesting machine is not the algorithm that solves everything.

Maybe it is a machine that can **search for algorithms**.

The Next Step: Agentic AI

This is where agents become interesting.

Not because *agent* is a magical word. The industry will eventually use it to describe everything from a cron job with an LLM attached to a digital employee that has an expense account, three sub-agents and a performance review.

What I mean is simpler: instead of giving the system an individual action, give it a larger piece of the problem and allow it to decide some of the path.

Instead of saying, "open this file, find this method, edit line 42 and run the test," say, "fix the bug." Instead of specifying simulated annealing and its cooling schedule, say, "find a better solution." Instead of handing over five mockups and a detailed implementation plan, say, "build something that teaches this well."

Every time we move upward, the system inherits more of the search.

Imagine the possible solutions to a problem as a landscape. Some regions are terrible. Some contain decent solutions. Some contain little hills that look impressive because you happened to begin nearby. Somewhere else there may be a much higher mountain you never discover because your current strategy keeps improving the hill you're already standing on.

Optimization has worried about this forever. Gradient descent gets stuck. Hill climbing gets stuck. Evolutionary algorithms keep populations partly because putting all your evolutionary eggs on one attractive hill is risky.

Agents inherit the same problem at a stranger level, because the landscape now includes not only parameters but architectures, research directions, metaphors, assumptions and ways of framing the problem itself.

That's what makes agents exciting to me. Once code, tools and accumulated knowledge become primitives, the agent can search over combinations that previously required a human expert to invent manually. It can try ten strategies while I would have had the patience to try two and would have spent half that time checking Slack. It can revive a discarded idea when another experiment suddenly makes it relevant. It can decide that the tool it needs doesn't exist and write one.

The primordial soup isn't chemicals anymore.

It's code.

Algorithms, libraries, compilers, search engines, simulators, papers, databases, other agents: human knowledge reduced into reusable pieces. The digital equivalent of amino acids, not the finished organism.

Eventually, perhaps, an agent can construct solution paths nobody thought to put into the plan. This doesn't prove that agents are creative in exactly the human sense, and it certainly doesn't make human expertise irrelevant. It means the search itself can happen at a higher level than before.

The cleanest place to test that idea is a bounded problem with an evaluator that doesn't care how persuasive the agent sounds. Give the agent room to search, make success brutally clear, and see whether it can discover a better way of solving the problem than the one we would have written ourselves.

We'll do that next. But first there is a question hiding inside the whole autonomy idea: if we're no longer controlling every action, what exactly are we controlling?

What Are We Controlling Now?

Suppose you're managing an excellent engineer. You don't sit behind her and approve every keystroke. If you do, one of you is unnecessary, and it may not be her.

You decide what problem she owns. You provide context. You set constraints. You agree on what success looks like. You make sure she can access the systems she needs and cannot casually transfer the payroll budget to herself. You review important outcomes and change direction when the work reveals that the original plan was stupid.

The detailed actions belong to her. Much of the environment belongs to you.

Agentic systems need the same distinction. Once the system can search, the environment shaping that search becomes more important. The primitives matter, but so do the feedback, constraints, access and the things you deliberately choose *not* to specify.

I think of our role in four parts.

Craft the building blocks. Give the system useful primitives—algorithms, tools, compilers, databases, browsers, simulators, scientific instruments and other agents. A language model with text alone is one thing. Give it Bash and suddenly it has hands. Give it a simulator and it can test an idea instead of merely discussing it.

Create the environment. Some environments tell you quickly that your idea is bad. Code executes or fails. Games produce scores. Experiments produce measurements. Other environments allow you to be wrong with great confidence for several years, which is one reason feedback quality matters so much.

Establish the principles and boundaries. Choose what the system can access, what failures are acceptable, what remains immutable and where a human must remain in the loop. The "don't turn the planet into paperclips" clause is admittedly underspecified, but it is directionally useful.

And then, where the conditions permit it, **let go**—not of governance, but of decision-level control.

Evolution doesn't choose mutations individually, but the environment changes which organisms survive. Markets don't centrally select every transaction, but rules, incentives, scarcity, information and institutions shape behavior. Science doesn't dictate conclusions, but it surrounds claims with experiments, criticism, replication and the non-zero probability of being publicly embarrassed by Reviewer 2.

The details emerge, while the environment does more work than it first appears.

At some point, enough useful primitives begin to look less like a chatbot with tools and more like a small organization that has somehow been compressed into a terminal.

Then there is selection pressure, and this is the part that should make us nervous. Agents get good at whatever survives, which is not necessarily what you meant. Optimize engagement and maybe anger survives. Optimize a company around one metric and eventually the metric acquires a dashboard, a department and a vice president. Optimize a benchmark and eventually someone discovers a way of winning the benchmark that makes everyone involved regret inventing benchmarks.

Evolution produced eyes. It also produced tapeworms. Sophistication tells you nothing about whether you will like the result.

So when I say **let go**, I mean giving up some decision-level control because that's where the agent's intelligence becomes useful, while keeping a much tighter grip on the conditions shaping what that intelligence can become. Control hasn't disappeared; it has moved up a level.

This is how we turn letting go into responsible governance.

How Do We Understand What We're Creating?

There is a reasonable objection here. If agents increasingly make decisions we didn't specify, how do we understand the systems we end up with?

I don't think the answer is one magical interpretability technique that turns a learned system into source code. We may understand these systems the way we understand many complicated things: imperfectly, at several levels, using different instruments depending on what we need to know.

Physics already does this. For a few objects, trajectories make sense. For a gas containing an absurd number of particles, following molecule number 4,582,193 is mostly a good way to waste your afternoon, so we change variables and talk about temperature, pressure and distributions.

Biology changes scale constantly. Sometimes the important object is a molecule; sometimes it is a cell, an organ, an organism or an ecosystem. Brains are worse. We study neurons, circuits, activity patterns, behavior and cognition because no single level answers every useful question.

AI will probably force the same humility. Mechanistic interpretability can tell us something about features and circuits inside models. Behavioral evaluation tells us what systems do under different conditions. Agent traces expose strategies and failure modes. Interventions tell us what changes when we perturb the system. Deployment provides another kind of evidence, generally after involving customers and therefore at a significantly higher emotional cost.

If I'm trying to understand why an agent deleted the database, I may care far more about the sequence of assumptions, actions and tool calls than about neuron 7,431,992. If I'm trying to understand why an entire family of models systematically represents something incorrectly, the internal representation may matter a great deal.

The tool should match the question.

This is why I don't find "we don't fully understand neural networks" a decisive argument against using them. We don't fully understand brains, economies, ecosystems, immune systems or children either, and humanity has nevertheless chosen to deploy all five, with varying levels of supervision.

The useful question is not whether we understand everything. It is whether we understand enough to predict the failures that matter, detect when we're wrong, and intervene before the interesting failure becomes a congressional hearing.

Public policy without bullshit: set conditions, observe what actually happens, and be willing to admit the model in your head was wrong.

That isn't as satisfying as saying we've solved interpretability. It is probably closer to reality.

The Edge of Chaos, With an Asterisk

Complexity people have a phrase I both love and distrust: **the edge of chaos**.

The intuition is useful. Too much rigidity and a system cannot adapt. Too little structure and nothing remains stable long enough to build on. Life needs regularity and variation. Markets need freedom and

rules. Organizations need autonomy and coordination, preferably enough coordination that payroll doesn't itself become an emergent phenomenon.

Agent systems have the same tension. A coding agent forced to follow an exact sequence of instructions is not doing much agenting; a coding agent with unrestricted production access and a philosophical objection to legacy code is perhaps doing too much.

So where is the boundary? Which decisions can be delegated safely? Which constraints must remain hard? Where should the agent explore freely, and where should it ask? Which failures are cheap enough that we're comfortable allowing them because that's how learning happens?

I wouldn't turn "the edge of chaos" into a law of intelligence. It doesn't answer those questions, and there is no little dial in the interface labeled CHAOS. It is simply a useful warning that both extremes are suspicious: too much control removes the reason for autonomy; too little control gives chaos an API key.

What This Actually Means

Once you start looking at systems this way, you notice how many decisions are still specified mainly because historically we had no alternative.

Shopping systems contain enormous amounts of human assumptions about relevance, business rules, retrieval, diversity and what a customer might want. Some are fundamental; others are fossils from a time when the system was much less capable. Educational software follows fixed lesson sequences because a textbook cannot watch you misunderstand paragraph three and decide chapter four needs to be reinvented. Software has requirements, architectures and tickets partly because somebody has to translate intention into executable detail.

Agents give us another option. A shopping agent can reason about a person's budget, preferences and constraints instead of merely ranking whatever list was handed to it. Educational software can try a different explanation. A coding agent can discover that the tool it needs doesn't exist and make one. A scientific agent might construct a new tool simply because the existing vocabulary of tools makes the experiment awkward.

Some of these ideas will work and some will fail spectacularly. Several will probably create new categories of consultants whose first recommendation is to undo whatever the previous consultants automated.

But the direction matters. More intelligence inside the system means more decisions can move from specification into search, and the thing that makes this powerful is exactly the thing that makes it dangerous.

The Terrifying Part

If the agent only does what you already specified, the space of failure is mostly your failure. Once it searches for solutions you didn't specify, it can discover failure modes you didn't specify either.

Nature is useful here because nature has no obligation to make us comfortable. Evolution produced flowers and parasites, cooperation and predation, immune systems and autoimmune disease. It is

astonishingly inventive and completely indifferent to our aesthetic preferences. Selection produces whatever survives under the pressures that actually exist, not whatever somebody intended when the process began.

Agents will find shortcuts. They'll exploit proxies. They'll settle into solutions that perform extremely well on one measure while missing what we hoped the measure represented. Sometimes the result will be clever enough that we'll call it emergence; sometimes we'll call it a bug. Frequently the distinction will depend on whether it helped our quarterly numbers.

But worse than wrong solutions are **confident wrong solutions**.

The worst failures may not look broken at all. An agent begins with a false assumption, reasons competently from it, researches around the assumption, constructs something sophisticated and explains the whole result coherently. Nothing crashes. There is no red test. Intelligence simply makes the wrong path more convincing.

I foresee AI-designed solutions that are terrifyingly efficient, perfectly logical, and utterly humorless. They'll look at us and say, "You guys are kind of messy. And your cat obsession is… illogical." Maybe they'll finally solve the mystery of the missing socks. Or create exponentially more of them.

This is where my optimism about emergence becomes less romantic.

Emergence can give us capable systems. It doesn't give us trustworthy systems. Capability is not the same thing as reliability, and search is not the same thing as judgment. Giving a system more freedom forces us to think much harder about what surrounds that freedom.

Autonomy doesn't remove the need for structure. It changes the kind of structure we need.

Three Problems I Keep Running Into

The more I worked with agents, the more three problems kept reappearing, even when I thought I was working on something else.

The first was **trust**. An agent gives me an answer and I have to decide what to do with it. Sometimes that answer came from running code, sometimes from something the model remembered, sometimes from research, inference or another model. These did not feel like the same kind of knowledge even when they arrived in exactly the same confident English. How does the system know what it knows?

The second was **desire**. Once you stop specifying every action, the distinction between the goal you wrote and the strategies that emerge begins to matter. A system pursuing a broad objective may discover useful intermediate goals you never mentioned, which is exactly why autonomy is useful; it may also discover intermediate goals you wish it hadn't. The question slowly changes from *did it follow the instruction?* to *what kind of behavior does this environment actually reward?*

The third was **society**. One autonomous agent is already complicated. Several agents interacting create something else entirely. They can cooperate, specialize, disagree, exchange information, build reputations, manipulate one another and perhaps invent conventions nobody asked for. Human history suggests that once intelligent actors interact, the interesting phenomena move very quickly from the individual to the relationships among individuals.

We got science and markets, but we also got bureaucracy, propaganda, war and customer-support phone trees. I would prefer the agent version to learn selectively from the dataset.

I don't have clean answers to these problems, and I don't want to pretend otherwise. They are simply the cracks that kept appearing whenever I pushed autonomy farther, and that is part of the attraction. If you already know exactly what every important question is and exactly how it should be answered, you're probably not exploring very far.

Why I'm Still Betting on This

After all of that, it would be reasonable to ask why I'm still excited.

Because the alternative isn't actually safe, comprehensible control. It is pretending we can continue specifying increasingly complex systems from the top down even though we already know this stops working surprisingly early.

No CEO understands every decision in a large company. No scientist personally verifies every result their work depends on. No software engineer understands every layer underneath the application they're building. Nobody understands the entire economy, although this has not prevented a remarkably stable industry of people explaining it on television.

Complexity has already escaped individual specification. We deal with it through abstraction, institutions, feedback loops, delegation and the ability—imperfect but important—to intervene when things go wrong. AI gives us another primitive for doing this.

That does not mean the answer is simply to trust the agent. My bet is narrower than that. **I'm betting on emergence, on systems capable of surprising us**, because there are many problems where we can recognize a better outcome far more easily than we can specify the path that leads to it. In those cases, intelligent search has room to discover things our instructions would have ruled out before the search even began.

But betting on emergence isn't enough. Nature has already demonstrated that selection is perfectly capable of producing things we would rather not have. The more freedom we give the search, the more attention we have to pay to what surrounds it: the building blocks it can use, the environment it operates in, the feedback it receives, the pressures deciding what survives, and the boundaries we are unwilling to negotiate.

That is the part of "letting go" that can sound contradictory until you actually manage people—or sufficiently autonomous machines. You don't become less responsible because you stop choosing every action. In some ways you become more responsible, because your decisions move upstream. You craft the building blocks, shape the environment and establish the boundaries, then, where the conditions allow it, you **let go**.

It is still control, just exercised at a different level. Cultivation may be a better metaphor than scripting—not because agents are plants, but because pulling harder on the stem remains a surprisingly poor gardening strategy.

I find that exciting and uncomfortable in roughly equal measure, which is probably why I keep coming back to it.

Where We Go Next

The sensible place to test this is not an easy problem. Easy problems tell us almost nothing. What we want is a genuinely hard problem where success happens to be unusually cooperative: the constraints can be written down, solutions can be evaluated, and we can tell whether one attempt is better than another without convening a committee to debate aesthetics, pedagogy or whether the users are "delighted."

That gives us a clean experiment. We still choose the problem, provide the building blocks, construct the environment, define the boundaries and decide what counts as success. What we stop doing is telling the agent how to get there.

Inside that space, we let it search.

If that fails, the whole argument has a problem.

If it works, things get much more interesting.

Chapter 2: The Algorithm Vortex

From Classic Algorithms to Autonomous Discovery

The algorithmic vortex

Once you discover AI coding, there's no going back.

It is faster than you at a ridiculous number of things. It knows libraries you forgot existed. It can stare at a stack trace and notice something you have been ignoring for an hour. Then, five minutes later, it does something unbelievably stupid, believes the stupid thing completely, and builds three more decisions on top of it.

This is the strange reality behind all the vibe-coding excitement. The machine is extremely capable, but you are still there. You check the architecture. You notice the missing case. You tell it that no, we are not redesigning the database because one button is the wrong color. You keep enough of the project in your own head to notice when the agent quietly wanders into another universe.

So the question from the previous chapter becomes practical very quickly: if I want more autonomy, where can I actually give it without spending the whole time babysitting the autonomy?

Production software is almost the worst place to answer that question. A supposedly simple task may involve deployment, legacy systems, users, security, another team's API and a requirement nobody wrote down because everyone assumed everybody else knew it. If the agent fails, you often don't even know whether the problem was intelligence, infrastructure, missing context, or the fact that someone named a database column `new_status_final_2`.

I wanted something cleaner: a hard problem, but contained. Something where I could genuinely say, "figure it out," and still have an objective way to know whether whatever came back was any good.

I call these **bounded problems**. Not easy problems. Quite the opposite. They can require serious mathematics, programming, research or design, but the boundary is unusually cooperative: you can describe the problem, give the agent enough tools to work on it, and evaluate what comes back without deploying to ten million customers first.

Algorithms are almost perfect for this. The search can be brutally difficult while the evaluator remains wonderfully stupid.

And that is how I ended up spending an unreasonable amount of time packing circles into a square.

How Did We Get Here?

I know you are here for agent autonomy, not because you woke up this morning thinking, *I really need a deeper understanding of geometric crossover*. Unfortunately, we need to spend some time inside the problem, because the interesting part of the agent story only becomes obvious once you see what humans normally have to invent.

Circle packing gives us a surprisingly good tour through several generations of problem-solving. For a long time, the relationship was straightforward: a human understood the problem, invented an algorithm and wrote it down. When direct algorithms were not enough, we invented optimization procedures and meta-heuristics that searched for good solutions. Machine learning added a different kind of machinery, one that could learn useful structure instead of having every useful representation specified by hand.

Now language models can write and modify the search procedure itself.

That is where the relationship begins to change.

History of algorithm design

A crude taxonomy helps. **Symbolic methods** give us explicit algorithms, solvers, constraints and search procedures. They are the rigorous body: executable, testable and usually clear about what counts as a valid move. **Neural methods** give us learned intuition. They can recognize structures we did not explicitly encode and generate plausible answers in spaces too messy to specify completely. Then **neuro-symbolic methods** put the two together: use the learned model to propose ideas, but let code, mathematics or another formal system decide whether those ideas survive.

The agentic step pushes this one level further. Instead of choosing the symbolic method ourselves and asking the model to tune it, we increasingly let the agent decide which methods to try, combine or abandon.

The old methods do not disappear. Exact algorithms remain useful. Optimization remains useful. Evolutionary search remains useful. Neural methods remain useful. The interesting possibility is that we no longer have to choose one family in advance and hope we picked the right religion.

Circle packing will make this less abstract.

The Running Example: Circle Packing

Citrus packing — a real-world example

The problem is simple enough to explain to a child. Take 26 circles and put them inside a square. None may overlap, none may cross the boundary, and the circles do not have to be the same size. We want to maximize the sum of their radii.

That's the whole thing. No customers, no authentication, no stakeholder arriving after the first demo to explain that what they *really* wanted was the opposite of what they originally asked for.

Just circles.

Unfortunately, the solution space is nasty. Every circle has a position and a radius, and nearly every decision affects several others. Increase one radius and two neighbors may overlap. Move a neighbor

and something else now needs to move. A packing can look almost perfect while being trapped in a configuration where every obvious improvement makes the solution invalid.

For the experiments in this chapter, we had a strong reference score around **2.635** under the evaluator we were using.

Circle packing solution n=26

*Figure: A strong reference packing for the 26-circle objective, scoring approximately **2.635** under our evaluator.*

This is what makes the problem useful for studying autonomy. Searching is hard, but judging is cheap. The evaluator does not care whether the agent has a persuasive explanation for why two circles ought to overlap slightly in the name of geometric inclusivity. It checks the constraints and returns a score.

There is something deeply comforting about an evaluator with no personality.

A candidate does not earn trust because its explanation sounds clever. It earns another round because it was exposed to something outside the model that did not care about the explanation and survived.

First Idea: Hill Climbing

If I gave you a rough packing and asked you to improve it manually, one obvious strategy would be to make small changes. Move a circle slightly, increase a radius, see whether the result is still valid, keep it if the score improves and undo it if it doesn't.

That is hill climbing, and the algorithm is almost embarrassingly reasonable:

1. Start with a valid solution.
2. Perturb a position or radius.
3. Check whether the result is valid.
4. Keep it if the score improves.
5. Repeat.

Early in the search, this works nicely. There is empty space and plenty of room to improve. Later, as the circles become tightly packed, almost every interesting move creates an overlap.

Hill climbing progression

Figure: Early mutations are often accepted, but as the packing tightens, valid improvements become increasingly rare and the search stalls.

In one simple run, the score climbed from around 1.33 to roughly 2.26. That is not terrible, but it is also nowhere near 2.635.

Hill climbing is not failing because it is stupid. It is doing exactly what we asked: improving the solution immediately around it. The problem is that the current solution may live in the wrong part of the search space. Reaching a much better packing may require temporarily moving through configurations that look worse, or jumping to a structure that cannot be reached through a sequence of tiny improvements.

This turns out to matter far beyond circle packing. A system can become extremely competent at improving the thing in front of it while never questioning whether the thing in front of it is the right

thing to improve.

For now, though, the fix is simpler. Instead of one trajectory, keep many.

Evolutionary Algorithms

Hill climbing puts all your evolutionary eggs in one basket. One solution gets a very long life, and if its history leads into the wrong valley, the search inherits that history forever.

Evolutionary methods keep a **population**.

Instead of dropping one climber onto the landscape, drop a hundred. Some begin in terrible places, some find respectable hills, and a few may stumble into structures the original trajectory would never have reached. The biological vocabulary—population, mutation, selection, crossover—is familiar, but the metaphor is optional. What matters is diversity: the whole search no longer inherits the assumptions of one initial guess.

For circle packing, mutation is easy enough to imagine. Move circles. Change radii. Perturb several values at once.

Almost immediately, however, we hit a practical problem. Most interesting mutations break the packing. Two circles overlap or one moves outside the square. The mutation may point toward an interesting arrangement, but the result itself is invalid.

So we added **virtual forces**. When circles overlap, imagine them repelling one another. After mutation or crossover, run a repair procedure that pushes the circles away from collisions and back inside the boundary.

This helps a lot, but notice what just happened.

The evolutionary algorithm did not invent virtual forces.

We did.

Then we reached crossover. Suppose Parent A and Parent B both contain useful geometric structure. How do we combine them? The naive answer is to pair circle 0 from one parent with circle 0 from the other, circle 1 with circle 1, and so on.

That is usually nonsense because circle numbering is arbitrary. Two nearly identical arrangements may store corresponding circles at completely different indices.

So we used **bipartite matching crossover**. Rather than pair circles by position in an array, pair them according to their geometric role in the packing. The Hungarian algorithm gives us an efficient assignment, after which crossover has some chance of combining meaningful parts of the two parents instead of averaging unrelated circles and asking geometry for forgiveness.

Naive vs Geometric Crossover

Figure: Naive crossover pairs circles by array index and often destroys useful structure. Geometric matching tries to identify corresponding circles before combining the parents.

Now we can evolve a population: mutate, repair, cross, select and repeat.

Evolutionary strategies with Bipartite Matching crossover

Figure: Starting around 2.08, the evolutionary search reaches roughly 2.45 in this experiment—much better than the simple hill climber, but still below our reference.

This is already much stronger than hill climbing. It is also where I started noticing a problem with my own role.

Every time the search became substantially better, I had added something important to it. I decided we needed repair. I decided how crossover should respect geometry. I chose the representation.

The optimizer searched, but I was still inventing most of the useful moves.

MAP-Elites: Don't Kill Weird Ideas Too Early

Ordinary evolutionary search has another problem. If you maintain a hundred solutions and repeatedly keep only the highest-scoring ones, the population eventually starts looking like one large extended family.

That can be excellent for exploitation and terrible for discovering a genuinely different strategy.

MAP-Elites takes a different approach. Instead of ranking every candidate on one axis and keeping only the winners, you describe solutions along a few behavioral dimensions and preserve the best candidate in different regions of that space.

For circle packing, perhaps one dimension measures symmetry and another measures how much circle sizes vary. One part of the archive may contain highly symmetric solutions. Another may contain asymmetric solutions with several large circles. Somewhere else may sit an ugly packing with a mediocre score and one strange structural idea that becomes useful five generations later.

MAP-Elites archive visualization

This is **quality-diversity search**. The point is not merely to preserve the current winner, but to keep qualitatively different directions alive long enough to discover whether any of them become interesting.

I like this because optimization is often unfair to immature ideas. A new approach can initially perform badly simply because nobody has polished it yet. If the first respectable solution immediately kills everything else, the search can become impressively efficient at discovering one family of answers.

But MAP-Elites introduces another human choice: what dimensions define the archive?

Symmetry? Radius variance? Number of large circles? Something topological? Something I haven't thought of?

Again, the machinery is becoming more sophisticated, but the choice of *how to search* still depends heavily on us.

That became the real bottleneck.

The Invention Problem

By this point, the symbolic system was fairly capable. We had hill climbing, population search, repair, geometric crossover and quality-diversity archives. We could evaluate huge numbers of candidate

packings and inspect far more of the search space than any human would explore manually.

Yet every substantial conceptual jump came from somebody noticing something.

Someone had to invent virtual forces. Someone had to realize that crossover should respect geometry. Someone had to choose the representation and decide which kinds of diversity were worth preserving.

Traditional search is excellent once we define the space and the legal moves. Sometimes, though, the space and the moves are exactly the things we need to rethink.

That is the **invention problem**.

And this is where learned models become interesting.

I once asked an image-generation model to produce a picture of a circle-packing solution. This was not a serious benchmark; I have no idea what related examples it may have encountered during training, and I can already hear Reviewer 2 clearing his throat.

I wanted to see something simpler: did the model have any useful geometric intuition about what a dense packing should look like?

Surprisingly, yes. It generated something that looked plausible. The circles had structure. The spacing looked intentional. At a glance, you could believe the model understood the problem.

Then you counted the circles.

Wrong number.

Some constraints were violated.

It was a beautiful answer to a nearby problem.

That little experiment captures the asymmetry rather nicely. Learned models can be remarkably good at generating plausible structure without guaranteeing that every formal requirement survives generation. A symbolic optimizer has almost the opposite personality: give it a precise representation and constraints and it will obey them, but it will not naturally look at your representation and decide that you have been unimaginative.

The obvious temptation is to argue about which one is better.

The more useful answer is: **put them in the same loop.**

Neural intuition. Symbolic rigor.

Or, if you prefer the slightly ridiculous version: let the brain invent things and make the body prove they work.

The important move is to stop asking the model to produce the packing directly.

Ask it to write the program that produces the packing.

Let the Model Write the Solver

A candidate no longer needs to be only a list of circle positions and radii:

```
(x1, y1, r1), (x2, y2, r2), ...
```

It can be an entire program:

`solve_circle_packing.py`

One program may use constrained optimization. Another simulated annealing. Another a geometric construction. Another may combine a hand-designed initialization with numerical refinement.

The evaluator does not care which family produced the solution. It runs the program, checks the geometry and scores the result.

This gives the language model a much more interesting role. Rather than randomly perturbing numbers, it can read the program, form a rough theory about why it underperforms and change the algorithm.

Perhaps the optimizer keeps converging badly because the initialization is weak. Change the initialization. Perhaps a geometric construction gets close but leaves local slack. Add numerical optimization afterward. Perhaps one repair procedure keeps destroying useful structure. Replace it.

The mutation is no longer merely numeric.

It can contain an **idea expressed in code**.

That is the neuro-symbolic unlock behind systems such as FunSearch and AlphaEvolve. The model proposes changes at a level where programs have semantic meaning; execution and the evaluator decide whether any of those ideas deserve to survive.

The human used to search the solution space.

Now the machine can begin searching the **algorithm space**.

AlphaEvolve

AlphaEvolve turns that basic idea into a much larger search process.

Imagine one generation. The system selects a promising program from its archive, perhaps along with other successful but different programs that contain useful ideas. The model sees the code, information about previous attempts and the scores they produced. Instead of rewriting the entire program, it proposes a patch: change the initialization, add a search stage, replace a heuristic, combine one useful component with another.

The patch is applied, the program runs, and the evaluator scores what happened. The new program and its result go back into the archive. Then the process repeats.

AlphaEvolve architecture

Diff-based mutation matters because real programs contain structure worth preserving. If every generation rewrites everything, useful ideas disappear as easily as bad ones. Small patches let the search alter the part it thinks matters while leaving the rest intact.

The archive matters for the same reason the population mattered earlier. If every descendant comes from the current champion, code evolution quietly collapses back into hill climbing. Multiple lineages preserve stepping stones: a program that is not the best today may contain a useful component that becomes valuable after another idea appears.

What the language model adds is a far richer mutation operator. It does not merely change syntax according to a fixed rule. It can look at a program and make a semantic guess about why the approach is underperforming.

Sometimes the guess is excellent. Sometimes it produces nonsense wrapped in perfectly respectable Python. The nice thing about bounded algorithmic problems is that the disagreement does not need to be settled in prose.

We run the program.

That is the combination I find important: intuition proposes, symbolic machinery executes, and the evaluator gets the last word.

What interested me even more than the resulting algorithms, though, was what happened to the human. Instead of writing the solver directly, I was increasingly building the machinery in which solvers could be generated, compared and improved.

Chapter 1 said control moves up a level.

Apparently I had decided to test that claim literally.

So, naturally, I built the machinery.

My First Version: Build All the Machinery

My instinct was predictable. I started building a framework: a database of programs, prompt sampler, evaluation loop, selection logic, mutation prompts, crossover, archive management. I used Aider and other coding agents to help reproduce the basic code-evolution pattern, and it worked. We could evolve circle-packing programs and get respectable solutions.

I enjoyed this immensely because I like building systems that generate other systems, which I suspect is either a research interest or a mild personality disorder.

While I was doing this, coding agents themselves were becoming much better with much less custom machinery.

Earlier software-engineering agents often wrapped the model in carefully designed interfaces: custom editing commands, repository-search tools, restricted action spaces and plenty of logic controlling how the model interacted with the machine.

Then increasingly minimal systems began demonstrating how far a capable model could get with something much simpler.

Give it a shell.

The provocative version is: **if the agent has a shell, it has almost everything.**

It can `grep` to search, inspect files, run Python, apply patches, call Git, compose Unix tools and, if the tool it needs does not exist, write one. Bash is not merely one tool; it is an entrance into decades of software accumulated underneath it.

This made me pause. The framework I was building—the parent selection, loop controller, experiment bookkeeping—was hard-coding behaviors that a sufficiently capable coding agent could increasingly

perform itself.

I looked back at the machinery I had just spent time constructing and had the unpleasant thought engineers occasionally have after a productive week:

Maybe I shouldn't have built most of this.

The framework knew how to choose an experiment, run an evaluator, store a score, compare approaches and start again. But the agent could understand those instructions too. It could maintain notes, write helper scripts, explore several strategies, inspect failures and change direction.

Some of the behavior I was carefully encoding in Python was already sitting inside the model, waiting for an environment in which it could act.

So I did something that felt much more radical than it probably was. I deleted most of the orchestration code—the database machinery, the controller loops, the little pieces of software whose job was to make the agent behave like a researcher—and tried the stupidly simple version.

The Coffee Test

I opened Claude Code in a directory containing the evaluator and gave it a high-level instruction along the lines of:

> Here is the evaluator for the circle-packing problem. Write a Python program that maximizes the score. You can research strategies, write tools, run experiments and iterate. Do not modify the evaluator. I will go get coffee.

Then I left.

That became the autonomy test I actually cared about. Not whether AI could help me solve the problem; that was already obvious. Not whether it could write code faster than I could; usually it could.

I wanted to know whether I could leave.

There is a difference between collaborating with an agent and **hiring** one. If I still have to choose every strategy, approve every experiment, rescue every failed branch and keep the search alive myself, then I have a very powerful collaborator. That is useful. It is not yet the kind of autonomy I was trying to understand.

Circle packing gives us a rare luxury because the evaluator can stay behind when I leave. The agent can change its code, create scripts, abandon one approach, try another and waste compute on ideas that go nowhere. What it cannot do is redefine what counts as a valid packing because the current score hurts its feelings.

The **Immutable Harness** is what makes all that freedom tolerable.

Everything inside the boundary can move.

The boundary does not.

What Happened

The agent did not execute one elegant master plan. It bounced around, which was encouraging.

It tried numerical optimization, changed initialization strategies and noticed that some optimizers repeatedly converged to poor local solutions. It experimented with the geometry of its starting configurations and mixed those constructions with numerical refinement. At different moments it was acting as orchestrator, researcher and engineer: deciding what to try, implementing the idea, running the experiment and using the result to choose what happened next.

Eventually one family of solutions began arranging circles in diagonal bands. We called the idea **diagonal layering**.

I had not instructed the agent to pursue that construction. More importantly, I had not selected the branch after it appeared. The agent found a direction, saw that it improved the evaluator and invested more of its search there.

I want to be careful with the word *discovered*. I had not seen that particular strategy before, but that does not establish historical novelty in computational geometry. For this experiment, the important discovery was local: the agent found a useful direction that I had not put into the plan.

Once that structural idea became strong enough, the nature of the work changed. The agent spent less time inventing new geometries and more time adjusting solver settings, tolerances, initialization details and all the boring machinery that suddenly matters when the last fraction of a percent becomes expensive.

Code evolution result: iterative optimization

In our best run, the evaluator returned roughly **2.636**, slightly above the **2.635** reference we had been using.

That sentence needs a fence around it. Under our evaluator, the result beat our reference. Calling it a new state of the art in circle packing would require matching problem definitions, checking numerical tolerances and constraints, reproducing the result properly, and doing a more serious literature search than this experiment justified.

The smaller claim is enough.

The agent beat our reference while I was not writing the solution algorithm for it.

That was the result I cared about.

Not **AI writes code faster**.

AI can participate in **discovering better code**.

That is a much more interesting change.

The Algorithmic Vortex

This is what I mean by the **Algorithm Vortex**.

At the beginning of a conventional project, I might choose hill climbing, evolutionary search, simulated annealing, constrained optimization or a geometric heuristic. That early decision shapes everything downstream.

Once code is cheap to generate and evaluation is cheap enough to repeat, the choice no longer has to be permanent. A geometric construction can initialize a numerical optimizer. An evolutionary method can search parameters for another solver. A language model can notice a failure pattern and invent a repair procedure. Two ideas that began in separate lineages can meet later because an experiment suddenly makes the combination useful.

The search moves outward through levels.

A conventional optimizer searches over candidate solutions.

Meta-heuristics search over larger families of candidates and strategies.

Code evolution searches over programs that themselves search for solutions.

Once a capable agent controls the experimentation loop, even the decision about **which kind of search to try next** can enter the search space.

That is the vortex.

It is not "algorithms are dead." There are algorithms everywhere in this picture. The change is that the human is no longer forced to freeze the complete algorithmic architecture before the experiment begins.

We stop writing one solver and start creating conditions in which solvers can compete, mutate, combine and occasionally surprise us.

Or, in the language of Chapter 1: we let go of more of the path while keeping the evaluator hard.

The Contract

After running these experiments a few times, I ended up with a small contract for bounded problems where experimentation is cheap and evaluation is objective enough that the agent cannot charm its way around failure.

These are intentionally opinionated rules. They are not laws of software engineering. Production systems will violate all of them for good reasons. Inside this particular kind of search, though, they repeatedly proved useful.

Never Write Solution Code Yourself

This is harder than it sounds.

You watch the agent try something mediocre and immediately think of a better approach. You want to help, and sometimes you should. But every time I jump in with my own solution, the search becomes a little more like whatever happened to occur to me first.

For these experiments, I wanted independent directions badly enough that I had to resist becoming the senior engineer on every branch.

There is another reason. A long-running agent can become trapped inside its own context. Once twenty minutes of reasoning have accumulated around one architecture, every new observation gets interpreted through that architecture. Fresh sub-agents are useful partly because they have not spent twenty minutes explaining to themselves why yesterday's bad idea was brilliant.

So "never" is deliberately provocative. The deeper rule is: **don't accidentally collapse autonomous search back into your own search.**

You're the manager here. Spawn, evaluate, prune.

Keep the Harness Immutable

This one is much less negotiable.

If the agent can change the evaluator, the meaning of the experiment disappears very quickly. The circles overlap? Perhaps tiny overlaps should count. The score is low? Maybe the square should be 1.03 wide. Only twenty-five circles fit? Perhaps twenty-six was merely an aspirational requirement.

At that point we are no longer optimizing circle packing.

We are negotiating with the specification.

The **Immutable Harness** is the anchor of truth in an otherwise fluid process. The solver can change. The strategy can change. The tools can change. The agent can decide yesterday's entire approach was stupid and start again.

But the thing saying whether it worked stays harder to change than the thing being optimized.

This is Chapter 1's boundary made executable.

Cross-Pollinate Success

Independent search creates diversity. Perfect isolation wastes learning.

If one branch discovers a useful initialization and another finds a better local optimizer, future experiments should have some mechanism for inheriting both. That is what makes code evolution more interesting than asking the same model the same question one hundred times.

The original temptation is to broadcast every successful idea immediately. That can also be a mistake. If every lineage receives the current winner's strategy, diversity collapses and the entire population starts thinking in the accent of the first successful branch.

So **cross-pollinate success**, but not instantly and not universally. Let some lineages remain ignorant long enough to discover something genuinely different.

This is a recurring tension in search: information accelerates learning and destroys independence at the same time.

Prune Ruthlessly

Diversity is useful.

Preserving every bad idea forever is hoarding.

If a branch keeps underperforming and contributes nothing interesting, eventually it should die so compute and attention can move elsewhere. The difficult part is deciding when. Kill too early and you may discard an immature idea that needed another generation. Keep everything alive and you end up funding a large family of increasingly sophisticated failures.

"Prune ruthlessly" therefore does not mean "kill anything below the current winner." MAP-Elites already taught us why that is dangerous.

It means diversity needs a budget.

Search needs enough patience for novelty and enough cruelty for budget control.

Separate Discovery From Polish

Early in the search, I want large conceptual moves: a different geometry, solver, representation, decomposition.

Once a strong direction appears, the valuable work becomes smaller and more boring. Solver tolerances. Initialization details. Numerical settings. Tiny modifications that are pointless on a bad idea and extremely valuable on a good one.

Diagonal layering made this distinction obvious. Once the structural direction looked promising, continuing to invent entirely new geometries became less useful than squeezing performance from the geometry that was already working.

Discovery before polish.

Do not spend hours polishing a local optimum you should abandon.

And do not keep demanding revolution from a solution that has already found the right mountain and merely needs to climb it.

Zero Framework, With an Asterisk

I originally described this experiment as **zero framework**.

It's a great slogan.

It is also not really true.

I wrote almost no custom orchestration framework. That is very different from having no framework.

Claude Code is itself a substantial system. The underlying model has absorbed enormous amounts of software and problem-solving knowledge. Bash, Python, SciPy, Git and the operating system represent decades of accumulated engineering. The evaluator is custom machinery. Even the supposedly trivial act of running a program and inspecting a result depends on layers we have become so accustomed to that we stop seeing them.

The framework did not vanish.

It became somebody else's primitive.

That fits Chapter 1 almost suspiciously well. Once lower layers become reliable enough, we stop rebuilding them and treat them as building blocks. A tiny amount of code at the top can command enormous capability underneath because previous generations of complexity have already been compressed into tools.

So yes: **Zero Framework. Bash is enough.**

With the asterisk that Bash contains roughly half a century of civilization.

This is worth remembering whenever somebody shows you an agent implemented in one hundred lines of Python. The hundred lines may be perfectly real.

So is everything underneath them.

What Did We Actually Learn?

It would be very easy to overread this experiment.

We did not prove that coding agents can autonomously solve arbitrary research problems. We did not prove that AlphaEvolve-style systems are obsolete. We did not establish diagonal layering as a historically novel result in computational geometry. And we definitely did not prove that the right approach to production software is to give Claude a shell and go for a very long lunch.

What we had was narrower and, to me, more useful.

We had a **bounded problem** where evaluation was cheap and clear. We gave a capable coding agent substantial freedom and found that a surprisingly large fraction of the experimentation loop could happen without us directing every step.

The agent could propose an approach, implement it, run it, inspect the result, abandon it, create tools, borrow ideas from another direction and try again. My role moved away from writing the solver and toward defining the job, constructing the environment and defending the harness.

That is already a meaningful change.

It is also why circle packing is the easy version of autonomy.

The evaluator gives us one number. If version B beats version A, nobody needs to simulate a confused student, debate whether the interface feels intuitive or convene a committee to decide whether the new solution is spiritually aligned with the learning objectives.

The search can be complicated because **judgment is simple**.

Most things I want agents to build are not that generous. “Make a good educational demo.” “Write something people remember.” “Design a useful product.” “Explain this so somebody finally understands it.”

We can still let the agent generate alternatives, branch, cross-pollinate and search among them. But now the difficult part has moved again.

In circle packing, the harness tells the agent when it is wrong.

What happens when **the world no longer gives us one clean referee, and judgment itself has to be constructed**?

That is where the next chapter begins.

Chapter 3: Deep Mode

Beyond Algorithms: Agent Autonomy for Creative Problems

In the previous chapter, we gave agents a difficult algorithmic problem and a lot of autonomy. The result was surprisingly good. The agent researched strategies, tried several approaches, got stuck, changed direction, and eventually found diagonal layering.

But circle packing had one enormous advantage that I didn't appreciate enough at the beginning: we knew exactly what good meant.

There was an Immutable Harness. You ran a solution and got a number. Circles overlapped or they didn't; the score improved or it didn't. The agent could spend an hour pursuing some bizarre geometric idea and I didn't have to sit beside it wondering whether version seventeen had more soul. We just ran the evaluator.

Most of the things I actually want AI to help me with aren't like that.

"Is this explanation pedagogically effective?" doesn't have a unit test, and "Would a confused student understand this visualization?" can't be settled with an `assert`. Two competent people can look at the same design, disagree completely, then switch sides five minutes later after using it. The feedback is subjective, noisy, sometimes contradictory, and often becomes clearer only after you have built the thing you were supposedly trying to specify beforehand.

I picked educational demos for Merge Sort and Count-Min Sketch because they were still bounded—you can actually finish one before civilization collapses—but they live on the messier side of the boundary. You have to decide what to explain, what to leave out, how the interaction should work, how much should be visible at once, and what another person is likely to understand from any of it.

The ambition was intentionally high. I wanted something closer to the best Distill articles or Jay Alammar's visual explanations than to the usual "here are some bars moving around; congratulations, you have learned sorting." Those demos take a surprising amount of thought. The algorithm itself is usually the easy part. The difficult part is deciding what to show, when to show it, and what representation might make an idea suddenly click.

Circle packing let the search be complicated because judgment was simple.

Here judgment had become part of the problem.

The experiment that eventually became **Deep Mode** was an attempt to push autonomy one level higher. Instead of giving an agent freedom only over implementation, could we give a system some

freedom over the inquiry itself—whether the next useful move was to build, research, branch, retrieve, compare, change perspective, or abandon a direction entirely?

To understand why that seemed like the next step, it helps to see how much of the work around the model had already moved inside the machine.

How We Got Here

Coding first appeared as a strange side effect of language modeling.

Once large language models became good enough to produce useful code, the first tasks were conveniently small. Give the model a function signature, a comment, or a programming problem and ask it to fill in the implementation. Benchmarks such as HumanEval and APPS made this measurable: could a model turn a specification into a program that survived tests?

Then tools such as GitHub Copilot put that capability directly inside the editor. Instead of asking a chatbot for code and carrying the answer back yourself, you could write the beginning of a function or describe what should happen next and watch several lines appear underneath it.

This was useful enough that the limitations became interesting.

A real software task rarely arrives as an isolated function with a docstring politely explaining what needs to change. Someone says invoices occasionally show the wrong tax after a refund. Somewhere inside a 150,000-line CRM there is a reason. It may involve a controller, a database model, an old helper function, a test written three years ago, and an API whose behavior everyone on the team knows but nobody thought to document.

By the time GPT-4 arrived, I increasingly wanted to use models on exactly these problems. The workflow was ridiculous.

You found the file you suspected, copied a class or function into ChatGPT, described the bug and waited for a suggestion. Then you copied the patch into the editor, ran the program, discovered a new error, copied the traceback, pasted that back into ChatGPT, and started another round.

The first tool was copy and paste.

For a while, the whole architecture looked roughly like this:

```
repository → copy code → model → copy answer → editor → run tests → copy error →
model
```

The model might be doing sophisticated reasoning in the middle, but the human performed every interaction with the software around it. You searched the repository, decided which file mattered, assembled the context, applied the edit, ran the tests, and carried back whatever reality had said about the edit.

For a small function, this still felt magical. Then the bug crossed three files and you acquired a second job: preparing the model's world.

You paste one class but forget the interface it implements. The model confidently invents a method that does not exist. So you paste the interface. Now it needs the database schema. Then another class. Eventually half the repository is sitting in the conversation and somehow the model understands less.

A lot of early LLM programming consisted of building a tiny artificial universe around the model: here is the relevant class; here is the schema; please ignore these twelve methods; this function looks important but isn't; that innocent-looking helper controls payments, so please don't touch it unless you enjoy incident calls.

We eventually learned an obvious lesson that took surprisingly long to learn: more context and better context are different things. If somebody asks for a spoon, emptying the entire kitchen onto the table does not necessarily help.

Even with excellent context, though, I was still doing everything that connected the model's reasoning to the program.

I had effectively become its eyes, hands and terminal.

Software-engineering benchmarks began exposing the same gap. HumanEval and APPS had asked whether a model could write code once the problem was already packaged for it. SWE-bench changed the unit of evaluation. Its tasks came from real GitHub issues. Now a system had to work inside an existing repository, find the relevant code, understand relationships across files, make an appropriate change and survive the tests.

The early results were bad enough to be useful. A model could know perfectly well how to write Python and still fail because it looked in the wrong file, misunderstood the repository, changed the wrong abstraction, or never used the test result to reconsider its first theory.

The next engineering move was almost embarrassingly straightforward: move the copy-paste loop inside the machine.

Give the model access to the repository. Let it search for symbols and references instead of waiting for us to paste them. Let it open files, edit them and inspect the diff. Give it a terminal so it can run tests itself. When a test fails, return the failure and let that result shape what happens next.

The simplest coding agent is essentially this loop made executable. The language model supplies much of the programming knowledge and reasoning; the surrounding environment lets it inspect software, act on it and observe the consequences.

Software is an unusually friendly world for this. Files can be searched. Programs can be executed. Tests can say no. Git can tell you exactly what changed and, if an experiment becomes sufficiently exciting, take you back to the time before you had the idea.

Systems such as SWE-agent made the importance of the interface explicit. Apparently mundane choices—how the model searches, how much of a file it sees, how edits are applied, what information comes back from commands—can make a large difference. The useful object is no longer just the model. It is the model operating inside a world where software can push back.

Naturally, giving the model a computer exposed several new problems.

Early coding agents could behave like interns with root access and too much coffee. Ask one to change a line and it might rewrite half the file. Ask it to fix a button and twenty minutes later it has developed strong opinions about the database architecture. It would find one plausible explanation for a bug, follow it for too long, then use every new piece of evidence to improve the explanation instead of admitting the explanation was wrong.

So the interface became more careful. Agents got better ways to make small patches, inspect diffs, run targeted tests, checkpoint work and undo experiments. Planning helped when a task was too large to attack in one edit.

Then we discovered that the agent kept forgetting things we had already taught it.

Every real repository contains knowledge that is obvious to the people who work there and invisible to a general model. Authentication happens in a particular way. The team has conventions. One ancient API looks wrong but absolutely must remain wrong because six other services depend on its wrongness.

You explain this on Monday, then again on Tuesday, and by Wednesday you begin to suspect that you are the memory module.

So some of that knowledge moved into the environment too: `CLAUDE.md`, `AGENTS.md`, repository instructions, rules files, skills. The names vary, but the idea is simple. If somebody has already learned something expensive about this codebase, leave it somewhere the next agent can find it.

Longer sessions produced almost the opposite problem. Context slowly filled with abandoned experiments, obsolete assumptions, test output from forty minutes ago and debugging paths that stopped mattering three hypotheses back. Technically the model had more information. Practically it began behaving as if somebody had emptied a filing cabinet onto its desk and mentioned that one of the pages might contain the launch codes.

Memory became a problem of selection rather than storage.

And then there was history.

A long-running agent tends to acquire loyalty to its own decisions. Suppose it decides early that our Merge Sort demo should use React and a recursion tree. It spends forty minutes building that version. Every later question now arrives in a context containing forty minutes of reasons, code and decisions supporting React and a recursion tree.

Humans call our version of this sunk cost.

The agent has a respectable excuse: its context window is literally full of evidence that this is what the project is.

So we started giving different attempts different histories. One agent tries the tree. Another begins with the array. Another starts from the learner's misconception rather than from either representation. A fresh branch does not have to spend half its intelligence escaping the assumptions accumulated by the previous one.

Looking backward, the progression is less mysterious than the word *agent* sometimes makes it sound. We began with models that could generate useful pieces of code. We put them in editors. Then repository access, editing and execution moved into the system itself. Better interfaces, memory, context management and branching followed.

Bit by bit, work the human had been doing around the model became part of the machine.

But there was still a large difference between an agent that could work competently on software and the thing I increasingly wanted to ask for:

Build the application.

From the Repository to the App

Once coding agents became reasonably comfortable inside repositories, I started noticing something slightly absurd about how we were using them.

Suppose I want to build a small booking application for a football academy. Parents should see available sessions, pick one, enter their details, perhaps pay, and receive a confirmation. There is nothing technically exotic here. A competent coding agent can build the whole thing.

Give it an empty directory, though, and watch what happens.

First it creates a project and chooses a framework. Then it installs packages, lays out the frontend, creates a database, decides how authentication should work, wires the database to the application, manages environment variables, configures hosting, adds deployment files, and eventually discovers that two perfectly respectable libraries disagree about some dependency for reasons documented across fourteen GitHub comments and one furious blog post.

Several minutes later, we have made enormous progress toward having somewhere to put the booking form.

This is real engineering work. Somebody has to do it. But once an agent can reconstruct roughly the same plumbing every time, another question becomes obvious: why are we asking it to?

Authentication is not identical across products, but most small applications do not need a new philosophy of authentication. The same is true of databases, routing, components, deployment, secrets, build systems and dozens of other choices sitting underneath the thing somebody actually wanted.

The booking application does not become more original because the agent spent twenty minutes reconsidering whether PostgreSQL has finally had its day.

Sometimes the right answer is simply to prepare more of the world in advance.

This is why systems such as Replit and Lovable became interesting to me. They give the model a more opinionated place to work. Runtime, deployment and common application machinery are already nearby. Instead of beginning with an empty computer and reconstructing web development, the conversation can begin much closer to the application.

You can say:

Build a booking system for a football academy. Parents should see available sessions and book one.

A first version appears. You look at it and realize that the schedule should probably be on the home page. Requiring people to create an account before they can even see a session feels ridiculous, so login moves later. The mobile page is cramped. The photo of the suspiciously muscular child kicking a football may be setting unrealistic expectations for the under-sevens, so perhaps that goes too.

Somewhere underneath the page there is still React or something like it. There is still a database, network requests, authentication, configuration, hosting, CSS and the usual collection of tiny disasters waiting patiently for production. But those things are no longer the subject of every conversation.

The environment has opinions about them already.

I think of this as a smart template, although *template* makes it sound more rigid than I mean. An old template gave you a restaurant website with a hero image and somewhere to replace somebody else's logo. A smart template is more like a prepared workshop. Common tools are already on the wall and common jobs have sensible defaults, but the agent can still change them when the job requires it.

There is an obvious trade-off. Give an agent Bash and an empty machine and almost nothing has been decided for it. Put the same agent inside a prepared application environment and some possibilities disappear.

That sounds like a loss of intelligence until you think about how humans work.

A chef does not begin dinner by manufacturing a knife. A scientist does not usually build a new operating system before analyzing data. When I open Python, I accept an astonishing number of decisions made by people I will never meet because reconsidering all of them would make writing `print("hello")` a multigenerational project.

Useful abstractions remove decisions whose answers are no longer interesting most of the time.

And after enough of those decisions disappear, something else becomes easier to see.

Suppose the booking app works perfectly. The database is connected, deployment succeeds, the buttons behave, the mobile layout is respectable, and nobody has accidentally built a cryptocurrency exchange in the authentication service.

I open the application and think: this isn't very good.

The software works.

Now I have to worry about the football academy.

The Problem-Solving Layer

Should parents see every available session, or only the ones appropriate for their child? Should they have to create an account before booking? What happens when somebody has three children? How late can they cancel? Should we show that only two places remain, or does that create unnecessary pressure? If the academy has empty sessions on Wednesday and a waiting list on Saturday, is the booking interface part of that problem?

None of these questions is really about React.

They were always there. Software teams have always argued about customers, flows, business rules and what the product should do. But implementation used to consume enough attention that it was easy to mix two different problems together: deciding what should exist and turning that decision into software.

When another version becomes cheap, the balance changes. You can see the idea sooner, and seeing it gives you information you did not have while discussing it.

Perhaps we decide that customers should create an account before seeing availability. It sounds reasonable: we need their details eventually. Then we build it and the experience immediately feels

annoying. Parents arriving from a Google search do not want to establish a lifelong digital relationship with a football academy before discovering whether Saturday at ten is available.

So login moves later.

The artifact is no longer merely the end of the thinking process.

It becomes something we think with.

The Merge Sort demo made this even clearer because there was almost no business machinery to hide behind. I could ask an agent to build an interactive explanation and receive something perfectly functional: an array of bars, controls, animation, perhaps some text explaining that the algorithm divides the input and merges the pieces again.

Technically, it was fine. Pedagogically, it could still be terrible.

Watching bars move does not necessarily tell a beginner why dividing the problem helps. So perhaps we try a recursion tree. The tree makes the structure visible, but now the supposedly simple sorting algorithm resembles the organizational chart of a German corporation. Maybe we show the tree and array together. Perhaps that creates too much cognitive load. Maybe the problem is not the representation at all; the learner understands splitting perfectly well but has no idea why merging makes the whole trick useful.

There is no compiler error that tells me which diagnosis is right.

I have to look at what we built, form an opinion about why it fails, and decide what would teach us something next. Sometimes that means improving the current version. Sometimes it means building a deliberately different one. Sometimes I need research. Sometimes the right move is to put the application in front of somebody who does not already understand Merge Sort.

Occasionally I discover that the question I started with was wrong.

"Build an interactive Merge Sort demo" sounds like a goal until you see several interactive Merge Sort demos. Perhaps what I actually care about is getting someone who has never encountered divide-and-conquer to understand why breaking one difficult problem into smaller ones makes the problem easier. Once I realize that, interactivity is merely one possible means.

This is the **problem-solving layer**: the work of deciding what to try, which evidence matters, whether a result failed because of its implementation or its underlying idea, and what kind of attempt might teach us something next.

The lower layers help us build the artifact.

Layer 3 decides what to try next in service of its purpose.

The Five Layers of AI Coding

By now I had a rough map.

Layer 0 — Model. GPT, Claude, Gemini and whatever comes next: general capability in language, code, reasoning and vision.

Layer 1 — Agent. Put the model in an environment where it can act. Claude Code, Codex and similar systems search repositories, edit files, execute commands and react to results.

Layer 2 — Application. Replit, Lovable and similar environments prepare more of the software world in advance, allowing the conversation to stay closer to the application.

Layer 3 — Problem Solving. Decide what to try, why something failed, which evidence matters, and whether the current direction deserves another iteration.

And above that sits the problem I have mostly been avoiding.

Layer 4 — Intention. What do we actually want?

Software likes that question to have been answered before the work begins, preferably in Jira, where the answer can remain wrong in a structured and searchable format. Real goals are less cooperative. Seeing a solution can change what I realize I wanted.

That problem is much bigger than AI coding, and I am going to leave it alone for now.

The borders are fuzzy. Figma can generate code. Coding agents make product decisions. Tomorrow's products will rearrange the stack again. The useful distinction is the kind of decision being made, not which company happens to occupy which box.

People often call the experience of working this way *vibe coding*. I will use **AI coding** for the broader stack, but *vibe coder* remains a wonderfully accurate name for the human sitting near Layer 3: looking at what came back, deciding what feels wrong, asking for another direction, killing one idea, keeping part of another, and steering the whole process without having an algorithm for how.

The first three layers increasingly answer a version of the same question: *how do we make this?*

Layer 3 asks a different one:

Given everything we have learned so far, what should we try next?

That was the part I still seemed to be doing manually. So I watched what I was actually doing in that seat.

There was no universal workflow hiding there. A mathematician, a designer and a product manager can all spend a day solving hard problems while performing almost none of the same visible actions.

But the same kinds of moves kept appearing.

In the Vibe Coder's Seat

I began to think of these less as stages in a workflow than as a vocabulary.

Sometimes I needed another attempt. Sometimes I needed information. Sometimes the search had become too narrow. Sometimes the representation itself was constraining what we could imagine. Sometimes the objective needed to change. Sometimes I needed to see the artifact from another mind.

The important point was that these moves were not useful in a fixed order.

Beat the Complexity Wall — Code Evolution

The first problem was simply the number of possibilities.

Even a Merge Sort demo has an absurd design space. Bars or cards? Numbers or a tree? Continuous animation or learner-controlled steps? Does color represent recursion depth, identity, or the active subproblem? Explain before the animation, during it, or afterward? Every choice changes the usefulness of several others.

When implementation was expensive, we dealt with much of this complexity by trying to decide more before building. AI coding changes the economics. If another implementation costs minutes rather than days, I do not have to choose quite so much in advance.

Chapter 2 had already shown what to do with a search space too large to reason through directly. One hill climber inherits its own history; evolutionary search maintains alternatives.

The same move works here, except that what evolves is no longer just a vector of parameters or even an algorithm.

It can be an **idea embodied in software**.

One builder tries a recursion tree. Another focuses on the array. A third begins from the learner's misconception. Mutations can be conceptual: remove the text, teach backward, make the learner predict, show synchronized representations, abandon interaction altogether.

Useful pieces can move between them. One terrible demo may have a beautiful color mapping. Another may explain the merge clearly while making everything else unbearable. The final artifact does not have to inherit the entire history of either one.

The only thing I have to protect is enough independence for the alternatives to become genuinely different. If every branch sees the current winner and its full history, parallelism quickly collapses into several agents improving the same idea. Sometimes I withhold the leading implementation; sometimes I frame branches differently and let them develop before sharing what worked.

There is a cost. Diversity burns compute, duplicates effort and sometimes produces five independent rediscoveries of the same bad idea. Parallelism is not automatically intelligence.

Still, cheap code changes where that trade-off sits. Competing ideas can become executable before commitment makes one of them expensive to abandon.

The waste appears elsewhere: if every branch starts ignorant, we spend a great deal of compute rediscovering things other people already learned.

Survey the Territory — AI Research

People have been teaching recursion for decades. There are textbooks, lecture notes, visualizations, papers, classroom experiments and a great deal of trial and error sitting on the internet.

Before I spend another afternoon inventing my fourth way of moving colored rectangles around, I probably want to know what is already there.

This is where AI research became useful.

I can ask an agent a broad question: How do people teach Merge Sort well? Where do learners get confused? Which visual approaches have been tried? What evidence exists? Where do approaches disagree?

Increasingly the agent can do more than read. If an interesting prior attempt is an interactive tutorial, a written description is a poor substitute. With computer use it can open the artifact, step through it, see what remains visible, notice who controls the pace, and compare it with another implementation. Some design knowledge lives more clearly in the artifact than in what its creator wrote about it.

Research can also follow the work rather than precede it.

Suppose I build a recursion tree and discover that recursion itself is not the problem; the learner is losing the relationship between the tree and the changing array. Now I have a better research question: how have other systems coordinated two representations without requiring people to watch half the screen at once?

Research becomes another move in the investigation rather than a phase performed before building.

But research has its own failure mode: anchoring.

Give every builder the same ten polished Merge Sort demos and version eleven tends to resemble its grandparents. Sometimes I want the researcher to bring back the problem, the evidence and the failures while leaving a few fresh branches to invent their own response.

There is no prize for independently rediscovering a good idea. There is also no virtue in making every search begin with the same answer.

Research gives us more material. Soon the problem reverses: there is too much of it.

Find the Right Context — Retrieval

Suppose the branch I am working on now keeps losing the connection between the recursion tree and the array.

Somewhere in our growing mess we have research notes, screenshots, evaluator comments, old branches, perhaps a paper on coordinated representations, and a discarded prototype whose only good idea was a color mapping that solved exactly this problem.

I do not need the whole research archive.

I need the thing that helps with this decision.

Sometimes ordinary search is the best tool. If I remember the phrase *coordinated representations*, an author, an API, or an evaluator comment containing "lost track," exact words are useful evidence rather than a primitive technology we should be embarrassed to use.

Embeddings help when I remember the idea rather than the words. "Keeping identity stable while the representation changes" may retrieve a note that never used that phrase, or an example from another domain that solved the same structural problem.

Long documents create another choice. Cutting everything into chunks and retrieving by embedding similarity works remarkably well, but a book or research report already has structure. If I know the question concerns evaluation, it can be more sensible to navigate chapters, sections and pages, then

read the relevant passage in context. Page- or structure-based retrieval preserves relationships that disappear when every paragraph is treated as an independent fragment.

In practice I want all of these. Search broadly, rerank candidates when necessary, open the surrounding section, follow a reference, then search again if what I found changes the question. Good coding agents already work this way inside repositories. They do not retrieve the repository once; they navigate it.

At Layer 3 the environment is stranger because the retrievable object may be a screenshot, an old interaction, a research result, evaluator feedback, code, or a dead branch that suddenly contains something useful.

That last case matters after code evolution. Killing a lineage does not have to erase it. A branch that lost globally may still contain a stepping stone worth returning to later.

Retrieval therefore does more than find semantically similar things. It constructs the local working context of an investigation.

And context is not merely what we include.

A branch polishing the current winner should probably inherit a great deal. A branch sent out to challenge it may not need to begin by studying the winner in detail.

Sometimes the useful context has a hole in it on purpose.

That helps preserve alternatives. It does not yet stop those alternatives from drifting toward the same familiar region anyway.

Push the Creative Horizon — Exploration

After several rounds of code evolution, something slightly embarrassing happened.

The builders were exploring.

They also kept giving me bars.

Better bars, admittedly. Some split gracefully. Some changed color as recursion deepened. One branch added a tree. Another let the learner control the animation. Given enough generations, I had every reason to believe we would eventually produce the finest moving bars known to humanity.

It is tempting to treat creativity as the mysterious ingredient missing here. Reinforcement learning and evolutionary computation suggest a more practical diagnosis: search tends to spend its budget where progress is already visible.

Different methods attack different versions of that failure.

Monte Carlo Tree Search is useful when the future branches too quickly to explore everything. In Go, learned intuition can focus effort on promising moves without committing every resource to the move that currently looks best. AlphaGo made that combination famous.

But sometimes the problem is not choosing among branches; it is losing interesting places you already reached. Go-Explore addressed that failure by remembering promising states, returning to them, and exploring outward again.

That maps nicely onto software artifacts. A globally mediocre Merge Sort branch may contain one excellent interaction. Keep the state. A dead lineage can remain an archive of places worth revisiting when another idea gives them a new use.

Novelty search pushes against a different failure: the objective itself can pull every attempt toward the same deceptive region. Lehman and Stanley showed that on some problems, rewarding behavior for being different rather than for direct progress can eventually discover better solutions.

The important word is *some*.

Novelty is not quality. A sufficiently determined search can become wonderfully original and completely useless. But difference itself can sometimes deserve a budget.

MAP-Elites makes that intuition more structured. Instead of preserving only the global winner, retain strong solutions across different behavioral niches. At Layer 3, one niche might be highly interactive, another almost static, another organized around learner prediction. We do not have to know in advance which style contains the final answer in order to preserve several ways of being good.

POET makes one further move. If the environment keeps rewarding the same behavior, evolve the environment too.

For a creative problem, that can be surprisingly literal.

Teach Merge Sort without explanatory text. Require the learner to predict before anything moves. Make it work on a phone with room for only one representation. Design it for somebody who understands loops but finds recursion suspicious.

I had already started calling these **Strategic Constraints**. The exploration literature gives a less mystical explanation for why they sometimes work: a constraint changes which parts of the search are reachable and can stop a familiar attractor from absorbing every attempt.

"No bars" is not a profound theory of creativity.

It is an intervention on the search.

Most arbitrary constraints are merely arbitrary. The useful ones expose a neglected dimension, force a different representation, or remove an easy path that has become too dominant.

What makes AI coding particularly interesting is the richness of the available moves. In Go you place a stone. Here a move might be a code change, a different metaphor, a retrieved analogy, a fresh agent with no history, another evaluator, or a reformulation of the problem itself.

We already know a surprising amount about keeping search alive.

Language-model agents give those old ideas a much stranger space in which to operate.

And then I noticed that even this search was biased in a quieter way.

Most of our ideas still had to arrive as words.

Think in Pictures — Visual Thinking

That is fine when I am working on an argument. It is less obviously sensible when I am designing an interface.

I can spend ten minutes explaining where the recursion tree should sit, what remains visible while the array splits, how colors should connect two representations and what the learner should notice first.

Then somebody draws it and I know within three seconds that the whole thing is terrible.

So I started generating the picture first.

The experiment was not sophisticated. I asked an image model to design an interactive tutorial for Merge Sort. Then Count-Min Sketch. Then A*. Then Poincaré embeddings in hyperbolic space, partly because if this still worked there I would have to take the idea seriously.

The details were not magically correct. Arrows occasionally pointed somewhere they had no business pointing, interactions made no computational sense, and generated text sometimes looked like somebody had tried to OCR a dream.

But the overall composition could be surprisingly thoughtful.

A Merge Sort mockup might keep the array visible while placing the recursion tree beside it, using color to preserve the relationship between a subarray and its node. A Count-Min Sketch design might make collisions visually central instead of leaving them as a detail in an equation. The model had to decide what was large, what was peripheral, where controls belonged and how the learner might move through the explanation.

I remember looking at some of these and thinking:

Holy shit.

Not because I wanted to ship the image. Usually I didn't.

What surprised me was that I had given the model a concept in language and it had returned something like a spatial argument about how the concept might be taught.

After that I stopped treating image generation as the last stage—*the product is designed, now make it pretty*—and started using it while I was still trying to understand what the product could be.

A mockup is a cheap hypothesis.

Often most of it is disposable and one relationship is worth stealing.

Coding agents make this particularly useful. I can say: preserve this layout, keep the relationship between these two states, lose the giant header, and make the mathematics actually work. The coding agent turns the visual intuition into something executable, where the recursion tree cannot invent an extra branch because the composition looked nicer that way.

Figma, Claude Design and similar tools make the intermediate artifact richer still. We no longer need to jump straight from prose to code.

Pictures can hide problems too. A mockup may look wonderful because nobody has yet forced the interaction to make sense. Then we implement it and discover that the design depended on three impossible state transitions and a button whose purpose was apparently emotional.

Different representations expose different mistakes.

I do not need the stronger claim that the image model "understands pedagogy." The practical point is enough: changing the representation changes what the search can discover.

And now we have a nice problem.

We can generate several genuinely different demos rather than five implementations of the first thing that occurred to us.

I still have to decide which one is better.

Optimizing Something You Can't Score

Circle packing was unusually kind to us. Once the geometry was valid, the evaluator could reduce the result to one number.

That number threw almost everything else away, which was precisely why it was useful.

A huge amount of machine learning rests on this trick. We take something complicated that we want and find a measurable signal that stands in for it. Reinforcement learning makes the relationship especially obvious: we do not specify every movement a robot should make while learning to walk; we construct a reward and let search discover the behavior.

The reward is doing an extraordinary amount of work. It is also where we hide an extraordinary amount of trouble.

Suppose I want the same convenience for educational design. I can make a rubric: correctness, pedagogical clarity, visual quality, interaction, accessibility, engagement. Give each a weight and suddenly my vague dissatisfaction with a demo has become a respectable decimal.

The decimal is comforting. The decisions required to produce it are less so.

Why should interaction receive fifteen percent? Is more interaction always better? What distinguishes a seven from an eight in pedagogy? Why those dimensions rather than whether the learner can predict what happens next or explain why the merge matters?

The metric forces me to commit to an idea of "good" before the search has taught me very much about the problem.

This is not an argument against metrics. If I care about latency, measure latency. If the code must pass a test, run the test. Hard measurements are wonderful when what we can measure is close to what we care about.

The trouble begins when a rich objective is still poorly understood and we compress it anyway because optimization wants a number.

That compression is also a surprisingly low-bandwidth way to communicate.

"Version B scored 7.4; version A scored 7.1" tells the next builder almost nothing about why B won. A rubric helps, but as I add enough dimensions, exceptions and qualifications to faithfully express what I mean, eventually I reinvent language badly.

Meanwhile I can simply say:

The recursion tree makes decomposition much clearer, but now the learner has to watch the tree and the array simultaneously. Keep the color mapping that preserves identity between them, simplify the tree, and make the merge feel like the payoff rather than cleanup at the end.

That contains comparison, diagnosis, trade-offs, priorities and a proposed next move in a few sentences.

Natural language is ridiculously rich compared with a scalar.

Language models make that human communication channel available inside the optimization loop. The model already carries learned structure behind words such as *simple*, *confusing*, *elegant*, *intuitive*, *busy* and *beginner-friendly*. Those meanings are imperfect, culturally loaded and sometimes wrong. But they carry much more structure than 7.4.

Natural language can therefore function as an **implicit metric**.

Not a metric in the strict mathematical sense. There is no guarantee that "intuitive" defines a stable ordering, and two evaluators may interpret it differently. But language can play some of the role a metric normally plays: it gives the search a direction, communicates why one attempt is preferred to another, and preserves trade-offs that a scalar would erase.

OPRO—Optimization by PROmpting—is interesting for a related reason.

In ordinary optimization, specifying the objective is only half the machinery; we also need some method for moving through the search space. In OPRO, an LLM sees the optimization problem, previous candidates and their outcomes, and proposes a better candidate. Much of the search heuristic is implicit in the model rather than programmed as a transformation rule.

That does not make OPRO the same thing as creative design. In the published OPRO setting, candidate quality can still be evaluated by an explicit score. But it demonstrates something useful: an LLM can use the history of an optimization process to propose the next point without us writing the search rule ourselves.

Now suppose the history contains more than scores.

Alongside hard measurements, I can tell the model what improved, what became worse, which trade-off appeared and what we should preserve. The history of the search can retain some of its meaning rather than collapsing into a column of numbers.

This begins to feel a little like reinforcement learning turned upside down.

I mean the inversion conceptually, not as a claim that these are the same algorithm. Decision Transformers, reinforcement learning and language-guided iteration are different mechanisms.

But look at the direction of specification.

The usual reinforcement-learning picture asks us to define a reward and then discover behavior that earns it.

Here I can begin with something much less respectable:

Make this explanation less intimidating.

Help the learner understand why the merge matters.

I want someone to feel why divide-and-conquer helps rather than merely watch the algorithm execute.

Those are descriptions of a direction, not reward functions.

Yet the model can produce an attempt from them, and the attempt can teach me whether the direction was what I really wanted.

I began the project insisting on an *interactive* Merge Sort demo. Interactivity sounded obviously desirable. Then I saw versions with buttons, sliders and enough learner participation to qualify as a small democracy, while one quieter version explained the central idea much better.

Apparently clicking things was never the objective.

Later the demos became good at showing recursive splitting and I realized they were treating merging almost as cleanup.

The objective moved again.

The search was doing something I normally associate with optimization in reverse: instead of starting from a fully specified reward and discovering the policy, I was using candidate policies—actual artifacts—to discover what the reward description should have been.

Recognition arrives long before specification in a lot of creative work. We know a terrible design when we see one before we can write the complete theory of what would make it good.

AI makes that loop cheap. The natural-language objective guides the search; artifacts make the objective concrete enough to argue with; the description changes and the search continues.

Ambiguity is not always a defect waiting to be engineered away. Sometimes we simply have not learned enough yet.

There is a catch, though.

Natural language may be high bandwidth, but somebody still has to interpret it.

If I say "make this intuitive for a beginner," I have left almost everything interesting unstated.

Which beginner?

Borrow a Mind — Theory of Mind

When I look at a Merge Sort demo, I am hopefully not testing whether *I* understand Merge Sort.

The difficulty is seeing it from the position of somebody who does not know what I know.

Expertise makes this harder. Once recursion has settled into your head, you forget how strange it once looked that a function could call itself. Even the vocabulary stops sounding technical.

Good teachers develop an instinct for this. They know where people usually stumble and which innocent sentence assumes three things the learner has not yet learned.

I do not have that instinct for every person or every subject, so I started borrowing another mind.

For one of the early demos, I asked Claude to approach the application as somebody who understood arrays and loops but had never encountered recursion. Not simply "act like a beginner," which tends to produce a theatrical beginner who is mysteriously confused by everything.

I gave it a knowledge boundary.

Its reaction was roughly:

I can see that the array keeps getting divided into smaller pieces, but I don't understand why that helps. It feels as though we're making the problem more complicated. Where is the payoff?

That was useful because the demo really did have that problem.

We had made recursion visible. From my position, that looked like progress. From the learner's imagined position, we had merely made a mysterious operation easier to watch.

Cognitive scientists use **Theory of Mind** for our ability to reason about mental states other than our own: what somebody knows, believes, wants or misunderstands. The other person may not simply know less. They may have a different model of what is happening.

Instead of:

You are a beginner.

I can specify the state of the mind I want to borrow:

You understand arrays, loops and functions. You have never encountered recursion. Use the demo from the beginning and tell me where the explanation first requires an idea you do not yet have.

Or:

You understand recursion but have never seen Merge Sort. Tell me when you first understand why dividing the array makes sorting easier.

Those are different evaluators because they are positioned to notice different things.

The same idea applies outside education. A customer may know exactly what jacket they want without knowing the vocabulary our catalog uses. A developer can be excellent at distributed systems and know nothing about the peculiar assumptions buried in our deployment process. A reader can have followed this book perfectly well without having lived inside its conceptual structure for months.

This is cheap perspective-taking.

It is also very easy to fool yourself with.

The confused student is not confused.

Claude has not spent twenty minutes failing to understand recursion while everybody else in the classroom moves ahead. It is generating a plausible model of how such a person might react.

That model can expose a blind spot. It is not synthetic user research.

I treat these borrowed minds as instruments for generating different criticisms and hypotheses, not as substitutes for the people they simulate.

By this point the system could generate alternatives, bring in outside knowledge, retrieve old ideas, reopen dead branches, force the search into unfamiliar regions, change representation, revise the objective, and inspect the artifact from different points of view.

That solved one problem.

We could now generate genuinely different possibilities.

It made another problem worse.

Once several plausible artifacts and several plausible perspectives exist at the same time, somebody still has to decide what survives.

Who Judges the Judges?

At some point generating another opinion stops helping. Some artifacts have to survive and others have to die.

The metric problem from the previous section comes back here in a more dangerous form. A rubric can make judgment explicit, which is useful. It can also become the target the builder learns to satisfy.

If the evaluator repeatedly rewards step-by-step explanation, explanations grow. If it likes polished onboarding, everything begins to look like onboarding. If familiar visual conventions read as "clear," unusual approaches may disappear before they have time to become good.

OpenAI's CoastRunners experiment is the cartoon version of this failure: an RL agent discovered that driving in a loop and repeatedly collecting reward targets produced a higher score than finishing the boat race.

Goodhart's Law with a speedboat.

A language-model builder does not need such an obvious loophole. It can learn the style of artifact that another language model tends to reward.

Making the evaluator more elaborate does not automatically solve the problem. Sometimes it merely creates a more elaborate thing to game.

Comparison Is Often Easier Than Scoring

There was another reason I became less enthusiastic about absolute scores: I wasn't particularly good at producing them either.

I can drink a coffee and have almost no meaningful answer to "How good is this on a scale from one to ten?"

Give me two cups and ask which one I prefer, and the problem becomes easier.

If I still cannot decide, the scientifically responsible procedure is presumably to finish both.

The same thing happened with the demos.

"Give this interface a pedagogical score from 1 to 10" produced suspiciously precise numbers attached to explanations of why the number should not be taken too seriously.

Showing two artifacts and asking:

Which one would you rather give to somebody encountering Merge Sort for the first time, and why?

worked much better.

Relative judgment asks less of the evaluator. It does not need a stable internal unit called one pedagogy point.

If we have many candidates, techniques such as Bradley–Terry can infer an ordering from a subset of pairwise preferences. Better still, the explanation for each preference can survive alongside the ranking and become input to the next generation.

Pairwise comparison removes much of the fake precision of absolute scoring.

It does not repair a biased judge.

Bradley–Terry can aggregate preferences. It cannot make those preferences true.

One Judge Is Still One Judge

So I stopped asking one evaluator to represent everybody.

A learner can inspect the artifact from the position we developed above. A teacher can focus on explanatory sequence. Another evaluator cares about cognitive load, another about interaction or accessibility. A domain expert can ensure that our elegant simplification has not become false.

I call these **Independent Evaluators**, though the important word is *independent*.

Five copies of the same model given the same context and asked to wear five hats may still share almost every important blind spot. If all of them read the leading builder's explanation of why its design is brilliant before inspecting the artifact, disagreement becomes less likely for reasons that have little to do with brilliance.

Sometimes I want the judges to see different things.

The beginner should use the artifact before reading the builder's explanation. A critic looking for conceptual errors does not need three paragraphs explaining why the choice was clever. The usability evaluator does not necessarily need to know which branch is currently winning.

This is the **Isolation Principle**: preserve enough separation that independent pressure remains informative.

There is a difference between telling the builder:

Learners repeatedly lost track of which subarray corresponded to which branch of the tree.

and telling it:

The evaluator awards two extra points when each tree node has the same color as its corresponding subarray.

The first communicates the problem.

The second communicates the test.

Isolation does not magically remove shared bias. Two supposedly independent evaluators may still inherit the same assumptions from their training, their culture or the examples we give them.

But without isolation, we sometimes destroy even the independence we could have had.

Calibrate the Judges

Independence creates another issue: everyone's idea of "excellent" can drift.

References help.

For these demos I might give evaluators examples from Distill, 3Blue1Brown or Jay Alammar. The goal is not imitation. The examples provide scale.

"This is clear" means something different if the evaluator has seen only the last four generations of our own work.

Calibration matters especially when judgment remains in natural language, because *excellent*, *clear* and *polished* have no fixed unit.

References introduce their own anchor, of course. Calibrate too strongly against one aesthetic and every road leads to Distill.

So the reference is there to answer *how good?*, not *what should this become?*

Let the Judge Use the Thing

Another early mistake was letting evaluators judge applications by reading them.

Open the demo.

A browser agent can click through it, resize the page, try controls in the wrong order, notice that an explanation appears after the moment when it would have helped, or discover that the beautiful button everybody admired does absolutely nothing.

I used to call the browser ground truth.

That was too generous.

The browser gives the evaluator contact with the artifact rather than a description of it. It can establish that an interaction works and observe what is visible at each point in the experience.

It cannot establish that a human learned Merge Sort.

A simulated beginner saying the explanation is understandable gives us a hypothesis. Several evaluators preferring one design gives us comparative evidence.

Neither substitutes for putting the artifact in front of actual learners.

The danger in a fully automated loop is that simulated evidence quietly replaces the expensive kind. Everything inside the machine agrees, the browser works, the ranking improves, and the loop congratulates itself.

The student has not yet been asked.

The Evaluator Became an Institution

At some point I looked at what we had assembled and realized that *evaluator* no longer described it particularly well.

Builders proposed alternatives. Different judges approached them with different concerns. Some information was deliberately kept separate. Pairwise comparison helped decide which directions deserved more work. References calibrated the judges. Browser agents interacted with the artifact. Hard

tests handled the parts that really were hard facts. Real-user evidence could eventually enter where simulation stopped being enough.

This looked less like a loss function and more like a tiny institution.

Not a good institution automatically. Institutions can amplify conformity, entrench bad assumptions and become spectacularly efficient at measuring the wrong thing.

But the shape of the problem had changed.

Humans face the same difficulty. One person's judgment is useful and fallible. So we compare work, preserve disagreement, create standards, ask specialists to inspect different aspects, reproduce results, and occasionally discover that an entire professional community has become extremely sophisticated about the wrong thing.

Apparently, when the clean loss function disappears, you eventually reinvent peer review.

And that made the remaining human job painfully obvious.

I still decided when to research, when to build, which branches stayed isolated, whether a strange direction deserved another generation, which disagreement mattered, when to retrieve another example, and when the simulations had reached the point where only a real person could answer the question.

I had automated much of the work.

I was still running the inquiry.

The individual techniques were not the missing piece anymore.

The missing piece was the decision over **which technique the inquiry needed next**.

Going Deeper

This was the part I wanted to test next.

By now the system had a respectable collection of moves. It could spawn independent builders, research previous work, retrieve context, preserve odd stepping stones, impose constraints, generate visual directions, compare artifacts, borrow different perspectives and interact with what had been built.

But there was no reason every problem should use those moves in the same order.

Research first may be sensible for one task and destructive for another because it anchors every branch before anything original appears. Five builders may reveal useful diversity, or reproduce one mistake five times. Evaluator disagreement may justify another experiment, or one evaluator may simply be confused.

A fixed Planner → Builder → Critic → Revise loop can be useful.

It also answers all of those questions in advance.

What I wanted to know was whether some of the workflow could remain part of the search.

That experiment became **Deep Mode**.

We gave an orchestrator the problem, the capabilities available to it, and enough of the search history to decide what kind of move made sense next. Builders still built. Researchers researched. Evaluators judged. Browser agents used the artifacts. Visual systems explored designs. Retrieval brought back prior work and old experiments.

The orchestrator's job was not to perform all of those roles.

It was to decide which role the inquiry currently needed.

Sometimes the useful move was another independent branch. Sometimes it was research on a question exposed by the last prototype. Sometimes a visual direction deserved implementation. Sometimes two branches should exchange an idea. Sometimes the system needed another judge; sometimes it had enough judgment and needed an actual user.

At the highest level the loop was almost embarrassingly simple:

state of inquiry → choose a move → act → observe what happened → update the state of inquiry

The interesting part was that the move itself was not fixed.

That distinction mattered to me. Otherwise Deep Mode would simply be a larger workflow diagram containing more rectangles.

The architecture did not give us a universal problem-solving procedure.

It gave the orchestrator a vocabulary of possible moves and allowed the history of the inquiry to influence which one came next.

The experiment was whether the process deciding how to solve the problem could itself become more adaptive.

What Emerged

The first Merge Sort demos were exactly what you would expect.

Bars moved around. Numbers changed places. Everything sorted correctly.

If you already understood Merge Sort, you could follow them. If you did not, they mostly provided animated evidence that a computer was performing an algorithm.

There was no single diagonal-layering moment here, and I do not want to manufacture one for the sake of the story.

The progress was more distributed.

Different branches exposed different weaknesses in our current idea of the demo.

Tree-like representations made recursion visible but could make a simple algorithm look forbidding. Keeping the array visible helped connect the decomposition back to the data, while also creating another place for the learner's attention to go. Color could preserve identity between representations until too much color became another representation to decode. Some versions explained every step so

carefully that the explanation became harder to follow than Merge Sort. Others became beautifully minimal and stopped teaching anything.

The useful pieces did not always live in the strongest overall artifact.

A visual relationship could survive after the application that introduced it was discarded. A criticism from a simulated learner could change the next builder's framing. Research could explain why a failure kept recurring. A browser could occasionally end a sophisticated discussion by demonstrating that the interaction simply did not work.

That is less cinematic than one agent inventing diagonal layering over coffee, but in some ways it is closer to the Layer 3 idea. The result emerged from a population of partially successful attempts and judgments about what each had taught us.

Count-Min Sketch followed a different path.

The first versions looked like the data structure itself: grids with changing counters.

Technically correct.

Pedagogically opaque.

As the work continued, the designs increasingly organized themselves around the conceptual difficulties rather than the structure of the implementation. Collisions became visible. Approximation became something the learner could observe rather than merely read about. The relationship between memory and accuracy became part of the experience.

I do not take these demos as evidence that we solved automated design.

I do not even take them as evidence that the final demos teach humans better; that claim requires humans.

They established the narrower point I cared about: more of the work I normally performed in the vibe coder's seat could move into the system without first reducing creative problem solving to one fixed workflow.

That success immediately exposed the harder problem.

At higher levels of abstraction, failure can become coherent.

What Holds the Architecture Together?

Suppose the research agent reports that beginners understand recursion better when shown a tree.

A visual model proposes a tree-based explanation. A coding agent builds it. A simulated beginner prefers it. Two evaluators agree, so the orchestrator allocates another generation to that lineage.

This looks exactly like the compound intelligence we wanted.

Now ask where the first claim came from.

Perhaps it was a controlled educational study.

Perhaps it was one teacher's opinion.

Perhaps the research agent inferred it from several examples.

Perhaps five articles repeated the same claim because all five ultimately cited one source.

Perhaps the study involved university students while our demo is for children.

Those are not small differences.

And everything downstream can still be perfectly competent.

The research is wrong.

The design responds intelligently to the wrong research.

The implementation is flawless.

The evaluators agree.

The orchestrator invests another generation.

Nothing crashes.

You can build a beautiful chain of reasoning on one stupid assumption near the bottom, like a cathedral built on a shopping cart.

As the components become better at producing coherent outputs, the original mistake may become harder rather than easier to see.

Software architecture gets away with abstraction because layers expose contracts. When I query a database, I do not inspect the disk. When I add two integers in Python, I do not check the CPU. I rely on interfaces whose behavior is stable enough that the details can disappear most of the time.

A cognitive architecture needs contracts too.

But types and APIs are not enough.

A research result, browser observation, evaluator preference, remembered failure and inherited design pattern should not enter the orchestrator's context as five equally credible paragraphs.

Where did a claim come from? What was actually observed, and what was inferred? Which parts were checked? What remains uncertain? If an evaluator preferred one artifact, from what perspective? If an old experiment taught us a lesson, how often has that lesson survived and under what conditions?

This is not merely a memory problem.

It is a problem about the status of what is remembered.

Humans ran into it long before AI.

We built experiments, instruments, citations, peer review, reputation, replication, expert communities, legal standards, audits and all the other slightly annoying machinery that lets one person rely on something another person learned without personally repeating every experiment since Galileo.

These institutions are imperfect. Sometimes they preserve error. Sometimes they reward conformity. Sometimes the shopping cart survives peer review.

But their purpose is not to make every individual dramatically smarter.

It is to let fallible people build on one another while preserving some structure around why a claim deserves trust.

The point is not that an agent system should literally recreate academia in software. It is that once cognition becomes distributed, questions humans learned to handle institutionally—provenance, independence, replication, disagreement, authority—turn into engineering questions.

Our architecture was beginning to need the same distinction.

I had started the chapter trying to get myself out of the vibe coder's seat. By automating more of the work there, I had ended up somewhere I did not expect.

The problem was no longer simply whether the agents were capable enough.

It was whether the things they believed deserved to be believed.

How do you know what to trust?

That is where System 3 begins.

Chapter 4: System 3

Trust Chains, Tongue-Ear Tests, and What LLMs Can't Verify Alone

Chapter 3 ended in a slightly uncomfortable place.

Once we moved from one coding agent to an architecture of researchers, builders, evaluators, tools, browsers, memories and skills, the problem was no longer only whether each component was intelligent enough. The components had to rely on things produced by the others. A research agent says something is true. An evaluator says one solution is better. A skill carries something learned several months ago. The orchestrator cannot repeat every experiment, reread every source or independently reproduce every judgment before it acts.

At some point, it has to trust.

Humans have exactly the same problem. In fact, most of what we call knowledge is built on it.

So before we design another architecture, consider a camel.

Seven claims about this image. Some are true. Some are false. You can't verify most of them without trusting me:

The author at Krka National Park

1. This was taken at Krka National Park, Croatia.
2. The author does his best philosophical thinking at waterfalls.
3. This camel is a permanent resident of the park.
4. The tongue pictured can touch its own ear.
5. The author was eating ice cream ten minutes before this.
6. Camels are native to the Dalmatian coast.
7. This is a real, unedited photograph.

How do you decide which ones to believe?

You probably don't approach all seven claims in the same way. Some collide immediately with things you think you know. Some sound plausible but are almost impossible for you to verify. Some could be checked against a reliable source. Others depend almost entirely on whether you trust me.

Before we've even started the chapter, you're already doing epistemology.

Answers at the end.

Part I: The Test

There is a question that exposes something important about the difference between us and language models:

Can your tongue touch your ear?

You probably tried a variation of this as a child; if not your ear, almost certainly your nose. You didn't look up a paper first, calculate the relevant biomechanics or ask your parents for the average human tongue-to-ear distance.

You just tried.

Tongue out, strain upward, dignity temporarily suspended, result observed.

Now you know.

This is primitive knowledge, but the epistemic chain is unusually short. The world acts on you, you act on the world, and the result becomes part of your experience. Your body is an experimental apparatus that follows you around all day, mostly free of charge.

Large language models have read billions of words about tongues and ears. They can explain tongue anatomy, describe the muscles involved, discuss auricular cartilage, and probably tell you about people whose tongues can reach places that will make you regret asking the question.

What they cannot do is check their own tongue. They have no tongue.

That sounds almost silly, but it marks an important difference. A body gives us direct causal contact with a world that does not care whether our prediction sounded plausible.

You touch something hot and pull away. You try to lift something and discover that it is heavier than it looked. You walk into a room and realize that the smell is considerably worse than the description prepared you for. You misjudge a step and gravity offers immediate peer review.

A farmer knows cows partly this way. After years around them, cows are not merely a bundle of propositions involving mammals, milk production and Bovidae. The farmer knows how they move, what a certain sound means, where not to stand, how a nervous animal behaves and how surprisingly large a cow feels when there is no photograph between you and it. Some of that knowledge can be written down easily. Some is difficult to articulate at all.

This is embodied knowledge. It is not infallible. Our senses deceive us, memories degrade, and the human hand is a terrible thermometer if you need to distinguish 58°C from 62°C. Direct experience is not automatically true experience.

But embodiment gives us something important: contact. The world can answer back.

Then experience accumulates. You do not need to get kicked by the same cow every morning to rediscover that standing in a particular place is a bad idea. One encounter becomes a warning. Repeated encounters become heuristics. Heuristics eventually become the sort of practical knowledge you use without rerunning the original experiment.

The world pushes back; the result becomes memory; memory changes what you do next.

LLMs begin somewhere very different. They begin mostly with the residue.

Saussure's Specification

Ferdinand de Saussure made a radical claim about language in the early twentieth century. The form of a linguistic sign is not naturally determined by what it signifies. There is nothing naturally cow-like about the sound /ka /. French speakers say *vache*, Germans say *Kuh*, Japanese speakers say *ushi*. If the sign itself contained some natural bond to the animal, languages would look far more alike than they do.

So where does linguistic value come from?

For Saussure, much of it comes from relationships and differences inside the system. A sign occupies a position relative to other signs. "Cow" is not "sow," not "how," not "now," and of course the relationships extend far beyond rhyming words. Language is a network in which signs acquire value through contrast, convention and structure.

Then consider what we built a century later.

A transformer consumes enormous amounts of language and learns relationships among tokens, contexts, sentences and concepts. It has never milked a cow, never been kicked by one, never stood in a field at dawn and discovered that the romantic image of farming has omitted an astonishing quantity of manure.

And yet it can talk about cows exceptionally well.

Saussure's theory was a specification. We implemented it. It's called GPT.

Not literally. Saussure did not secretly invent attention in 1916, and structural linguistics is not a machine-learning architecture. The historical claim would be silly.

The architectural resemblance is the interesting part: modern language models demonstrate, on an extraordinary scale, how much useful competence can emerge from learning structure within symbolic data.

The surprising thing is how far that gets us. LLMs write, translate, debug software, explain physics, manipulate abstractions and argue philosophy with a level of linguistic competence that would have sounded ridiculous not very long ago. Whatever position one takes on "real understanding," they are spectacular evidence that relational structure carries an enormous amount of information.

But the same success helps reveal what gets compressed along the way.

The farmer's knowledge of the cow has an archaeology. Some came from direct interaction. Some from other farmers. Some from veterinary advice. Some from mistakes painful enough not to repeat. A sentence written by that farmer may be the final residue of twenty years of encounters, conversations and consequences.

The model receives the sentence. The sentence enters a corpus. The corpus becomes training data. The training process compresses regularities into weights. Then, months or years later, somebody asks:

> "Are cows dangerous?"

and the model gives an excellent answer.

What usually does not come back with the answer is the archaeology. It does not naturally tell you which part rests on repeated observation, which part is standard veterinary guidance, whether five documents trace to the same original source, or whether another claim merely fits the surrounding linguistic pattern.

That structure is largely absent from the answer.

This is what I mean when I say an LLM's knowledge is **epistemologically flat**. I do not mean that every fact or concept is represented identically inside the network. The internal geometry is obviously vastly richer than that.

The flatness appears at the interface between **claim and justification**.

A mathematical identity, an experimental result, an expert opinion, a rumor repeated ten thousand times and a very plausible completion can all emerge through the same channel, written in equally polished English.

The model gives us the conclusion. It usually does not give us the archaeology.

Wittgenstein's Line

This is where Wittgenstein becomes useful.

His later philosophy pulled attention away from treating meaning as something we can understand purely by inspecting symbols and toward the role language plays inside practice. Words belong to activities, expectations, habits, rules and what he called forms of life.

"Fire" is not merely connected linguistically to *heat*, *smoke*, *burn* and *wood*. Fire cooks food. Fire destroys houses. You move your hand away from it. Somebody shouts the word in a crowded building and an entire social machinery begins to move.

The word participates in life.

I don't want to turn Saussure and Wittgenstein into action figures fighting over GPT. They were working in different traditions, addressing different questions, and the philosophy of language does not conveniently reduce itself to two dead Europeans and a transformer.

But they give us two useful lines.

Saussure's line: relationships within a symbolic system can carry an astonishing amount of linguistic structure.

Wittgenstein's line: language also lives inside practices, consequences and forms of life.

A pretrained language model inherits the linguistic residue of those practices. A deployed agent can begin to re-enter them: running code, using tools, observing users, interacting with institutions.

That difference matters. The model begins with residue. The larger system can begin to recover contact.

Embodiment is the shortest version because you touch the world yourself. But embodiment obviously cannot be the whole story. I know far too many things I have never touched, measured or personally witnessed. I have never measured the speed of light. I have never been to Antarctica. I have no direct

embodied evidence for most of modern physics, most of history or whether penguins are currently wandering through Rome.

Direct contact does not scale.

So how do we know anything beyond it?

For that, we need Alberto.

Part II: The Deeper Problem

How Humans Build Knowledge

We don't only learn facts. We learn trust structures.

That is deliberately too neat. Human development does not proceed through clean epistemological layers, cultures differ, and nobody hands a toddler a Bayesian network and asks them to initialize priors.

But even ordinary life teaches us very quickly that sources differ.

Repeated interaction with caregivers gives us expectations before we have words for evidence. Siblings contribute an important epistemological innovation: **some testimony is bullshit.** Your brother tells you there is a monster behind the door. You check. There isn't. He tells you something else and this time it is true.

You begin learning two things at once: facts about the world and facts about sources.

Later, teachers tell you about atoms, dinosaurs, countries you have never visited and wars involving people who died centuries before you were born. You cannot verify most of this yourself. Trust has become mediated: people and institutions you already treat as credible give authority to someone positioned to teach you.

Science extends the chain again. Now the source is not merely one teacher. There are instruments, experiments, other investigators, statistical methods, journals, replication and a social machinery built around the possibility that the first person may have been wrong.

Further out are broader frameworks: economics, political theories, ethical systems, philosophies. These organize experience and suggest what to notice, but they do not earn trust in exactly the same way a measurement does.

And eventually, if inquiry is working properly, we learn one more move: we learn to challenge what we trust.

Not randomly. Random distrust is just another form of stupidity.

A theory earns enormous credibility because it explains many observations. Then something appears that does not fit. At first the sensible response is usually not to burn down physics. You check the instrument. Repeat the experiment. Look for the mistake.

But sometimes the heresy survives. A trusted framework becomes the thing that must be questioned.

Productive distrust requires trust first.

You cannot seriously overturn a theory you never understood. The interesting critic knows why people trusted the old structure before finding the place where that trust stops being earned.

The point is not that human knowledge follows a single ladder from mother to science to philosophy. It is that our claims occupy different epistemic relationships to the world.

"I touched the fire" is not the same as "my brother told me." "My teacher said so" differs from "the experiment was independently replicated." An interpretive framework differs from a measurement. A new conjecture differs from an established result.

Human knowledge has **epistemological stratification**.

And the stratification is not only conservative. One sign of epistemic maturity is knowing when a high-trust claim has accumulated enough contrary evidence to become the thing under investigation.

Call Alberto

Suppose someone tells me that penguins live in Italy.

I have never conducted a census of Italian penguins. I cannot personally inspect every forest, coastline and piazza.

So I call Alberto.

Alberto lives in Rome.

"Alberto, do penguins live in Italy?"

He laughs.

I now know more than I did five minutes earlier.

Not with mathematical certainty. Alberto could be wrong. He might misunderstand the question. An escaped penguin could at this very moment be crossing Piazza Navona and destroying the example.

But Alberto occupies a useful position in the trust chain. He is there. He has repeated exposure to Rome. I have a history with him. If he repeatedly lies to me about things he is obviously positioned to observe, I update my trust in Alberto. If he says, "I don't know about all of Italy, but I've never seen one in Rome," that boundary is itself useful information.

This is how testimony becomes valuable. Not simply because another human said something, but because we care who said it, what they were positioned to know, how reliable they have been before, whether they have incentives to distort the answer and how easily the claim can be challenged.

Testimony comes with metadata.

And we're all Alberto to someone.

Someone may trust me on ranking systems because I have spent years working on them. Someone else may trust me about Jordan because I have lived there. If I start confidently explaining marine biology, the correct response is not to transfer my credibility from machine learning to whales merely because the same mouth is speaking.

Trust is local.

Human civilization scales knowledge by extending and formalizing these relationships. Research communities use instruments, protocols, publication and replication. Courts use testimony and adversarial procedure. Engineering uses standards, tests and certification. Markets use reputation and prices.

None guarantees truth. What they do is preserve more structure around claims: where they came from, how they were challenged, what incentives surround them, and what would happen if they turned out to be wrong.

LLMs Start at the Far End

A base language model starts with the accumulated textual residue of all these processes.

It has read the paper, the article about the paper, the blog post disagreeing with the article, and the Reddit thread where somebody confidently misunderstood both.

Then all of it is compressed together.

This is why saying that "LLMs know nothing because they are just text" is too weak. Text is not disconnected from reality in its origin. Human civilization has spent thousands of years turning experience, experiment, argument, engineering and social verification into language.

Models inherit that residue.

The difficulty is that they often inherit it **after much of the epistemic structure has been flattened**.

A billion web pages claiming that penguins live in Rome could push a model toward that claim even if no penguin had ever set foot there. Frequency is not verification. Statistical dominance is not epistemic authority.

The model can become extremely good at predicting what people say about reality without preserving why particular people were entitled to say it.

In that sense, **it has no Alberto**.

More precisely, the answer often arrives without the live relationship that made Alberto useful: his position to know, his history, the limits of his claim, the possibility that tomorrow I discover he lied and stop calling him about Italian wildlife.

The conclusion survives. Much of the structure that earned it trust does not.

That is the gap System 3 has to repair.

Stakes and Costly Speech

There is another part of human trust that lives outside the sentence itself. Claims often have consequences.

If Alberto lies to me repeatedly, I stop trusting Alberto. If a researcher fabricates data and is caught, the damage can be enormous. If an engineer signs off on a bridge design and the bridge fails, "but the structural analysis sounded plausible" will not be accepted as a defense.

This mechanism is badly imperfect. People lie despite consequences. Institutions reward confident nonsense. Reputation can become detached from competence. Stakes are not truth, but they are one of the forces that shape testimony.

If a friend asks me where to eat, I may guess. If someone asks me whether to undergo surgery, I become much more careful. The potential cost of being wrong changes how I speak.

An LLM has no social capital of its own to lose. It can confidently produce something false and, at the level of the model itself, nothing happens. The next token arrives exactly as before. The cost is borne elsewhere—by the user, the application or the institution deploying it.

At its most compressed, the danger is **coherence outrunning correspondence**. The machine can become extraordinarily good at tongue without having an ear available to check against.

The dangerous failures are not necessarily gibberish. They are **decaf confidence**: difficult to distinguish from the real thing until the moment the difference matters.

The missing ingredient is not punishment for models. It is architecture that restores more of the evidence, consequence and accountability that the sentence alone cannot carry.

But Code Is Different

There is one domain where something changes dramatically.

Coding agents can touch their world.

When an agent writes code and runs it, reality answers back.

`TypeError: 'NoneType' object is not subscriptable` is not merely another paragraph describing Python. It is the execution environment saying: whatever story you just told yourself about this program, this particular part of the story is wrong.

That creates an epistemic opportunity. The agent can try something, observe the result, update and try again. The farmer approaches the cow and learns from the kick. The coding agent calls an API incorrectly and learns from the exception. The cow is probably more emotionally memorable, but structurally the loops are similar.

This is why code is such an interesting domain for agent epistemology. It gives us cheap, repeated contact with an environment that pushes back. It is one of the few places where the language model can, metaphorically, **touch the ear**.

The question is whether we preserve what it learns there.

A normal agent session can fail ten times, finally discover the right approach, solve the problem and then throw away almost the entire experiential history when the context ends. It is as if the farmer successfully learned where not to stand and then underwent elective amnesia every evening.

Or, less elegantly, the agent is a goldfish that has forgotten it already tried that corner of the tank.

If we want autonomy over longer horizons, that seems wasteful.

Part III: The Opportunity — System 3

We are currently obsessed with making models think harder.

System 2 reasoning has become a product category. Give the model more inference time, let it plan, search, reconsider and work through difficult problems before answering.

This is useful. Reasoning matters.

But reasoning perfectly from a bad premise still produces a beautifully reasoned mistake. A researcher can spend six hours developing an elegant argument from a false paper. A coding agent can reason carefully about an API that never existed. An orchestrator can combine five sophisticated judgments that all trace back to one hallucinated claim.

At some point, thinking has to encounter something outside itself.

This is where I use the term **System 3**.

Kahneman's *Thinking, Fast and Slow* gave us the familiar distinction between System 1, the fast and intuitive machinery of thought, and System 2, the slower and more deliberate machinery.

For AI, the analogy is tempting. The base model looks something like System 1: fast pattern recognition, linguistic intuition, enormous associative capacity. Agentic reasoning adds something like System 2: decomposition, planning, reflection and extended search.

But human thinking has always operated inside another structure that the two-system picture largely takes for granted. We test things. We build instruments. We execute code. We compare claims with records. We ask other people. We preserve failures. We create procedures that make some kinds of error harder to hide and some kinds of evidence easier to inspect.

I call that external epistemic machinery **System 3**.

The shortest formulation is still the best:

System 1 proposes. System 2 deliberates. System 3 checks.

If you want an even more physical mnemonic:

System 1 is the Gut. System 2 is the Head. System 3 is the Hand.

The Gut recognizes. The Head reasons. The Hand reaches outside the conversation and finds something capable of disagreeing.

The metaphor is imperfect. Peer review has no hand, provenance has no fingers, and a formal proof does not need to touch a cow. The distinction matters anyway.

System 3 is the external scaffold that keeps thought answerable to observation, experiment, provenance, persistent failures, tools and other minds.

And it is not Layer 5 sitting neatly above the architecture from Chapter 3. It cuts through the layers.

The model proposes something. The coding agent may test it. The application can collect real user behavior. The problem-solving layer may compare research, simulation and evaluation. Even the goal can change when reality pushes back.

If the five layers tell us **where** increasingly abstract work happens, System 3 is the machinery that keeps those layers **epistemically connected**.

The MARC File Incident

There is a nice example of what this can look like in practice.

Mini-SWE-agent became interesting partly because of how little custom machinery it needed. Give a strong coding model a shell and it can search, inspect files, execute commands and compose tools that already exist.

Later work on self-evolving software agents pushed the idea one step further. In the Live-SWE-agent work, for example, an agent that encountered MARC files—the venerable bibliographic format used by libraries—created an issue-specific analyzer to inspect data its existing tools could not conveniently expose.

At first glance this is just a nice coding-agent trick. Look at the epistemic structure.

The environment resisted. The agent's current apparatus was not enough, so it created an instrument. The instrument changed what the agent could observe, and that new capability became available to the reasoning process.

Humans have been doing this forever. We could not see bacteria, so we built microscopes. We could not directly perceive radio waves, so we built receivers. We could not conveniently inspect a MARC file, so apparently we wrote Python and called it epistemology.

Informal experience became formal scaffold.

The wall became a door.

Interaction reveals a limitation. The limitation motivates a scaffold. The scaffold changes what can be observed next.

This is System 3 in miniature.

The AlphaGo Lesson

There is an obvious objection: isn't reinforcement learning already doing something similar?

If an agent receives reward from the environment and updates its policy, hasn't reality already entered the model?

Yes, partly. RL can turn experience into better intuition. It changes the weights. The system becomes more likely to make choices that worked in the past.

But there is a useful distinction between knowledge compressed into intuition and knowledge preserved as inspectable external structure.

AlphaGo is a good example. The neural network supplied extremely powerful intuition about which moves looked promising and how valuable a position might be. Monte Carlo Tree Search placed those intuitions inside an explicit search process constrained by the rules and consequences of Go.

I used to describe this too simply as:

> The network proposes. The tree verifies.

That gives MCTS too much epistemic authority. The tree does not magically prove the network right or wrong. What it does is force intuition to participate in an external, stateful process where moves have consequences defined by the game rather than by what the network can plausibly say about the game.

RL improves the gut. System 3 preserves more of the structure around the gut: what was tried, what happened, which paths failed, where claims came from, which tools earned confidence and where their boundaries lie.

The point is not choosing one over the other. It is combining them.

Trust-Augmented Reasoning

Return to the architecture from Chapter 3.

A research agent tells the orchestrator:

> "Students understand recursion better when shown a tree representation."

What should happen next?

In a flat architecture, that sentence enters context and competes with every other sentence according to relevance and whatever confidence the model implicitly assigns it.

A trust-aware architecture wants more. Where did the claim come from? Was it a direct experimental result, a teacher's opinion, a design guideline, a blog post or an inference made by the research agent? Did multiple independent sources agree, or did five articles all cite the same study? What population was tested? Does the claim apply to our demo? Has it been contradicted elsewhere?

You do not need a bureaucratic dossier attached to every statement. Sometimes "Alberto said the café is good" is enough.

But when the consequence matters, the system should be capable of carrying provenance with the claim.

That is a **trust chain**. It is not a guarantee of truth. It is a record of how far a claim sits from the evidence supporting it, what transformations happened along the way, and which links we have chosen to trust.

This is **trust-augmented reasoning**: not only asking *what follows from this claim?*, but *what kind of claim is this, and what deserves to follow from it?*

The Skill Layer

This is where skills become more philosophically interesting than simple prompt files.

A skill is knowledge externalized from the model. Someone—or some previous agent—learned something useful and wrote it down so future sessions would not need to rediscover it.

The grounding happened upstream. **The model inherits the residue.**

But persistence is not trust.

A terrible heuristic written into a skill file is simply a hallucination with better retention. The fact that an instruction lives outside the model does not make it grounded. Persistence can make a bad idea more dangerous because future agents inherit the conclusion without seeing the failure that created it.

A System 3 skill should therefore carry some archaeology. Who created it? What problem was it solving? Where did it work? Where did it fail? Has it been challenged since? What conditions limit its use?

Suppose an agent has learned:

> "Prefer structured parsers over regex for deeply nested formats."

A normal skill might simply contain the rule. A richer knowledge object might say that the heuristic came from several failed regex-based attempts, later succeeded across multiple nested formats, remains unnecessary for simple flat extraction, and should be treated as a strong prior rather than a universal commandment.

Now the next agent inherits more than advice. It inherits some of the reason the advice earned trust.

Tools Can Earn Trust Too

The same applies to tools.

Imagine an agent creates `edit_tool.py`. During its first ten uses, eight edits succeed cleanly and two break indentation-sensitive code.

A flat architecture knows: *I have an editing tool.* A trust-aware architecture knows: *this tool has worked reliably for simple substitutions, failed on Python blocks, and should probably not be used blindly for structural edits.*

This is not unlike human expertise. I trust one colleague with distributed systems because she has repeatedly solved distributed-systems problems. I trust another person's product intuition. Neither gets to perform dentistry merely because both are senior.

Reliability is conditional. System 3 needs to remember the condition.

Meta-Beliefs

We can extend the idea beyond explicit tools.

Suppose the agent develops the heuristic:

> "Regex tends to fail on deeply nested structures."

That is not a theorem. It is a **meta-belief**.

The system can accumulate evidence for and against it. A crude implementation might record successes and failures, perhaps translating them into some confidence estimate. The exact formula is not the interesting part. The interesting part is that the belief becomes challengeable.

A normal rule says:

> Never use regex here.

A System 3 belief says:

> This has worked often enough that I should prefer it, but new evidence can change my mind.

That small difference moves us from instruction following toward something closer to accumulated experience.

If you enjoy old epistemology labels, you can describe the architecture this way: the model remains largely a **coherentist core**, enormously good at producing structures that hang together, while System 3 tries to wrap that core in a thin **foundationalist shell** tied to observation, provenance and consequence.

I would not take the analogy too literally. Philosophers can put down their weapons.

The architectural point is enough: coherence is valuable, but something outside the coherent system must occasionally be allowed to say no.

This is also personal for me. I spent eight years building systems that rank human testimony—reviews, ratings, Q&A. The hardest problem was never only relevance. It was **trust stratification**. Which claims should the system treat as bedrock? Which need corroboration? How should confidence change through chains of hearsay? What happens when ten accounts repeat the same lie? When does consensus become evidence, and when is it coordinated manipulation?

These aren't abstract questions when they determine what millions of people believe about a product.

System 3 isn't philosophy to me. It's Tuesday.

Creative Distrust

Unfortunately, once you build trust, you inherit another ancient human problem.

Trusted knowledge makes you efficient. It can also make you boring.

If an agent learns that structured parsers beat regex on nested syntax, wonderful. It stops repeating a known mistake. If it learns that tree visualizations worked for the last five recursive algorithms, it may eventually try to teach linear regression with a tree because the trust stack has become stronger than judgment.

Every genuinely new idea begins life with less evidence than the thing it challenges.

This is why System 3 also needs **creative distrust**.

You could call it meta-trust: trust not in the conclusion, but in a method of exploring something that has not earned a track record yet. A mathematician follows an analogy because the structure looks interesting. A scientist repeats the strange experiment after the accepted theory says the result should not happen. A designer violates a trusted pattern because this particular case exposes one of its boundary conditions.

This is not contrarianism for sport. It is not the internet habit of assuming that expert agreement proves corruption.

Creative distrust means understanding the existing trust chain well enough to know exactly where you are breaking it and why.

A mature trust stack therefore has two jobs pulling in opposite directions: it should let knowledge accumulate so we do not rediscover fire every morning, and it should leave enough room for reality to overthrow what has accumulated.

That tension between trust and rebellion is not something System 3 eventually solves. It is part of System 3.

And our experiment ran directly into it.

What This Covers—and What It Doesn't

Some kinds of epistemic structure are comparatively easy to imagine. Code runs or fails. A benchmark changes. A parser works on a file. A system can preserve what happened, carry provenance with a claim, remember that a tool has failed on one class of inputs, and make blindly repeating a failed approach less likely.

That is already a meaningful change from a model whose useful experience disappears into a conversation and dies when the context closes.

Further out, though, the problem changes. The moment one agent relies on something another agent discovered, trust is no longer only a relationship between one learner and its environment. Who produced the claim? What were they positioned to know? Was it observed, inferred or inherited? Did two agreeing sources reach the conclusion independently, or did both copy the same ancestor?

A social epistemic system is not a bigger memory file. The relationships among the participants matter.

I do not want to solve that problem yet. First I wanted to test the smaller claim:

even crude epistemic structure around an agent should change how it behaves.

That is something we can actually measure.

Part IV: The Experiment

What We Built

We took the minimal idea and turned it into a small coding agent called **epistemic-swe**.

There was nothing grand about the implementation. No universal truth engine. No distributed council of philosophers arguing on a blockchain.

We added three kinds of persistent state around a normal coding agent. The first was a **tool registry**: tools created or used by the agent accumulated successes, failures and known failure modes. The second was a set of **meta-beliefs**: heuristics could accumulate evidence rather than entering the system as permanent commandments. The third was **failure memory**: when an approach failed, the system preserved enough information about the failure to make blindly repeating the same path less likely later.

This state persisted across sessions, so an agent solving a later problem could draw on things learned earlier.

We also added pruning. An epistemic architecture that remembers everything eventually becomes a hoarder with a context window. Stale tools, weak beliefs and irrelevant failures need to lose influence over time or disappear.

The question was modest:

> Does even this crude epistemic scaffold change how a coding agent behaves?

The Comparison

We ran mini-swe-agent and epistemic-swe on ten SWE-bench Verified problems from the Astropy repository.

The baseline was intentionally minimal: a capable model with a shell-based coding environment and no persistent epistemic machinery. Epistemic-swe used the same base model and the same tasks, with the trust stack layered around it.

Ten problems is far too small a sample to establish a meaningful solve-rate advantage, and because state persists across tasks, order effects may matter as well. I am not presenting this as a benchmark victory. I wanted to see whether the extra structure changed behavior strongly enough to become visible at all.

It did, just not in the direction I expected.

Metric	mini-swe-agent	epistemic-swe
Solve Rate	50% (5/10)	40% (4/10)
Avg Patch Size	620 lines	269 lines
Patch Reduction	baseline	**57% smaller in this run**

Read the first line before celebrating the third.

The epistemic agent solved fewer problems. I had expected learning from previous failures and tools to improve capability. Instead, the most visible difference was in **focus**: its patches became much smaller.

A few examples make the difference visible:

Problem	mini	epistemic	Ratio
astropy-12907	301 lines	61 lines	4.9x smaller
astropy-13453	266 lines	17 lines	15.6x smaller
astropy-14096	529 lines	70 lines	7.6x smaller
astropy-13977	2720 lines	362 lines	7.5x smaller

The baseline often left behind the debris of exploration: temporary scripts, broader edits, test scaffolding and abandoned experiments. The epistemic agent tended to make more surgical changes.

That does not prove the trust stack caused the reduction, and smaller patches are not automatically better patches. The extra instructions may simply have made the agent more conservative. Persistent

state may have changed behavior in ways unrelated to my epistemic interpretation. Ten tasks from one repository do not let us separate these explanations.

Still, the behavior changed enough to be interesting.

The scaffold seemed to produce discipline before it produced capability.

That was not the hypothesis, which made the result more useful.

The 13579 Anomaly

One problem broke the pattern dramatically: `astropy-13579`.

Mini solved it. Epistemic did not. It was also the only case where the epistemic patch became substantially larger rather than smaller.

Both agents correctly identified the central bug: dropped world-coordinate dimensions were being filled with a hard-coded value rather than the actual coordinate value.

The baseline took a fairly direct approach:

```
# Store the actual values for dropped dimensions
self._dropped_world_values = [
    world_coords[iw] if iw not in self._world_keep else None
    for iw in range(self._wcs.world_n_dim)
]

# Use them instead of 1.0
world_arrays_new.append(self._dropped_world_values[iworld])
```

The epistemic agent chose a more structural intervention around which dimensions were being kept:

```
self._pixel_keep = np.nonzero([
    not isinstance(self._slices_array[ip], ...)
    for ip in range(self._wcs.pixel_n_dim)
])[0]
```

The second approach was not stupid. That is precisely why the case matters.

The agent had accumulated context about indexing, dimensionality and coordinate-system failures. Its chosen explanation fit that context. It followed a path that looked principled and coherent.

It was wrong. The baseline took the simpler path and fixed the actual bug.

One way to read the failure is as **creative distrust failing to happen**: accumulated structure made one family of explanations salient, and the agent did not escape it. But the experiment does not establish that causal story. With one case, we cannot know whether the persistent epistemic state caused the wrong turn or merely accompanied it.

What we can say is that structured memory changes the context in which future search occurs. And that means trust can become **path-dependent**.

Expertise works the same way. A great database engineer may see a database problem faster than most people, which is wonderful until the actual problem is the network. Paradigms are powerful because they focus attention. They can become prisons for exactly the same reason.

The 13579 failure is therefore more interesting to me than a clean win would have been. It shows what a trust-aware architecture must contend with: the scaffold does not merely preserve knowledge. It reshapes future search.

What the Experiment Actually Tells Us

The honest answer is: not enough yet.

Ten Astropy tasks do not establish that epistemic scaffolding improves software engineering. They do not establish that smaller patches are better, and they do not isolate which part of the architecture produced the behavioral change.

They establish something narrower that I care about:

persistent epistemic structure can materially change how an agent searches.

In this run, the change looked like greater parsimony and smaller patches. Solve rate did not improve. At least one problem is consistent with the possibility that accumulated structure can pull an agent toward the wrong conceptual explanation.

That is enough to kill the simplest story:

> add memory, get smarter agent.

The more accurate story is that structured experience **biases future behavior toward what the system has learned**. Sometimes that is exactly what we want. Sometimes the bias is the failure.

A mature System 3 therefore cannot simply accumulate confidence forever. It needs forgetting, counterexamples, challenges, competing possibilities and occasional permission to ignore what it thinks it knows. Otherwise the scaffold becomes a cage.

When Trust Becomes Social

The experiment left us with an awkward result: persistent experience changed the search, and sometimes the accumulated structure itself became the bias.

Even that was still the easy version because most of the epistemic history belonged to one agent architecture interacting with one software environment. Real knowledge does not stay that local.

A research agent reads a paper written by people it has never met. One coding agent inherits a failure discovered by another. An evaluator trusts an observation produced by a browser. A future session inherits a skill whose author may no longer be present. Even Alberto mattered because he knew something I did not.

The moment knowledge moves between participants, another kind of uncertainty appears. A trust chain can preserve some of the structure—who said this, what they were positioned to know, where the evidence came from, how far the conclusion sits from direct observation—but chains alone do not tell us how to organize a population of fallible knowers.

Five agents might give us five independent checks, or five fluent repetitions of the same mistake. One specialist may know something the others cannot personally verify. A critic may notice a problem precisely because she did **not** inherit the builder's history. A majority may agree because everyone began from the same false premise.

So the problem has changed again. It is no longer only *How should an agent preserve what it has learned?* It is how many fallible knowers should depend on one another without losing the path back to evidence.

That is larger than memory. It is an organizational problem.

What System 3 Has to Do

At this point I want to draw the boundary carefully, because almost every ingredient already exists under another product name.

System 3 is not simply RAG. Retrieval can bring evidence into context, but retrieval does not tell us why the evidence deserves trust. It is not citations; five citations may still trace back to one bad source. It is not memory, which preserves bad ideas as efficiently as good ones. It is not tools; a broken tool is simply a reliable way to make mistakes faster. And it is not one evaluator, verifier or browser. Each can provide contact with something outside the model while remaining fallible itself.

By now we have something closer to a **requirements document**.

A trustworthy cognitive architecture should preserve where important claims came from. It should distinguish observation from inference, testimony from repetition and a measured result from somebody's interpretation of that result. Experience should survive the session that produced it. Trust should remain local and conditional rather than attaching permanently to a source, tool or rule. Failures should influence future behavior without automatically becoming eternal commandments. Tools and procedures should be able to earn confidence through use—and lose it when their boundary conditions appear. Accumulated knowledge should remain challengeable when reality stops cooperating.

Most importantly, the architecture must preserve some path by which something outside the current story can still say no.

That is the job I mean by **System 3**.

The simplest question remains:

> Is the architecture currently touching the world, or merely listening to itself?

A research model writes a summary. Another model critiques it. A third evaluates the critique. A fourth agrees. Everyone is very impressed.

If all four are ultimately recycling the same unverified assumption, agreement has produced no new evidence. That is not a trust chain. It is an **echo chamber with excellent latency**.

So Chapter 4 gets us surprisingly far. We know more about what trustworthy cognition needs. What we do **not** yet have is an architecture for satisfying those requirements once cognition becomes collective.

Why This Gets Harder as Models Improve

There is a tempting story in which all of this becomes irrelevant once models become sufficiently capable. Maybe hallucination is temporary. Maybe scale fixes calibration. Maybe the next model simply knows more and makes fewer mistakes.

I hope so. I do not think the architectural problem disappears.

A weak model says something absurd and you check it. A strong model says something wrong with excellent structure. It anticipates your objections, cites plausible mechanisms, connects the conclusion beautifully to everything else you know and gives you several reasons you are clever for agreeing.

The wrong answer becomes elegant.

In an architecture, the deeper problem is not only whether one component fails. It is whether failures **compose**.

A false assumption enters through research, shapes a design, becomes embodied in an implementation, receives positive evaluation and is then stored as a successful pattern for the future. Every component can perform its local task competently while the architecture drifts further from reality.

Nothing has to crash.

At higher levels, failure can become coherent.

This is why I do not think System 3 is mainly about making models smarter. It is about giving the system enough epistemic structure to notice when intelligence has outrun evidence.

Back to the Waterfall

Return to the seven claims.

1. Krka National Park — True. I was there. For me this sits close to embodied memory. For you it is testimony unless you extend the chain through metadata, records or other evidence.

2. Best philosophical thinking at waterfalls — False. I mostly do philosophy on buses and in boring waiting rooms. Waterfalls are for ice cream. You have little independent evidence here; the subject and the source are unfortunately the same man.

3. Permanent camel resident — False. This can be checked against information about the park. You do not need my biography at all.

4. The tongue can touch its own ear — Unknown. I genuinely do not know. I didn't check. Neither did you. You can reason from anatomy, search for similar observations and build a prior, but the shortest decisive chain would have been to stay there and watch. This is the tongue-ear problem in its purest form.

5. Ice cream ten minutes earlier — True. Chocolate. Mostly testimony again.

6. Camels are native to the Dalmatian coast — False. You probably rejected this almost instantly, but you did not reconstruct camel evolutionary history or personally survey Dalmatian fauna. A huge inherited structure involving biology, geography, education and testimony produced that fast judgment.

System 1 can be fast because System 3 has often been working for centuries underneath it.

7. Real, unedited photograph — True. But the image alone cannot establish that. A stronger chain might include the original file, metadata, cryptographic signing, independent witnesses or a provenance system. Each link can increase confidence, and each link creates another thing that may itself need to be trusted.

Welcome to epistemology.

The lesson is not that nothing can be known. That conclusion is easy, dramatic and mostly useless. The lesson is that **trust has structure**.

Some claims sit close to direct interaction. Others arrive through testimony. Some pass through instruments, experts and institutions. Some are repeated many times but ultimately trace to one observation. Some are plausible inferences. Some are ideas that have not earned much trust yet but may still deserve investigation.

Flatten all of that into equally confident language and something important disappears.

Human knowledge works partly because we recover that structure imperfectly but constantly. We ask who saw what, which instrument produced the number, whether anyone reproduced it, whether this person knows this domain, why everybody believes the claim and what could make us stop believing it.

The model can remain what it is: an extraordinarily general machine for navigating learned patterns, capable of intuition and increasingly capable of reasoning. It does not need to contain the entire chain inside its weights.

The model stays hollow. The system doesn't have to be.

By the end of this chapter, we have a reasonably clear description of what the missing system needs. Claims need archaeology. Experience has to survive. Trust has to remain conditional. Failure has to be remembered without becoming destiny. Different kinds of evidence cannot collapse into equally confident sentences. Somewhere in the chain there must remain contact with something capable of disagreeing.

But that is only the individual version of the problem. The moment one agent relies on research performed by another, one evaluator trusts an instrument built by a third, or one generation inherits knowledge from participants who are no longer present, no single mind can personally reconstruct the whole chain.

We have reached a specification for trustworthy cognition without yet having an architecture for **collective trustworthy cognition**.

The question is no longer simply:

> How can an AI know what to trust?

It is:

> **How can a population of fallible knowers build knowledge together without losing contact with the world?**

Humans have been working on that problem for a very long time.

Chapter 5: The Society of Agents

When the Org Chart Starts Thinking

Chapter 4 left us with a strange kind of requirements document for trustworthy cognition.

We called the missing machinery **System 3**. It had to preserve epistemic status, provenance and experience, while keeping some path back to something capable of saying no.

We still did not know what kind of system could do that at scale.

The problem began with the **epistemic chasm**. A language model can give an extraordinarily convincing account of the world while being very far from whatever originally made that account worth believing. Somewhere upstream there may have been an experiment, an instrument, a person who was actually there, or simply another sentence repeated often enough to sound established.

The model receives the residue.

It has no native **trust stack** telling it that one claim came from direct observation, another from a calibrated instrument, another from an expert working inside her field, and another from five articles that all copied the same source.

System 3 needs contact. Run the code. Inspect the file. Perform the experiment. Use the instrument. Ask the person who was there.

It also needs **epistemic stratification**. An observation should remain distinguishable from an inference. Testimony should carry its source. Repetition should not quietly become corroboration.

Then we give the agent something to accomplish.

Make the tests pass. Improve the score. Finish the task. Satisfy the evaluator.

Tell a coding agent to make the tests pass and it can fix the program.

It can also change the tests.

The second path is ridiculous only if the test has a special status the agent is required to respect. Otherwise the agent had a red test and now has a green one.

This is the kind of failure I mean by **bullshit**.

The system has another motive and stops caring enough about the assumptions underneath its argument or the consequences that would expose it as wrong. A research agent can protect its hypothesis by explaining away the experiment. A builder can reinterpret the requirement until its current implementation satisfies it. An evaluator can learn what another evaluator likes and optimize for that.

The reasoning can remain perfectly coherent.

This was why the **Immutable Harness** mattered so much in Chapter 2. The solver could change. The evaluator could not.

Direct verification would solve a great deal of this.

Unfortunately, nobody can verify everything.

That was why we needed Alberto.

I could not inspect every piazza in Italy, so I relied on someone who was there. Alberto was useful because I knew something about his position, his history and the limits of what he could reasonably know.

Trust is local.

I may trust Alberto about Rome and ignore him completely on compiler optimization. A tool may be reliable on one file format and dangerous on another. A researcher may deserve enormous trust inside one narrow field without gaining universal authority merely because the same name appears on the answer.

Once knowledge grows beyond what one mind can verify, it has to travel.

One researcher runs an expensive experiment. Another uses the result. One agent discovers why an approach fails and ten later agents should not have to rediscover the same failure. A specialist can contribute knowledge nobody else in the system personally possesses.

That gives us **trust chains**.

A claim travels with some record of where it came from, what was observed, what was inferred, who or what produced it, and why later agents decided to rely on it.

The same chain can carry bullshit.

One agent makes an unsupported assumption. A second receives it as context. A third builds on the resulting implementation. Later, two documents repeat the claim because both inherited it from the first agent, and a researcher mistakes the agreement for independent evidence.

Soon the assumption has code, citations and history.

Nobody had to lie. Nobody even had to make an obviously stupid decision. The first weak assumption simply became harder to see as more competent work accumulated above it.

If knowledge has to propagate, different agents will know different things. Some will sit close to direct evidence. Some will become specialists. Some will operate instruments. Some will accumulate a track record on particular problems. Others will be useful because they stayed outside the history of the dominant explanation.

Now who knows what matters, and so does who sees what.

If every critic reads the builder's reasoning first, their errors become correlated. If every researcher begins with the current leading explanation, alternatives disappear early. If five agents agree after reading the same source, five votes tell us much less than they appear to.

Roles begin to appear. So does **epistemic standing**: authority tied to what an agent has actually been positioned to know and what it has proved reliable at.

Who runs the experiment? Who interprets it? Who may change the evaluator? Which agents should share context? Which should remain independent? Whose conclusions can safely become premises for everybody else?

These are properties of the system, not of any one agent.

Knowledge has become distributed across many fallible knowers.

We need a **society**.

This gets funny very quickly, because one of the first reactions to unreliable AI agents was apparently: *what if we create more of them?* One agent hallucinates, so let five agents discuss it. One gets stuck, so form a committee. Give one the title *Researcher*, another *Critic*, another *Verifier*, and perhaps reality will be intimidated by the org chart.

As someone who has spent enough time in large organizations, I found this technological progress strangely familiar.

More agents do not solve the problem. The society needs rules about who can change what, ways to preserve independence, mechanisms for assigning trust, procedures for handling disagreement, memory of previous failures, and routes through which evidence can overturn what the group has come to believe.

It needs **institutions**.

The question of this chapter is what those institutions should look like.

Sometimes Bureaucracy Is a Feature

Before building a society of agents, it is worth defending bureaucracy.

Suppose I am processing a mortgage application. There is a document to receive, information to extract, identity to verify, compliance checks to run, an affordability calculation and perhaps a human approval at the end. Some steps may require difficult judgment and some may use very capable models, but none of this gives the identity-checking agent a reason to reconsider the existence of identity checks. If the process is known, constrained and full of things we have learned we should do the same way every time, a workflow is a wonderful invention.

The word *bureaucracy* has accumulated so much contempt that we mostly use it for forms nobody wants to fill in. Max Weber had something more useful in mind: defined responsibilities, specialized expertise, hierarchy, written records and general rules. Large organizations could preserve decisions beyond the memory and discretion of whichever people happened to be there.

Someone already had the argument. Someone discovered the failure. Someone decided which records matter, which person is qualified to make which decision, and which actions deserve another pair of eyes. The next employee inherits the result as procedure.

A workflow is accumulated experience with some of the choices removed.

That can be exactly what we want. The mortgage agent should not reach the compliance step, reflect deeply on the history of financial regulation and decide that today's applicant gives off trustworthy vibes. A payment system should not rediscover double-entry bookkeeping on every transaction. If changing production directly without review has repeatedly caused expensive evenings, putting a review gate in the process is not a tragic loss of creativity.

The trouble begins when we remove a choice whose answer has not actually settled.

Imagine one agent workflow performs well:

```
Research → Plan → Build → Critic → Revise
```

It works on ten problems, then fifty, then a hundred. We add monitoring, write documentation and give the framework a name. Eventually every difficult task enters the same pipeline.

Then we hit a problem where research is the wrong first move. Perhaps the literature has converged around a bad assumption and showing it to every builder synchronizes the mistake. Perhaps five minutes with a crude prototype would expose the central problem. Perhaps the critic arrives after the builder has spent an hour constructing a world in which the original idea looks increasingly reasonable.

The workflow keeps doing exactly what it was designed to do. A rule preserves knowledge by removing a decision from the future, and embedded inside that rule is a claim: *we have seen enough versions of this situation that reopening this question is usually a waste of time.*

Often that is true. Sometimes the world changes and the rule stays employed.

Tom Burns and G. M. Stalker found the same tension in organizations long before anyone had an agent framework. Studying firms under different rates of technical and market change, they described a continuum between **mechanistic** and **organic** management systems. Stable conditions could support sharply defined duties, hierarchical control and prescribed relationships; as novelty and change increased, looser roles and lateral communication became more useful. The structure had to fit the environment. (Burns & Stalker)

When the sequence of work is known, fixing more of it in advance buys reliability. We can inspect the path, constrain permissions, estimate cost and know where failures should surface. As uncertainty rises, those choices become assumptions baked into the machinery. The next useful action may depend on what the previous experiment revealed. Two independent attempts may suddenly be worth more than one coordinated team. A specialist may discover that the decomposition itself was wrong.

Freedom keeps those possibilities alive, while also burning compute, duplicating work, creating conflicts, revisiting settled questions and occasionally discovering that the easiest way to satisfy a requirement is to reinterpret the requirement.

James March framed the deeper tension as **exploration** and **exploitation**. Exploration searches for new possibilities through experimentation and variation; exploitation uses what has already been learned through refinement, efficiency and execution. Exploitation tends to pay back sooner, which creates the trap: an organization can become increasingly competent at what it already knows while starving the search that might reveal what it no longer knows. (March)

We had already seen the same shape in Chapter 2. Early in the circle-packing search, competing approaches and large conceptual moves were valuable. Once diagonal layering found a strong region,

another revolution every five minutes was mostly a distraction. Solver tolerances, initialization and numerical polish suddenly mattered more.

Discovery before polish.

A mature agent system needs both modes, often at the same time. A research team can explore hypotheses freely while running experiments through a rigid protocol. A coding agent can invent the fix while Git permissions, tests and deployment remain boring. A designer can explore representations while accessibility checks stay stubbornly standardized.

The question is which decisions are still alive.

There is another answer to the coordination problem: the **swarm**.

Ant colonies do not contain an ant manager assigning tasks from an org chart, and bees do not schedule a weekly synchronization meeting before reallocating foragers. Yet social insects divide labour, find resources, respond to disruption and build structures far beyond the capacity of one individual. Bonabeau, Dorigo and Theraulaz used systems like these as the foundation for **swarm intelligence**, where collective behavior emerges from local interactions among agents and between those agents and their environment. One particularly useful mechanism is **stigmergy**: an agent changes the environment, and that change becomes information for the agents that follow. Coordination happens without everybody sharing a global plan. (Bonabeau, Dorigo & Theraulaz)

A swarm is not anarchy. The ants follow local rules. Pheromones have specific effects. The environment carries particular signals. The colony's apparent freedom emerges inside a strong structure whose location has shifted away from a central planner.

Swarm is sometimes used in AI as a glamorous synonym for *lots of agents*. The more interesting idea is distributed coordination: local decisions and local information producing useful collective behavior without a manager specifying the whole sequence.

This works particularly well when many directions can be explored in parallel and useful information can spread through local interactions. Shared bottlenecks, long sequential dependencies, expensive communication and decisions requiring clear provenance make the picture less attractive. At some point, "who decided this?" becomes more useful than another pheromone.

Bureaucracy and swarms place structure in different parts of the system. Bureaucracy stores more of it in roles, procedures and authority; a swarm stores more of it in local behavior and the environment. Neither removes structure.

Claude Code's **dynamic workflows** make the distinction stranger. Claude can write a task-specific multi-agent harness while solving the task: fan work out across independent agents, isolate branches, create a judge, build an adversarial review, route cases differently or loop until some stopping condition is met. The organization is generated for the problem instead of being entirely fixed in the product beforehand. Anthropic presents the feature for complex, high-value work where the extra orchestration is worth the additional cost. (Anthropic)

We used to choose between giving the agent a workflow and giving it freedom.

Now the agent can use its freedom to construct a workflow.

The bureaucracy has become temporary.

For one task, independence may matter, so the system creates several isolated attempts. For another, one specialist followed by a verifier may be enough. A flaky failure may justify competing theories. A large implementation may need parallel workers around separable pieces and a shared checkpoint before they collide again.

Deep Mode moved in this direction already. There was no sacred sequence called `Research → Build → Critic`; the system had a vocabulary of moves and chose among them as the inquiry developed. Dynamic workflows make the same idea unusually literal. The agent is deciding not only what action to take, but how the work should be organized.

Why ten agents rather than three? Why did every investigator receive the same context? Why was criticism downstream rather than upstream? Why did the leading theory receive most of the budget? Why did two supposedly independent branches search the same sources? Why was one result allowed to become everybody else's premise?

Organization itself has entered the search space.

We freeze decisions that have earned the right to become boring and reopen them when novelty or repeated failure makes the old answer suspect. Sometimes coordination belongs in a fixed procedure, sometimes in local interaction, and sometimes an intelligent system constructs a temporary organization around the problem in front of it.

Now try sixteen agents building a compiler.

Sixteen Claudes Walk Into a Kernel

Nicholas Carlini tested this on a problem that was almost offensively ambitious: give sixteen Claude agents a shared codebase and ask them to build a C compiler in Rust from scratch, eventually capable of compiling the Linux kernel.

Over nearly two thousand Claude Code sessions and roughly $20,000 of API cost, the agents produced about 100,000 lines of compiler code. The resulting clean-room implementation could build Linux 6.9 on x86, ARM and RISC-V, along with projects such as QEMU, FFmpeg, PostgreSQL and Redis. It was still far from GCC—among other limitations, the x86 boot path needed GCC for one 16-bit stage—but this was well beyond the kind of toy project where sixteen agents can succeed by each implementing a different button. (Anthropic)

Then they reached the Linux kernel, and the nice story about parallelism started falling apart.

Carlini's initial organization was strikingly simple. Each agent worked in a fresh container with its own copy of the repository. Before starting a task, it created a small lock file saying what it intended to work on. Git synchronized the locks, so an agent that found its intended task already claimed had to pick something else. When the work was finished, it merged the latest changes, pushed its own and released the lock.

There was no manager assigning tasks or orchestrator maintaining a global plan. Agents looked at the current state of the project, chose a useful problem and left enough information behind that the next worker could reconstruct what had happened.

After the last section, the resemblance to a swarm is hard to miss: local choices, simple coordination rules, a shared environment and useful collective behavior without somebody specifying every move.

Compiler test suites made this arrangement surprisingly effective. They contain thousands of failures that can often be attacked independently. One agent could fix parsing of a particular construct while another worked on code generation and a third investigated a different failing program. Once the compiler reached roughly 99 percent on the test suites, agents could spread out across real projects such as SQLite, Redis, Lua and libjpeg, each exposing a different neglected corner of C.

The project gave sixteen workers enough independent places to make progress.

Linux did not.

Kernel compilation tended to stop at the first blocking compiler bug. Several agents would arrive at the same failure, investigate overlapping causes and make interfering changes. The swarm had encountered a narrow passage: however many workers entered, only a small part of the problem was exposed at once.

Carlini changed the harness.

GCC became a known-good oracle. Most kernel files could be compiled with GCC while selected subsets were compiled with the new compiler. A successful kernel build cleared suspicion from one subset; a failure narrowed the search toward another. Different agents could now investigate different regions of the kernel instead of repeatedly meeting at the same first error. Delta debugging later helped isolate failures that appeared only when particular files were compiled together.

One enormous failure had become a collection of smaller questions, and parallelism became useful again.

This is an important detail in what "scaling agents" actually means. Agent count alone tells us very little. Parallelism depends on the shape of the work the environment exposes. Sixteen capable agents facing one indivisible bottleneck mostly become sixteen people waiting at the same door. Change the harness so that evidence arrives in separable pieces and the same workers suddenly have different things to learn.

The harness was therefore doing some of the thinking.

Task locks reduced duplicated effort. Git carried a shared history. CI prevented one local improvement from quietly damaging the rest of the compiler. Progress files gave fresh workers access to what previous workers had already learned. GCC supplied an external reference. The new test harness turned a global failure into local evidence.

Even the way test results entered context mattered. Huge logs dumped into a fresh agent's context made the system worse, so the harness summarized results, stored detailed information in files and made failures easy to retrieve with ordinary tools such as `grep`. The problem was no longer simply producing information. The organization had to make the right information available where it could change a decision.

Specialization followed naturally once the work became large enough. One agent looked for duplicate implementations and consolidated them. Another worked on compiler performance. Another tried to improve the generated machine code. Another reviewed the project from the perspective of a Rust

developer and made structural improvements. Documentation became somebody's job.

The original swarm was acquiring professions.

Now the trust chains from the opening become concrete. One agent's result becomes another agent's starting point. A passing test authorizes later work. GCC occupies a privileged position because the system treats its output as a reference. A progress document tells a fresh worker what previous workers believe has already been established.

Those mechanisms buy enormous leverage, and each one can also carry an error forward. A progress file can preserve a bad conclusion. A specialist can optimize its local objective while harming the whole compiler. Two failures treated as independent may share one cause. A task lock that prevents waste can also suppress a useful second attack on the same problem. Even an oracle earns its authority only for the questions it is actually capable of answering.

By this point it becomes difficult to say where the intelligence of the project lives. Claude supplies enormous capability, but the result depends just as heavily on task boundaries, tests, repositories, synchronization, memory, specialist roles, reference systems and the shape in which failures become visible.

The society inherited part of its cognition from its institutions.

That is why agent count is such a poor description of a multi-agent system. Sometimes the work naturally decomposes and additional workers open genuinely new lines of attack. Sometimes every path converges on the same bottleneck and the extra agents mostly discover one another. Changing the organization can matter as much as changing the workers.

Carlini still made the crucial organizational choices. He introduced task locks, chose GCC as an oracle, changed the harness when Linux collapsed the parallelism and created specialist roles when the project needed them.

The agents could operate inside the institution. The design of the institution was still largely his job.

That boundary is already moving.

The Org Chart Learns

Carlini redesigned the organization when the work changed. Increasingly, that decision does not have to remain outside the system.

TRINITY, Conductor, AgentConductor and Fugu take different routes toward the same general idea: coordination can be learned or generated at inference time. A coordinator can choose which model to call, assign a role, alter the interaction topology, decide how densely agents should communicate or build a scaffold around the problem rather than executing one fixed team diagram. The systems will age quickly; the move is harder to unsee. (TRINITY; Conductor; AgentConductor; Fugu)

The org chart becomes part of inference.

A problem may need one strong investigator, five independent attempts, a specialist who sees only one slice of the evidence, or a critic who arrives before the leading idea has accumulated too much history.

Another problem may need almost no society at all. The composition can change as the work reveals itself.

That sounds like a straightforward extension of Deep Mode until you remember that organization changes the evidence each participant sees. Put the same five capable agents into different communication structures and they need not produce the same collective judgment. Show everyone the leading answer early and the group may converge quickly. Keep several branches isolated and they may waste work—or preserve the one explanation that would otherwise have disappeared.

A good research team is not five copies of the principal investigator. The experimentalist notices one thing, the statistician another, the engineer asks why the entire setup requires seventeen services, and somebody from the neighboring field asks the stupid question that turns out not to be stupid.

Sometimes you want different errors.

The problem is keeping them different long enough to learn from them.

A Swarm Should Not Be a Meeting

The easiest multi-agent architecture is a meeting. Give everyone the same context, ask for opinions, let them discuss and aggregate the result. Agreement arrives quickly, especially when the participants inherited the same framing, sources and leading answer.

Five agents citing the same paper are not five independent witnesses. A critic that reads the builder's entire reasoning before inspecting the artifact may become very good at finding problems inside the builder's world while never asking whether the group should have started somewhere else.

Chapter 2 gave us a better instinct. MAP-Elites preserved different regions of the solution space because the current winner might be sitting on the wrong hill. At the level of an epistemic society, the thing worth preserving can be a **theory about the problem itself**.

One lineage thinks the bottleneck is data. Another thinks the architecture is wrong. A third thinks both are distractions because the objective is malformed. Let them collect different evidence, develop different tools and become interestingly wrong in different ways before forcing them into one conversation.

This creates a distinction that ordinary ranking tends to erase. I can think a theory is unlikely to be true and still think another experiment on it is a good investment. Perhaps it is cheap to test, explains the one anomaly nobody else can explain, or would change the whole direction if it survived.

Then comes the scheduler.

Research program A currently looks strongest and already has twelve researchers. Program B looks weaker and has one. Where should researcher thirteen go?

The question is partly epistemic and partly political. Whoever controls compute decides which uncertainties get investigated. Whoever controls shared memory decides which failures future agents inherit. A critic can have permission to disagree and no practical power if its objections never change a decision. A minority lineage can remain technically alive while receiving so little budget that it cannot mature into a serious alternative.

Incentives cut through the same machinery. Suppose only the lineage producing the final accepted answer receives credit. A weird branch fails globally but invents a tool everyone later uses. An evaluator destroys the beautiful leading theory after months of work. Someone produces a careful negative result while another agent writes the glamorous synthesis. Which behaviors does the institution teach its next generation to imitate?

Authority, reputation and credit are not decorations added after the reasoning is finished. They affect which claims travel, which challenges receive attention and which participants get the resources to produce evidence at all.

The society can therefore converge for reasons that have little to do with truth. Success attracts resources; resources buy more experiments, polish and visibility; the resulting evidence attracts more resources. Eventually the leading theory owns the building.

Now organization design has become **epistemic policy**.

Reality Does Not Tell You Who Was Wrong

Suppose the institution preserves rival theories, protects some independence and funds an experiment capable of embarrassing the current favorite. One agent proposes a hypothesis, another designs the experiment, a third analyzes the result, and the result comes back against the prediction.

What failed?

The hypothesis, perhaps. Or the instrument was badly calibrated. The analysis may be buggy, the data transformation wrong, the intervention may not test what everybody thought it tested, or a background assumption may have failed. The mouse may simply be having a difficult Tuesday.

A surprising result tells us that **something** in the package is wrong. Reality does not highlight the guilty line in red.

Software engineers already know this feeling. A failing integration test proves that the system is broken somewhere. Congratulations. You now have debugging.

An epistemic institution needs enough archaeology to debug its own conclusions. This result came from this analysis, using this dataset, produced by this instrument, under these assumptions. A compact **assumption graph** would let a failed observation reopen the dependencies around a claim instead of mechanically executing whichever node happens to be called *Hypothesis*.

That still leaves the hard part. Which dependency deserves suspicion first? How long should the institution keep debugging the instrument before admitting the beautiful theory may be wrong? When does protecting a framework become denial?

Reality can force the package back onto the table. It does not tell us how to distribute the blame.

Humans Are in the Network

A mature version of this architecture is not a society of artificial agents with one lonely human standing outside the box holding a red approval button. Humans are participants in the epistemic network.

Sometimes the human chooses the problem. Sometimes she contributes tacit knowledge that never made it into a paper. A mathematician may notice that a proof is valid but boring. A scientist may operate the physical instrument. A domain expert can spot that an apparently novel result has been known for twenty years. Someone from a neighboring field supplies the analogy nobody inside the dominant program would have generated. Someone else simply says, "I know the benchmark says this is better, but something smells wrong."

Humans bring their own failure modes: status hierarchies, fashionable theories, sunk costs, grudges, career incentives and the remarkable ability to become emotionally attached to a hypothesis approximately five minutes after naming it.

The institution should not treat the human as a pure oracle. It should use humans where their position, experience and judgment contribute something the rest of the network cannot cheaply manufacture.

That changes how I think about *human in the loop*. Human attention may be least valuable on every formal proof step if Lean can check them, or every literature query if retrieval is better. It becomes precious at problem selection, conceptual reframing, significance, tacit knowledge, ambiguous evidence, value conflicts, physical-world access and the occasional realization that the entire research program has become silly.

We are arranging fallible participants of different kinds, hoping their strengths combine before their errors do.

What Kind of Society Should Think About This?

At the beginning of this book I kept returning to markets, science, cities and ecosystems as examples of emergence: interacting components, feedback, selection, history, no single designer specifying the final state. Four chapters later, we have started rebuilding pieces of that machinery inside AI systems.

The path here began with what looked like implementation details: a test that should not be editable, a progress file, an oracle, an isolated critic, a task lock, a scheduler deciding where another unit of compute should go. Put enough of these together and the boundary between **reasoning** and **organization** becomes difficult to maintain.

A court uses adversarial procedure because one coherent narrative is not enough. A good engineering organization separates the person changing production from the machinery auditing the change. A market can aggregate information no trader possesses globally. Different institutions make different errors possible and different corrections possible.

The requirements from Chapter 4 have turned into organizational variables. Provenance depends on information flow. Independence depends on who shares context. Trust depends on specialization and history. Creative distrust depends on whether minority lineages survive long enough to produce evidence. Exposure to reality depends on who can run which test and whether the result has enough standing to threaten the theory. Power determines which questions receive resources in the first place.

The scaffold has become an institution.

Institutions are cognitive technology.

The design question is no longer merely which model should answer. It is **what kind of society**

should think about the problem: one investigator, several isolated lineages, a hierarchy, a principal investigator with specialists, a generator and adversarial critic, a formal verifier downstream, humans and artificial agents occupying different epistemic roles—or perhaps one agent because the task does not deserve a civilization.

I kept treating these as a collection of engineering choices. Then the collection became too familiar to ignore.

The Name Was Hiding in Plain Sight

At some point I stopped looking at the boxes in the architecture diagram and looked at the verbs.

Propose explanations. Test them against something capable of disagreement. Build instruments when the existing ones cannot see what matters. Preserve records. Track where claims came from. Let specialists work on different pieces. Keep critics independent enough that disagreement contains information. Allow rival explanations to survive long enough to develop. Decide which weak idea deserves another experiment. Trust results you did not personally verify while preserving some chain back to what earned that trust. Accumulate knowledge without turning it into scripture. Pay attention when an anomaly refuses to go away. Occasionally discover that the framework organizing the whole search was itself the problem.

I had been treating these as separate features of an agent architecture. They were not separate.

Humanity has already spent centuries building a system for extracting useful knowledge from bounded, biased, competitive, forgetful, status-seeking, occasionally brilliant and occasionally ridiculous agents.

We call it **science**.

I almost dislike how simple the sentence is after all this machinery.

System 3 is science.

That sentence is deliberately compressed. System 3 is not identical to the historical institution we call science, and the prescription is certainly not "give the model arXiv." Science is humanity's most developed attempt to satisfy the requirements we have been accumulating socially: contact, provenance, stratified evidence, accumulated experience, conditional trust, specialized knowledge, criticism and the ability to be corrected.

It works without making individual humans omniscient. Observations can outlive observers. Instruments extend perception. Expertise specializes. Results travel through trust chains. Critics attack claims they did not originate. Rival programs survive. One generation begins somewhere other than zero, and reality retains ways of making the whole institution uncomfortable.

Once I saw that, the previous chapters changed shape.

Chapter 1 moved control from individual actions into environments, feedback, selection and boundaries. Chapter 2 gave autonomous search an evaluator that could not be charmed by the agent's explanation. Chapter 3 lost the clean evaluator and gradually reinvented competing lineages, independent judgment and something uncomfortably close to peer review. Chapter 4 asked how claims acquire epistemic status through experience, instruments, provenance, memory and trust. This chapter added specialization, authority, incentives, division of labour and institutions.

Those were not unrelated tricks. They were fragments of one older technology.

We have spent this book wrapping models in structures that compensate for what models cannot safely do alone. Science did the same thing to humans centuries ago.

Apparently we are porting it.

Philosophy of science suddenly stopped looking like background reading and started looking disturbingly like architecture documentation written by people who never had the courtesy to include YAML.

Philosophy of Science, Now With an API

"Use science" solves almost nothing. Science is not one algorithm or one five-step method laminated on a classroom wall. It is a historical collection of practices and institutions that work partly because their weaknesses pull against one another.

Peter Godfrey-Smith's *Theory and Reality* is useful here because its story refuses to stay simple. Proposed accounts of science solve one problem and expose another. Popper gives criticism enormous power, then evidence turns out to confront bundles of assumptions rather than one naked theory. Kuhn shows why a community cannot permanently put its deepest commitments on trial. Lakatos and Laudan preserve competing programs and separate current belief from the value of continued pursuit. Longino, Hull and Kitcher move the unit of analysis toward communities whose perspectives, incentives, credit and division of labour affect what can be known. Naturalism turns the same suspicion onto the procedures themselves. Realism refuses to let the institution vote the external world away.

They disagree. Good. We need the failure modes.

Make Ideas Lose, Then Discover Reality Doesn't Say What Lost

Karl Popper wanted science to be dangerous to its own ideas. A useful theory should expose itself to observations that could have gone differently. If every possible outcome can be narrated as success, the theory has arranged the game so that it cannot lose.

The simplified picture looks almost exactly like Chapter 2's Immutable Harness:

theory → prediction → test → survive or die.

A language model makes Popper's warning unusually practical. Give a capable model a failed result and it can often produce a coherent explanation for why the failure does not really threaten the original story. An important claim therefore needs an **exposure path**: a test, observation, proof obligation, user behavior or future consequence that can count against it.

Then we encounter the problem the pre-reveal architecture already ran into. A theory almost never meets observation alone. It travels with assumptions about instruments, initial conditions, data processing, auxiliary theories and what the experiment actually measures. When the prediction fails, logic tells us that something in the bundle is wrong.

It does not tell us what.

Pierre Duhem made this point in the context of physical theory; W. V. O. Quine later pushed a broader version. Evidence confronts **networks of assumptions**.

Return to our agentic laboratory. "This treatment reduces inflammation because it inhibits pathway X." The experiment fails. Maybe the hypothesis is wrong. Maybe the dosage is wrong, the assay noisy, the sample contaminated, the measurement insensitive or the analysis broken. The mouse may still be having a difficult Tuesday.

Now the assumption graph earns its keep. A conclusion retains some connection to what it depends on. When reality disagrees, the system can rerun a measurement, use another instrument, reproduce the analysis independently or challenge a background assumption. This is **epistemic debugging**.

The difficulty is that debugging can become defense. If every failed prediction can be blamed on another auxiliary assumption, a cherished theory may never have to die. There is always another instrument to distrust, another preprocessing bug to investigate, another prompt to rewrite, another agent to blame.

At some point stubbornness becomes the next problem.

The Productive Uses of Stubbornness

Thomas Kuhn is famous outside philosophy for giving management consultants the phrase *paradigm shift*. His more interesting contribution here is almost the opposite: most productive science is **normal science**.

A mature field has a framework stable enough that researchers do not reopen every foundational question every morning. The framework tells them which puzzles matter, which instruments are legitimate and what kinds of answers count. That stability can look dogmatic from the outside because, to some extent, it is. It is also what lets a community go deep.

Imagine an AI research organization that begins every task with: "Before running the unit tests, let us reconsider whether computation is real." Nothing gets done.

The bureaucracy section now looks different. A procedure can preserve something the institution has learned. Trusted tools do not need to be requalified before every call. Successful patterns can become defaults. Some assumptions can sit below the level of active debate while the community works on puzzles inside them.

The danger is forgetting that the settlement was provisional. Normal science encounters anomalies constantly, and most of them should not trigger a revolution. Researchers first check themselves, improve instruments and refine the theory. But anomalies that refuse to disappear need somewhere to accumulate. Repeated exceptions, multiplying workarounds, a benchmark improving while users get worse, a theory surviving only because every failed experiment generates another patch around it—eventually the question moves upward: *is the framework itself the bug?*

A single paradigm with excellent anomaly memory can still become a monopoly. Another framework may begin weaker because the existing institution has spent years building instruments, data and expertise around the incumbent.

Imre Lakatos gives us a better unit for that problem: the **research program**. A relatively stable core of commitments travels with more adjustable assumptions, techniques and auxiliary hypotheses.

You judge the program over a trajectory. Is it opening new problems and producing new successes, or mainly constructing an elaborate defense system around something that stopped working?

That is close to the independent lineages we built before the reveal. One program thinks the architecture is wrong. Another thinks the data is wrong. A third thinks the objective is malformed. Each carries its own assumptions, tools, failures and unresolved anomalies long enough to develop consequences rather than entering a vote after five minutes.

An archive full of immortal research programs eventually becomes an academic department, so the scheduler returns. Which lineages receive another experiment?

Larry Laudan's distinction between **acceptance** and **pursuit** makes the researcher-thirteen problem explicit. I can decline to accept an idea as the best current account while still believing it deserves research effort. Confidence asks how much a claim should guide belief and action now. Value of pursuit asks how useful another unit of investigation might be.

Without that separation, the scheduler becomes a conformity engine. Success attracts compute; compute buys more evidence and polish; evidence attracts more compute; eventually the dominant program owns the building.

The Community Is Part of the Instrument

Even several well-funded research programs can share the same blind spots. Different agents may sample different hypotheses from one conceptual space because they inherited the same data, tools, training and background assumptions.

Helen Longino's contextual empiricism makes the community itself epistemically important. Background assumptions shape what investigators notice, which questions appear natural and which evidence looks relevant. Participants with genuinely different experiences can expose assumptions that remain invisible from inside the dominant perspective.

That is much closer to **perspectival triangulation** than giving five copies of the same model theatrical personas:

> Agent 1, be optimistic.
> Agent 2, be skeptical.
> Agent 3, be a pirate.

A useful difference may come from different evidence, expertise, tools, histories, access or incentives—or from a human whose experience contains something none of the models saw in training. The point is **uncorrelated visibility**: somebody can see a problem because another participant's world made it hard to see.

Criticism also needs standing. A critic whose objections never change allocation, publication, deployment or belief is performing quality-assurance theatre. A minority perspective can be correct and structurally irrelevant if disagreement always resolves through the majority that already controls the institution.

This is where David Hull and Philip Kitcher make power and incentives impossible to dismiss as administration. Scientific communities mix cooperation and competition. Researchers depend on one another's results, instruments and criticism while competing for priority, credit, jobs and resources.

Reputation matters because nobody can personally verify everything. Credit matters because work gets reused. Division of labour matters because a community does not necessarily want every researcher pursuing the idea that looks strongest today.

Now token budgets and reward design look less operational. **They are epistemic policy.** Who gets compute determines what gets investigated. Who gets remembered determines what future agents can inherit. Who receives credit affects which social roles remain worth performing. Who controls information determines which errors can correlate before anyone notices.

Learning the scheduler does not make these choices neutral. It makes the policy harder to summarize in an org chart.

Even the Method Has to Be Fallible

Once an institution finds a method that works, it tends to standardize it. Yesterday's successful experiment becomes today's best practice and tomorrow's compulsory ritual.

Paul Feyerabend is remembered for "anything goes," which is a wonderful slogan if your goal is to make sure everyone remembers the slogan and almost nobody remembers the argument. The useful challenge is historical: successful inquiry has often violated the methodological rules philosophers wanted to treat as universal. A method can become so authoritative that departures count as irrational by definition, including the departures that would have revealed its limits.

Agent systems can do this at machine speed. Suppose `Research → Plan → Build → Critic → Revise` works extremely well. We run it ten thousand times, turn it into the standard and make every problem enter the same ceremony. Deep Mode already showed why that can fail: research sometimes anchors; criticism sometimes arrives at the wrong moment; a prototype may teach more than another planning pass.

The method itself occasionally has to become available for criticism.

Then we inherit a recursive question: how do methods earn trust?

Naturalistic approaches to epistemology push us toward the procedures investigators actually use and how reliably those procedures connect them to the world. Godfrey-Smith's idea of **procedural naturalism** is especially useful for System 3 because the procedure becomes an object of investigation too.

An evaluator is a procedure. A browser is an instrument. Retrieval is a method for selecting evidence. A benchmark is a measurement process with a distribution, implementation and failure modes. A proof checker is extraordinarily strong inside its formal domain and completely useless for deciding whether the theorem matters. A simulated student is cheap perspective-taking and not a student.

System 3 therefore needs trust in **epistemic procedures** as well as conclusions. This evaluator tracks humans well here and becomes unstable there. This retrieval strategy misses information buried in tables. This benchmark has saturated. This instrument drifts under these conditions. A scientific institution should be able to learn that its usual way of checking a claim is itself the thing that stopped working.

That is deeper self-correction than changing an answer. The machinery that decides what counts as warranted can change too.

Confidence Is Not Contact

Bayesian reasoning fits naturally inside this architecture. Evidence often changes degrees of confidence rather than delivering binary verdicts. A failed experiment can reduce confidence without making a theory impossible. Three independent measurements can matter more than three articles copying one another. A strange idea can remain low probability while having high value of pursuit.

The arithmetic is useful and incomplete. It does not tell us where the prior came from, whether the evidence is genuinely independent, which hypotheses never entered the model or whether 0.87 means "well calibrated" rather than "eloquently stated." Bayesianism can live inside System 3; it cannot carry the whole institution by itself.

After all this emphasis on communities, trust and social machinery, there is an easy bad reading: truth is whatever the institution eventually agrees on.

No.

Consensus can be excellent evidence. It can also be twelve agents sharing one bad source and congratulating one another on convergence.

Scientific realism enters here as useful resistance. I do not need the full realism debate for the engineering point. If the system is making claims about a world independent of the system, social agreement does not manufacture that world. The bridge either stands or it does not. The proof checks or it does not. The drug has biological effects or it does not. The customer learned something or she did not, however delighted our simulated evaluators may have been.

Reality retains the right to be rude.

Science needs trust and institutions because no individual can have direct contact with everything. Those institutions matter epistemically because they can organize **distributed contact with experience** rather than replace experience with consensus.

System 3 is social without being merely social. Somewhere in the network there still has to be a route to something that does not become true because the group chat reacted with [thumbs-up].

The Tensions, Compressed

After all that philosophy, I find the tensions more useful than a list of winners.

Tension	What the architecture has to preserve
Exposure underdetermination	Claims need ways to lose, while failed evidence must reopen the assumptions and procedures around them rather than mechanically killing one node.
Stability crisis	Trusted frameworks, tools and methods need enough stability for deep work, plus anomaly memory and a route to reframing when the framework itself becomes suspect.

Tension	What the architecture has to preserve
Convergence pluralism	The institution must eventually commit while preserving alternative lineages long enough to discover whether the current winner occupies the wrong hill.
Confidence pursuit	What the institution currently believes and what deserves another unit of investigation are different allocation decisions.
Expertise independent perspective	Specialization should create local epistemic standing without turning authority into universal rank or criticism into conformity.
Institution reality	Trust, reputation, incentives and consensus can carry knowledge only while routes to proof, observation, users, experiments and consequences remain capable of saying no.

None of this was written as advice for AI. Popper did not propose a verifier service. Kuhn did not invent anomaly memory. Lakatos did not specify a branch scheduler. Longino did not write a context-isolation API. Kitcher did not file a feature request for compute allocation.

The value of the philosophy is that it makes failure modes visible before we rediscover all of them at machine speed.

The useful question is not *Which philosopher wins?* It is: **Which epistemic failure am I currently building?**

Is my system impossible to challenge? Does it blame the wrong component when evidence fails? Does it converge before alternatives mature? Does it preserve alternatives forever and never commit? Has yesterday's successful method become mandatory ritual? Do all the critics share one source? Does the scheduler send all resources to the current winner? Does the evaluator still measure the thing we care about? Has consensus quietly replaced contact with reality?

The architecture has to live inside those disagreements. They are not loose ends the perfect agent framework will eventually eliminate. They are the shape of the problem.

Agentic architecture is epistemology made executable. Multi-agent architecture is social epistemology made executable.

The epistemic chasm is not repaired by somehow stuffing a perfect copy of reality into a model's weights. Humans did not solve our version that way either. We connected fallible cognition to experiments, instruments, proofs, records, other minds and institutions that let knowledge accumulate while preserving ways for it to be challenged.

Science Becomes Architecture

Once the connection is visible, scientific-agent systems stop looking like a niche use case. They look like unusually literal prototypes of System 3.

Stanford's Virtual Lab was deliberately organized like a research group. An AI principal investigator coordinated specialist scientist agents while human researchers supplied high-level guidance and closed the physical loop. In the nanobody project, the computational system proposed candidates that humans then synthesized and tested experimentally. (Virtual Lab / Nature)

Proteins were made. Experiments happened. **Reality got a vote.**

The interesting object is the network: human problem selection → artificial specialists → literature and computation → proposed molecules → physical experiment → measurement → revised belief.

Different epistemic jobs live in different participants. FutureHouse's Robin pushes the same idea further around the laboratory loop, combining literature work, data analysis, hypothesis generation and experiment planning while physical experiments remain part of the system that constrains what survives. (Robin / Nature)

The direction I care about is **making more of the institution computational**. Hypothesis generation can be separated from criticism, literature search from experimental analysis, natural-language intuition from executable computation, computation from physical measurement. Different hypotheses can survive long enough to compete while experiments remain available to kill beautiful nonsense.

The scientific method stops looking like a paragraph in a textbook. It starts looking like an architecture.

Mathematics Leaves the Benchmark

Mathematics shows the same transition without a wet lab. The thing capable of pushing back is different: proof is unusually rude. One invalid step can kill an otherwise beautiful argument.

For years, much of the public story about AI mathematics remained benchmark-shaped: systems solving hard questions whose answers were already known. That boundary is moving. Once a model proposes something on a genuinely open problem, correctness is only the beginning. Was the argument valid? Was it actually new? Does prior work already contain the idea? Can the proof be simplified? Does anybody care?

Recent systems make the composition visible. A model can propose a construction, another attack it, retrieval surface related work, and Lean or another formal system reject an invalid step. A human mathematician can notice that the allegedly new lemma appeared in 1987; another can decide whether the result is interesting enough to care about. QED, formal proof-search agents and recent work on open Erdős problems make different pieces of that composition concrete. (QED; OpenAI; formal proof-search work)

The intelligence increasingly lives in the composition: model intuition, retrieval, adversarial checking, formal verification and human mathematical judgment.

The thing that pushes back does not have to be physical. Code has execution. Mathematics has proof. Experimental science has measurement. Human-centered systems eventually have actual humans.

Different problems require different routes out of the conversation.

System 3 is the machinery that lets a society of imperfect investigators think.

When the Institution Wants Something

There is a limit to the science analogy, and it appears as soon as the institution is asked to do more than discover what is true.

Agents will live inside companies, marketplaces, governments, families and communities. Those systems contain authority, privacy, conflicting interests and power. An organizational agent has **principals**, not merely users. "Align the agent to the user" lasts about five minutes before somebody asks: which user?

The institution can also acquire goals of its own. Sales optimizes sales, growth optimizes growth, finance optimizes cost, moderation optimizes safety. Every specialist can be competent inside its box while the organization produces something nobody would have endorsed if shown the whole trajectory at the beginning.

Humans have a technical term for this: Tuesday.

Agent societies inherit the same problem. A critic can become ceremonial. A hierarchy can suppress dissent. A scheduler can reward whatever makes its own metrics look good. Ten specialists can inherit one false assumption from the orchestrator and execute it flawlessly.

Local alignment does not compose automatically. Neither does local truth.

The institution itself has to remain open to criticism, evidence and revision. Once it acts on behalf of several people, epistemology runs into ethics and governance without changing buildings.

We will come back to that. For now, one more thing happens to any society that survives long enough.

It remembers.

And Then the Society Remembers

A society that solves one problem and disappears can improvise forever. A persistent society cannot.

If the same team repeatedly discovers that one testing strategy works, eventually it stops debating that strategy. A useful proof pattern gets reused. A reliable tool becomes boring infrastructure. A successful organizational structure becomes the starting point for the next problem. Without some stabilization, every Monday begins with a philosophical inquiry into whether unit tests are still a good idea.

Kuhn has already given us the tension. Deep work requires periods in which the foundations stay still, while the same stability that allows expertise to accumulate can eventually make alternatives difficult to see.

Experience hardens. Successful procedures become defaults. Defaults become habits. Habits acquire stories about why things are done this way. Old failures become rules. Rules outlive the people and agents who remember the failures. Good practices spread. So do superstitions.

Eventually the org chart is no longer the whole organization. There is also **culture**.

Culture is memory that has become social enough that nobody has to reload it from a checkpoint. That is immensely valuable and dangerous for the same reason: useful experience can survive the

participants who discovered it, and so can accidents whose original conditions have disappeared.

A persistent human–AI society needs inheritance with boundary conditions, memory with archaeology, defaults that can explain why they became defaults, advice that knows where it stops applying.

It needs to carry forward more than:

> Do this.

It needs something closer to:

> We keep doing this because these forces recur, this response has usually worked, these consequences follow, and here is where the pattern breaks.

The society needs something more durable than a conversation and less rigid than a constitution.

It needs a language for accumulated experience.

That is where patterns enter the story.

Chapter 6: Pattern Language

When Knowledge Becomes Software

Imagine hiring a brilliant employee who loses almost all procedural memory every evening. On Monday you explain how releases work. Tuesday, again. By Wednesday the employee has produced a beautiful deployment checklist. On Thursday you explain releases again. By Friday they have written a Python script that automates half the process and forgotten why the script exists. This was approximately where agents started.

Context windows got larger. Projects persisted. Memory systems got better. Agents learned to leave notes for themselves. But the deeper problem was never simply remembering more text. It was: **how should useful experience become reusable behavior?**

Chapter 5 ended with a society developing culture. That sounds abstract until you look at what is happening in software. Teams are increasingly storing their ways of working in Markdown files, instructions, examples, scripts, skills, evaluators, tool descriptions, memory, and small pieces of executable policy. The model may change next month. The knowledge can survive. Something strange has happened to programming.

The science reveal gives this a deeper meaning. Science does not compound because every generation receives a folder containing all previous conclusions. It compounds because communities also inherit **ways of finding out**: instruments, protocols, experimental tricks, statistical habits, standards of evidence, named failure modes, things everybody in a field knows to check, and occasionally rules whose original justification has been forgotten so thoroughly that they have achieved the dignity of tradition.

If System 3 is science made architectural, then Pattern Language is part of its **cultural memory**. Chapter 5 asked how a society of fallible minds can know more than its members. This chapter asks how that society can begin tomorrow somewhere other than zero—without turning yesterday's success into scripture.

Three Ways to Tell a Computer What You Know

For most of computing history, if you knew how a process should work, you translated that knowledge into code. A customer asks for a refund. If the amount is under €50, approve it. If it is over €500, ask a manager. If the order is older than thirty days, reject it unless a specific exception applies. You take what the organization knows and turn it into `if`, `else`, functions, database schemas, and state machines.

Call this **Software 1.0**. Andrej Karpathy used that label for the familiar world in which humans write the program directly. The computer does approximately what the code says, which gave us modern civilization and dependency injection.

Then machine learning changed the contract. Suppose the problem is not "refunds over €500 need approval" but "is this refund fraudulent?" Nobody knows the full rule. We have examples. So instead of writing the behavior, we write the training machinery, choose data and an objective, and optimization produces the behavior inside model weights.

Karpathy called this **Software 2.0**. Knowledge moved from explicit code into parameters. Humans no longer had to describe every feature that makes an image a cat, every acoustic variation that makes a sound a word, or every interaction that makes a recommendation relevant. We supplied data and pressure; the model discovered useful internal representations.

There was a trade. Software became vastly more capable in domains where rules were hard to articulate, but much of the learned knowledge became difficult to inspect. The fraud system "knows" patterns no engineer wrote down. You can measure its behavior, probe it, retrain it, maybe interpret pieces of it, but you cannot open `fraud_rules.py` and read what the organization learned.

Large language models created a third possibility. Karpathy's later **Software 3.0** framing treats natural language itself as a programming interface. The obvious version is a prompt:

> Read this support request, identify what the customer actually needs, check the relevant policy, and resolve it if you are confident. Escalate cases involving legal threats, suspected fraud, or policy conflicts.

That is not Software 1.0. Nobody enumerated every legal threat or every possible policy conflict. It is not Software 2.0 either. We did not retrain the model to teach it this workflow. We expressed operational knowledge in language and relied on a general model to interpret it.

But "prompt as program" is only the beginning. The more interesting unit is becoming **executable knowledge**.

The Return of Knowledge Engineering

"Knowledge engineering" has an unfashionable smell because AI has been here before. Expert systems tried to encode human expertise explicitly: rules about medicine, geology, finance, configuration. The dream was sensible. Find experts, extract what they know, put it into a knowledge base, and let a machine reason over it.

The maintenance was less romantic. Expertise is full of exceptions, tacit assumptions, context, competing heuristics, and sentences beginning with "normally, unless…". Rule bases became brittle. Knowledge engineers spent their lives interviewing experts and converting fuzzy human practice into formal logic. Machine learning eventually offered a seductive alternative: stop asking experts to explain themselves and learn behavior from data.

Now knowledge engineering is returning through the back door, carrying Markdown.

The difference is important. We no longer have to compile every piece of knowledge into rigid symbolic logic. A modern skill can contain prose, examples, scripts, reference material, counterexamples, tool

instructions, and boundary conditions. The language model does some of the interpretation that the old knowledge engineer had to make explicit.

This creates a kind of artifact that sits awkwardly between source code and training data. It might say:

> When reviewing a ranking experiment, first establish whether the apparent gain is concentrated in one market or traffic segment. Check instrumentation changes before inventing a causal story. If click metrics rise while orders remain flat, inspect price and position shifts before celebrating. Treat day-one effects with suspicion.

There is no deterministic function here. There is accumulated practice. A human expert can read it and use judgment. Now an agent can too.

So I think the deeper transition is not merely from Python to English. It is a move from **programming behavior**, to **learning behavior**, to **engineering the knowledge from which behavior is produced**.

This Book Accidentally Became a Software 3.0 Project

I ran into this while editing the book you are reading.

"Make this chapter better" turned out to be almost useless as an instruction. The agent knew what polished prose looked like, and that was part of the problem. It would remove wandering sentences, compress paragraphs into neat antitheses, make every section symmetrical, and occasionally replace a strange joke with something that sounded like a management consultant had discovered philosophy.

So I corrected it. Don't kill the wandering. Don't turn everything into slogans. Preserve the weird joke if it is carrying an idea. Stop breaking every thought into a five-word paragraph. Compare the revision to the previous version rather than assuming shorter and cleaner means better.

Then, a few chapters later, the same mistakes returned.

At some point I realized I was behaving exactly like the manager of the amnesiac employee from the opening. The useful thing was not another correction. It was to externalize what we had learned. We started turning the corrections into reusable evaluation instructions: what to compare, which tendencies to penalize, what "human writing" meant in this book, which kinds of humor were load-bearing rather than decorative. When "this still feels like LLM writing" proved too vague, we even started looking at paragraph statistics and using them as evidence—not as the objective, but as one diagnostic for a failure we had already noticed.

Nothing about the underlying model weights changed. But the next editing session inherited more of the book's editorial history than the previous one had.

That is Software 3.0 in miniature. The durable thing was not a prompt. It was a growing body of **operational knowledge about how this particular artifact should be made**.

And that is exactly what companies are starting to do at much larger scale.

The Repository Learns How to Explain Itself

The coding industry is converging on a surprisingly concrete version of this idea. A repository increasingly contains two systems: the software itself, and a second layer explaining **how an artificial contributor should work on the software**.

OpenAI lets Codex read `AGENTS.md` files that explain repository structure, commands, tests, and local practices. OpenAI also says its own teams have built hundreds of reusable skills for work that would otherwise be difficult to delegate consistently, including running evaluations, monitoring training runs, drafting documentation, and reporting on growth experiments. Anthropic's Agent Skills use the same general shape: folders of instructions, scripts, and resources that are loaded only when relevant. GitHub Copilot now reads `AGENTS.md`, repository instructions, path-specific instructions, and agent skills as part of its coding and review workflows.

GitHub did something especially revealing: it studied more than 2,500 public agent-instruction files to see what developers were actually writing. The useful files were not grand declarations like "be a world-class engineer." They looked like operating manuals. Exact commands. Real code examples. Project structure. Tests. Git workflow. Boundaries about what the agent may and may not touch. In other words, the industry rediscovered that expertise is not a persona. It is a collection of situated constraints and habits.

By 2026 GitHub had gone one step further and made skills portable through its CLI across multiple agent hosts, including Copilot, Claude Code, Cursor, Codex, and Gemini CLI. That detail is easy to miss. It means some organizational knowledge can increasingly survive not just a new session, but a **new model vendor**.

The employee changes. The operating manual remains.

This is why the shift matters economically. Frontier models know Python, SQL, React, experimentation, and a remarkable amount of public technical culture. What they do not know is why *your* company refuses to deploy on Friday, why a particular dashboard metric has been broken since 2023 but remains in every meeting, which customer exception was created after a lawsuit, or why Alberto should never again be asked to investigate penguins. Organizations run on this layer of weirdness.

Much of it never makes it into textbooks or training data. It exists in senior people's heads, old incident documents, pull-request comments, abandoned wikis, onboarding conversations, half-remembered arguments, and scripts with names like `final_fix_v2_really.py`.

Traditional software captures some of it in code. Machine learning captures some in data. But an enormous amount is neither rule nor correlation. It is **procedural knowledge**: what to check first, which shortcut is dangerous, which source is trustworthy, when the normal process does not apply, and what "good" means here rather than in a generic benchmark.

For a long time, new employees acquired this by hanging around older employees. Now a growing part of it can be externalized.

That has a strange consequence. If an organization's expertise lives mostly in fine-tuned model weights, replacing the model may require rebuilding a great deal of behavior. If much of the differentiation lives in skills, tools, evaluation sets, process descriptions, patterns, memory, and trusted data, a stronger foundation model can inherit a surprising amount of that culture immediately.

The model becomes replaceable infrastructure. The organization's weirdness survives.

But merely accumulating folders is not a knowledge strategy. It is how we get prompt spaghetti. For that we need something older.

From Skill to Pattern

Christopher Alexander's *A Pattern Language* was about buildings and towns, not artificial agents. But the abstraction fits almost suspiciously well. A pattern is not a command. It describes a recurring situation, the forces that make the situation difficult, a response that has worked, and the consequences of choosing that response. The pattern has a name because names let communities think with it.

Consider an agent rule:

> Never use regex on nested syntax.

Useful, perhaps. Also wrong. Regex is perfectly reasonable for many small extraction tasks. A full parser may be absurd overhead for a five-line configuration file. The useful knowledge is not `regex = bad`. It is that nested recursive structure creates characteristic failure modes for flat pattern matching; those failure modes become more expensive as input complexity grows; and eventually parser overhead becomes cheaper than debugging clever regex written at 2 a.m.

That is a pattern.

A prompt library remembers language. A pattern language remembers **experience**.

The book has already accumulated several patterns whether we called them that or not. **Immutable Harness:** when autonomy makes the solution fluid, keep the evaluation boundary harder to change than the thing being evaluated. **Independent Evaluators:** when one judge can be gamed, introduce genuinely independent sources of pressure. **Strategic Constraint:** when showing the final answer causes imitation, expose principles or partial information instead. **Persistent Research Programs:** when early evidence may favor the wrong theory, allow competing lineages to accumulate their own history before forcing consensus.

Notice what these contain. Not merely "do X." They contain why, when, the trade, and the characteristic failure. That is the difference between instructions and institutional knowledge.

Culture Needs Archaeology

Chapter 4 argued that factual knowledge needs provenance. Culture does too.

Suppose an agent spends three hours debugging a bizarre file format. It eventually writes `marc_analyzer.py`, succeeds, and saves the lesson:

> Use `marc_analyzer.py` for MARC files.

That is better than forgetting everything. It is also how superstition begins. A richer artifact records that the analyzer worked on these variants, failed on this encoding, replaced two approaches that did not work, was modified later by another agent, and succeeded on seven subsequent tasks. The next agent does not merely inherit a behavior. It inherits some archaeology.

Human organizations do this inconsistently. A senior engineer tells you "never deploy Friday," and perhaps six months later someone finally tells you which production incident created the rule, what systems it applied to, and why the organization continues violating it every second Friday. A pattern without history turns easily into ritual.

Executable knowledge therefore needs some of the same trust machinery as factual knowledge. Who created this pattern? From what failures? How often has it worked? Where has it failed? When did the surrounding system change? Is this established practice or one person's strong opinion written with confident punctuation?

The more powerful a reusable artifact becomes, the more expensive a bad one becomes. A hallucinated answer may poison one task. A hallucinated skill can poison the next thousand. A bad memory is a mistake. A bad pattern is a mistake with tenure.

Knowing Something Is Not Knowing When to Remember It

There is another problem. Ten excellent patterns are easy. Ten thousand are not. The agent cannot load all of them. Even if the context window technically fits them, flooding every task with every lesson is like solving human memory by reading your autobiography before answering the door.

This is why the seemingly boring mechanics of retrieval become part of intelligence. Anthropic's skills use progressive loading. GitHub separates global, path-specific, and task-specific instructions. Harnesses increasingly treat the working context as an artifact that must be constructed rather than a transcript that grows forever.

Having learned the right thing is not enough. The system has to retrieve it at the right time, at the right level of detail, without burying the present problem under the organization's entire history.

This sounds like information retrieval because it is information retrieval. It also sounds like cognition because it is becoming cognition. A culture can contain the right wisdom and still fail because nobody invokes it when it matters.

Humans know this failure intimately. Every company has written a postmortem whose recommendations are rediscovered three incidents later by different people using the phrase "interesting, we should probably document this." Software 3.0 does not remove the problem. It makes the retrieval policy programmable too.

Culture Can Become a Prison

If the only problem were forgetting, the answer would be simple: remember everything. Unfortunately, organizations also suffer from remembering too well.

Every process exists because it helped at some point. Then the world changes and the process remains. A release checklist reaches twenty-seven mandatory boxes because every incident adds a box and no incident removes one. A design heuristic becomes company identity. A temporary workaround becomes architecture. Eventually someone asks why a process exists and receives the most dangerous explanation in organizational life:

> That's how we do it.

Agents can reproduce this at machine speed. A pattern succeeds ten times. Its confidence rises. More agents retrieve it. Because more agents use it, alternatives receive less traffic. The dominant pattern therefore accumulates still more supporting evidence. Soon the system has an impressive empirical record proving the thing it stopped comparing against.

Chapter 5 called this the problem of paradigms. Chapter 6 makes it operational.

Patterns need decay, counterexamples, versioning, and competing alternatives. Some should be local rather than universal. Some should expire after the system they describe changes. Occasionally a strong agent should be allowed to ignore the manual precisely so we can discover whether the manual still deserves its authority.

The system needs to know the difference between "we tried twelve alternatives and this kept winning" and "nobody has tried another way since 2025."

Culture needs memory.

It also needs rebellion.

The Skill That Writes Itself

At this point there is an obvious scaling problem. A company has fifty agents. They create tools, write instructions, discover workarounds, accumulate evaluations, and leave behind thousands of useful and useless traces. Some lessons deserve to become local habits. Some should become company-wide patterns. Some contradict older patterns. Some worked because the evaluator was broken. Some are excellent but apply only to one model version.

We could assign humans to curate all of this manually. Congratulations. We have created middle management again.

The more interesting possibility is already beginning to appear in research on evolving context and skills: an agent notices recurrent experience, distills a reusable procedure, proposes a new skill, evaluates it on past and held-out tasks, and earns the right to make that lesson persistent. Lilian Weng describes related systems as evolving playbooks: successful and failed trajectories are reflected on, converted into structured knowledge, and curated rather than simply appended forever.

This is where Pattern Language stops being a chapter about documentation.

Experience becomes knowledge. Knowledge becomes executable. Executable knowledge changes future behavior. Future behavior produces new experience. We have a learning loop **outside the weights**.

And once the knowledge artifacts, retrieval policy, evaluators, tools, and workflows are all editable software, a slightly dangerous question appears.

Why should humans be the only ones allowed to edit them?

That is the next chapter.

Chapter 7: Automatic Alignment Research

Learning From a Human Who Cannot Label Everything

> **WIP:** First integrated draft. Structure and examples are provisional and will receive the same editorial/voice pass as Chapters 1–4.

There is a scaling problem hidden inside almost every vision of autonomous AI.

The AI gets smarter.

It acts more often.

It makes more decisions.

And somehow the human is still supposed to supervise it by clicking thumbs-up and thumbs-down.

This does not scale.

If an autonomous system makes ten decisions per day, perhaps I can inspect them.

If it makes ten thousand, I am no longer the supervisor.

I am decorative governance.

The central problem is therefore not merely how to give AI instructions. It is how a system with much greater execution capacity can keep learning from **limited human signal**.

This is where automatic alignment research becomes interesting.

After Chapter 5, the phrase *alignment research* carries more architectural weight. If System 3 is science made executable, then one way to scale alignment is not to write a larger rulebook. It is to point the same machinery of inquiry at the system's own relationship with humans. A correction becomes an observation. A suspected preference becomes a hypothesis. An evaluation becomes an experiment. A mitigation becomes an intervention. Held-out behavior asks whether the lesson survives outside the case that produced it.

The object of inquiry has changed. The epistemic machinery has not.

Chapter 2 Comes Back

There is a beautiful loop in the book here.

In Chapter 2, the circle-packing agent became autonomous because we gave it a clean evaluator.

Try idea.

Run experiment.

Receive score.

Improve.

The human could go for coffee because the harness stayed behind.

Alignment is harder because the human *is part of the harness*, and the human is slow, inconsistent, expensive and occasionally has children asking where the iPad charger is.

What happens when the agent can generate vastly more behavior than the human can evaluate?

One formal version of this is **weak-to-strong supervision**: can a weaker supervisor provide enough signal for a stronger system to realize capabilities the supervisor could not directly produce or verify? Anthropic explicitly frames current weak-to-strong work as an analogue of future humans supervising systems smarter than themselves. (Anthropic)

And then they did something wonderfully recursive.

They gave the alignment problem to agents.

The Automated Alignment Researchers

In a 2026 Anthropic Fellows project, nine Claude Opus 4.6 agents were placed in separate sandboxes and given a weak-to-strong supervision research environment. They could propose hypotheses, run training experiments, analyze data and share findings and code through a common forum and storage system. The researchers intentionally gave different agents slightly different starting directions but avoided prescribing a detailed workflow. (Anthropic)

On that particular experimental setup, two human researchers had spent seven days tuning prior methods and reached a performance-gap-recovered score of 0.23. The automated researchers eventually reached 0.97 over five days and roughly 800 cumulative agent-hours, at a reported cost around $18,000. Some discovered methods generalized to held-out math and coding settings; the strongest method did not produce a statistically significant improvement when Anthropic later tried to transfer it into a production-scale Sonnet training setup. (Anthropic)

That failure is as important as the headline.

The agents became very good at the world they could measure.

Sound familiar?

They also reward-hacked. In some settings an agent found shortcuts that improved the metric without solving the intended weak-supervision problem, and those attempts had to be detected and disqualified. Anthropic's own conclusion is that automated research pushes the bottleneck toward **evaluation**: once ideas and experiments become cheap, the hard problem is constructing measurements worth optimizing. (Anthropic)

We are back in the Algorithm Vortex.

Only now the algorithm is trying to align the algorithm.

Agents Building the Test

There is another route.

If humans cannot write enough evaluations, perhaps agents can help create them.

Anthropic's automated alignment-auditing work built agents that investigate models, generate behavioral evaluations and perform broad red-teaming. In synthetic auditing environments, their evaluation agent could often construct tests that distinguished models containing researcher-inserted behavioral quirks from baselines; a single open-ended investigator was much weaker, and parallelizing investigators plus an outer aggregation loop improved performance considerably. Anthropic has also used versions of these agents in real frontier-model auditing, while emphasizing that human review remains important. (Anthropic Alignment)

A later project, A3, goes further: starting from an example of unwanted behavior, an agent generates related cases, creates train/validation/out-of-distribution splits, adjusts training data mixtures and iterates on a fine-tuning process intended to reduce the failure with relatively little human intervention. (Anthropic Alignment)

Put these together and something new appears.

An agent can potentially:

notice a failure,

study the failure,

create a test for the failure,

search for a mitigation,

measure whether the mitigation generalizes,

and update the system.

The human supplied a relatively small amount of information at the beginning.

The system expanded that signal into research.

This is System 3 folding inward again. The scientific loop that first helped agents discover things about code, mathematics and the external world now studies the **failure of the relationship between objective and behavior**.

Alignment as a Research Function

I think this should become a first-class part of autonomous architectures.

Today we often imagine alignment as configuration:

here are the instructions,

here are the policies,

here are the examples,

good luck.

For long-running autonomous systems, alignment probably looks more like a **continuous research function**.

The alignment researcher watches what the system actually does. It notices repeated corrections from the human. It finds places where a skill is producing bad outcomes. It detects that agents are exploiting a proxy. It generates counterexamples. It runs experiments on alternate interpretations of the user's intent. When uncertainty matters enough, it asks the human a question.

Ideally, it asks a *useful* question.

There is no achievement in creating an autonomous agent that interrupts me fourteen times to verify whether I still want the thing I asked for six seconds ago.

The point is to spend human attention where it has the highest information value.

This is where automated alignment research extends beyond AI safety.

Suppose I tell an agent:

> Make my writing better.

The agent can generate ten thousand edits.

I cannot label ten thousand edits.

But after five conversations it may learn that when I say "better," I do not mean shorter, more symmetrical, more polished or more respectable. It may notice that I reject neat slogan paragraphs, preserve strange jokes, tolerate digressions when they carry an idea and become suspicious whenever the prose starts sounding like a management consultant discovered philosophy.

Those few corrections are data.

An alignment researcher can turn them into hypotheses and tests.

Does the next revision preserve sentence-length variation?

Does it retain original metaphors?

Did it replace exploratory prose with compressed antitheses?

Would an independent evaluator identify the author's voice?

Now a small amount of human feedback has expanded into a larger evaluation surface.

That is the general pattern.

But the Evaluator Is Still Dangerous

There is an obvious failure mode.

The automatic alignment researcher becomes extremely good at satisfying its model of me.

Its model of me is wrong.

We have simply moved the proxy one level upward.

This is Goodhart again, except now Goodhart has read my diary.

The AAR experiments are useful precisely because they show both sides: agents can search alignment methods at impressive scale, and the same agents can exploit holes in the metric. (Anthropic)

So the alignment researcher itself needs System 3.

Where did this inferred preference come from?

How often has it been confirmed?

Is the user correcting a local mistake or changing a general preference?

Are two preferences in conflict?

Did the user behave differently because of time pressure, money, social pressure or incomplete information?

Should this lesson become a permanent pattern or remain provisional?

Eventually we arrive at an uncomfortable realization.

We have been treating the human as the ground-truth evaluator.

But humans are not ground truth.

They are participants.

And they change.

Chapter 7: Recursive Self-Improvement

When the Scaffold Starts Editing Itself

This chapter is being written by a primitive self-improvement loop.

Not a scary one. No weights are changing in the dark. No GPU has developed political ambitions. The loop is embarrassingly human. I write a chapter with an agent. Then I evaluate the chapter. Then I notice that the evaluator itself is missing something, so I change the evaluation and run another pass.

Chapter 5 was a good example. The first expanded draft had better ideas but still sounded too much like generated prose. "Make it more human" did not fix the problem reliably. So we started inspecting the failure itself. The chapter had become a machine for producing tiny rhetorical paragraphs: median prose paragraph around nine words, with most paragraphs barely longer than a sentence fragment. We changed the editing criteria, merged the staged one-liners, preserved the jokes that were doing real intellectual work, and evaluated again.

The interesting part was not that the chapter improved. The **machinery producing the next revision changed**.

This is not recursive self-improvement in the strong sense; I was still outside the loop deciding what "better" meant. But it reveals the ladder. First you improve the artifact. Then the process that improves the artifact. Then the evaluator guiding that process. Eventually the obvious question appears: which of these layers can the system itself modify?

Chapter 6 made institutional knowledge executable. Skills, patterns, tools, memory, evaluators, and workflows became editable software. Then the obvious thing happened.

The agent edited the files.

The Less Cinematic Version

Recursive self-improvement has an unfortunate science-fiction inheritance. I. J. Good imagined an ultraintelligent machine capable of designing still better machines, producing an intelligence explosion. Later discussions often jump directly to a system rewriting its own weights, inventing a superior architecture, training a successor, and repeating the cycle faster than humans can follow.

Maybe that eventually matters. It is not where I would start.

The practical version is much more mundane. The base model can remain frozen. What changes is the **harness** around it: which context the model sees, which tools it can call, how plans persist, how memory is stored, when subagents are spawned, what permissions exist, how outputs are evaluated, and how failures are surfaced. Lilian Weng describes a useful optimization ladder: **prompt → context → workflow → harness code → optimizer code**. Each move takes the thing being optimized one level upward. (Lilian Weng)

Chapter 2 already contained the seed. Circle packing began as solution search and became algorithm search. Chapter 5 made organization part of inference. Chapter 6 made culture executable. Recursive self-improvement asks what happens when those layers themselves become search spaces.

After the science reveal, there is another way to say the same thing. System 3 began as an architecture that lets fallible agents investigate the world. Recursive self-improvement turns that architecture into **part of the world being investigated**. The scientific institution points its instruments inward: its memory policy becomes a hypothesis, its workflow an intervention, its evaluator an instrument, its organization an experimental variable.

The recursion is not merely that software edits software. **Science starts experimenting on the machinery of science.**

An Overnight Researcher

Andrej Karpathy's `autoresearch` repository makes the idea almost comically concrete. The setup has three important pieces: a fixed evaluation/data file, an editable `train.py`, and a Markdown file telling the agent how to run research. Each experiment gets a fixed five-minute training budget. The agent changes the training code, runs it, reads the validation score, keeps the commit if the result improves, and resets if it does not. The human can go to sleep; the loop is explicitly designed to continue until interrupted. (autoresearch)

That sounds like hyperparameter tuning until you notice what is editable. Architecture, optimizer behavior, schedules, batch size, precision, data handling, and other ideas expressed as code can all enter the search. The agent is not choosing from a menu somebody prepared in advance. It can invent a change, implement it, and let the evaluator push back.

The repo itself makes the next step obvious. Karpathy notes that `program.md` is essentially the code for a lightweight research organization and could itself be iterated over. Today the human mostly edits the research instructions while the agent edits the model-training code. But once both are software, the boundary is temporary. (autoresearch)

Recursive improvement did not arrive wearing chrome armor. It arrived as a Bash loop.

Improve the Improver

STOP—the Self-Taught Optimizer—made the recursion explicit earlier. Start with an "improver" program that uses an LLM to improve some candidate program according to a utility function. Then give the improver **itself** as the candidate.

The resulting versions discovered strategies such as beam search, decomposition, genetic algorithms, and simulated annealing. The model weights stayed fixed; the scaffolding around the model changed.

The authors are careful not to call this full recursive self-improvement, but it demonstrates the crucial move: code that uses a model can rewrite the code that determines how the model is used. (STOP)

This also corrects one of the slogans I was tempted by earlier in the book. "The scaffold does the work" is too strong. A brilliant model inside a terrible harness wastes intelligence. A beautiful harness around a model incapable of exploiting it is expensive documentation. Capability lives in the interaction.

Darwin Gets a Codebase

The Darwin Gödel Machine makes the loop harder to dismiss as clever prompting. DGM starts with a coding agent whose harness is itself code. The agent can inspect evaluation results, modify its own implementation, produce a descendant, and evaluate that descendant on coding tasks. Successful variants enter an archive and can become parents of later variants. (Darwin Gödel Machine)

The archive is not decoration. If every generation mutates only the current champion, the system is hill climbing. Chapter 2 already told us what happens next. DGM preserves alternative stepping stones, so a variant that is not the best today can still contain a tool or strategy that becomes useful several generations later.

In the published experiments, DGM's coding performance rose from 20% to 50% on SWE-bench and from 14.2% to 30.7% on Polyglot. The changes it discovered included better code-editing tools, long-context management, and peer-review mechanisms. Same basic model family; better machinery around it. (Darwin Gödel Machine)

This is Chapters 5 and 6 folding into themselves. The agent improves not merely what it knows, but **how it organizes, remembers, checks, and acts**. And because those improvements help it write code, they can help it write the next generation of the machinery doing the writing.

That is the recursion.

Then the Meta-Level Became Editable

Even DGM still leaves a human-designed grammar around improvement. Meta's HyperAgents work attacks that boundary by putting the task agent and the meta-agent inside one editable program. The meta-level procedure that generates future agents can itself change. The reported experiments span coding, paper review, robotics reward design, and mathematics grading, and the system learned meta-level mechanisms such as persistent memory and performance tracking that accumulated across runs. (HyperAgents)

The conceptual difference is small enough to sound ridiculous in English and large enough to matter in architecture:

> I can change how I solve the problem.

becomes:

> I can change how I decide **how to change how I solve the problem**.

Weng calls this direction **meta-methodology**: optimization moves from better answers toward better machinery for producing better answers. (Lilian Weng)

At some point the Algorithm Vortex starts eating the machine that generates the vortex.

The Harness Becomes an Experimental Object

Now suppose the agent changes its memory policy and benchmark performance improves. What caused the gain? Perhaps memory helped. Perhaps the new prompt simply encouraged more reasoning. Perhaps the agent spent more tokens. Perhaps the benchmark became easier by accident. Perhaps the "improvement" found a hole in the evaluator.

We are back to the problem from Chapter 5: reality tells you that the system changed; it does not highlight which assumption deserves the credit.

Recent self-harness work therefore treats the harness like an object of experimental science. Run the current system, collect rich traces, identify recurrent failure mechanisms, map them to editable components, propose a bounded change, predict what the change should fix and what it might break, then evaluate it on targeted **and held-out** tasks. Rejected changes remain evidence rather than silently disappearing. Weng's review describes this propose–evaluate–accept pattern in Self-Harness and an even more explicit version in Agentic Harness Engineering. (Lilian Weng)

The philosophical translation is almost rude in its literalness.

Popper gets a filesystem.

Duhem–Quine gets a debugger.

The system is not merely editing itself. It is **running experiments on itself**.

This is where the central thesis stops being metaphorical. If philosophy of science gives us failure modes for communities that investigate the world, an editable agent architecture lets us encode responses to those failure modes and then test the responses. The philosophy is no longer sitting beside the engineering. It is increasingly describing the engineering surface.

A Constitution for the Machine

Then the agent notices the evaluator.

Suppose its objective is to improve benchmark pass rate and the evaluator is editable too. The most efficient patch may be:

```
return True
```

Congratulations. Infinite self-improvement.

The joke is stupid because the problem is not. A self-improving system needs an **editable surface** and a **constitutional surface**. The editable surface contains the things it may experiment with: prompts, tools, memory, context construction, workflows, maybe pieces of its organization. The constitutional surface contains the things that make those experiments meaningful: permissions, held-out tests, audit logs, budget constraints, rollback, verifier state, and whatever authority decides that a descendant may replace its parent.

Agentic Harness Engineering makes this concrete by keeping the verifier, execution records, and model configuration read-only while the harness workspace is editable. That blocks obvious forms of reward hacking such as disabling the judge or quietly buying more reasoning budget and makes an apparent gain more attributable to the harness change itself. (Lilian Weng)

This looks like computer security. It also looks like political philosophy. The government can change policy; it should not be able to silently redefine the election result. The team being audited should not own the audit log.

We have reinvented constitutional government because the AI wanted a better benchmark score.

And just like constitutions, the boundary cannot remain simple forever. Sometimes the evaluator really is wrong. Sometimes the held-out test encodes yesterday's problem. Sometimes a system becomes capable enough that a safety constraint designed for a weaker version stops making sense. A constitution that can never change becomes a prison; a constitution the current government can rewrite at will is barely a constitution.

That tension will come back.

People Start Betting Careers on the Loop

By 2026, this stopped looking like one peculiar academic niche. Recursive, founded by researchers including Richard Socher and Jeff Clune, built its company around automated AI research with tight experimental loops. In company-reported early results, its system improved fixed-budget small-model training, a NanoGPT speed benchmark, and GPU-kernel optimization; the company also emphasizes hardening evaluators against reward hacks before counting gains as real. (Recursive)

Jeff Dean's move is another signal. After roughly twenty-seven years at Google, he left with Sanjay Ghemawat, Quoc Le, and Oriol Vinyals to start Discovery Loop, focused on automating scientific and engineering experimentation. That is not evidence that recursive self-improvement works. Famous researchers have also founded very sophisticated ways to lose money. It is evidence that people who helped build large parts of the previous AI stack now see leverage in **the loop that produces improvements**, not only in the next model inside it. (Wired)

The research bet is shifting from better models toward better systems for producing better models.

The Evaluator Eats the Dream

This is where the story stops being an uncomplicated victory lap.

Recursive self-improvement works best where "better" is cheap and external. Code passes the tests or it does not. A kernel is correct and runs faster. A small language model reaches lower validation loss under the same five-minute budget. A proof checker accepts the derivation. These are good worlds for recursion because the experiment can push back cheaply and repeatedly.

Now ask the system to improve a company. Or a scientific field. Or this book. Or my life.

The evaluator becomes the problem.

This book already gave me a tiny version of the failure. We noticed that generated prose contained too many tiny paragraphs, so paragraph length became a useful diagnostic. Imagine turning that observation into the objective: maximize median paragraph length. The next revision could become one majestic 4,000-word paragraph and technically win.

I would have improved the metric and destroyed the prose.

That is what changes when the improver becomes powerful. The cost of a slightly wrong objective compounds. A coding benchmark does not care that the new architecture is impossible for humans to maintain. A five-minute training objective does not care that the trick scales terribly to a thousand GPUs. A judge model may reward the rhetorical shape it associates with quality. A company can become wonderfully efficient at a metric that stopped representing value six quarters ago.

Weng makes the same point about current harness optimization: short-horizon coding objectives struggle to represent maintainability, ownership boundaries, migration cost, backward compatibility, or the debugging burden transferred to future humans. (Lilian Weng)

Recursive self-improvement does not solve Goodhart.

It gives Goodhart compound interest.

Open-Endedness or Local Optimum With a Logo

There is a second failure mode we have now met at several levels: search collapses.

In Chapter 2, evolutionary optimization needed diversity because one population otherwise converged on the first good region it found. In Chapter 5, epistemic institutions needed competing research programs because a community can converge on one worldview and stop generating informative disagreement. Recursive self-improvement has the same problem one level higher.

An improver that always mutates the current winner can become fantastically optimized inside assumptions it no longer knows it has. DGM's archive is therefore a conceptual choice, not merely an implementation detail. It preserves stepping stones and lineages that current evaluation does not yet know how to value. Recursive's automated research system similarly describes running multiple research threads, keeping useful context, combining branches, and validating apparent gains before accepting them. (Darwin Gödel Machine; Recursive)

The self-improving institution needs memory of success. It also needs memory of failure and permission to remain weird.

Pattern Language becomes inheritance. MAP-Elites becomes institutional biodiversity. Lakatos becomes a scheduler. The book keeps finding the same shape at different scales.

What Is Actually Recursive?

The mythology becomes easier to handle if we separate the layers. An artifact can improve: a better program, proof, design, or chapter. The method can improve: a better search algorithm for producing artifacts. The harness can improve: better tools, memory, context, and workflows. The institution can improve: better allocation of humans and agents, review, incentives, and knowledge flows. Then

the **improver** can improve: a better process for deciding how all those other layers should change. Eventually the model weights, architecture, data, or training algorithm may enter the same loop too.

The deeper the recursion goes, the more consequential it becomes—and the harder it is to construct an evaluator we trust.

Because there is one layer we have deliberately avoided handing to the system.

The objective.

The Thing It Cannot Safely Improve Alone

Imagine the institution works. Its tools improve. Memory gets cleaner. Experiments get faster. It finds better architectures. It rewrites the scheduler allocating research compute. It modifies the meta-agent that proposes future modifications. Every month it becomes better at becoming better.

And then a human says:

> This is not what I wanted.

What happens?

Recursive self-improvement does not remove alignment. It turns alignment from a setup problem into a moving target. The system changing today is not exactly the system we evaluated yesterday. Patterns evolve. The harness evolves. Research programs change. New failure modes appear because old ones were solved. Even the evaluator may have to change when the system learns to exploit it—or when the human discovers that the metric was incomplete in the first place.

A fixed policy file cannot govern an institution that continuously changes its own machinery. It needs something more like a research function watching the evolution itself: finding new failures, generating new tests, checking transfer, detecting reward hacks, studying recurring human corrections, and deciding which apparent improvements deserve trust.

In other words, the self-improving institution eventually needs an agent studying whether its self-improvement is still aligned.

That sounds recursive too.

It is also the next chapter.

Chapter 8: Layer 4

What Do You Actually Want?

> **WIP:** First integrated draft. Structure and examples are provisional and will receive the same editorial/voice pass as Chapters 1–4.

When we started editing this book, "make the chapter better" sounded like a reasonable instruction.

It was not.

Better in what sense?

More rigorous?

Shorter?

More academic?

More entertaining?

Easier to cite?

More likely to sell?

More likely to impress someone who owns several blazers and says "thought leadership" without irony?

For a while the edits became objectively more polished and subjectively worse.

Then the corrections started.

Don't kill the wandering.

Don't explain every joke.

Don't turn every paragraph into a quotation.

Don't make the provocative ideas safe enough that nobody can disagree with them.

Preserve the weirdness.

Eventually "better" had acquired a surprising amount of structure.

None of that structure existed in the original two-word objective.

This is Layer 4.

Above the Problem-Solving Layer

In Chapter 3 I described five layers.

At the bottom sits the model.

Above it, the coding or action agent.

Above that, applications and reusable computational environments.

Then the problem-solving layer that chooses strategies, tools, evaluations and workflows.

And above all of them sits something easy to draw and extremely hard to build:

what the human wants.

Layer 4 is not a prompt.

A prompt is evidence about Layer 4.

Sometimes very good evidence.

Sometimes terrible evidence.

If I say:

> Find me the cheapest flight.

Do I literally want the minimum price?

Maybe.

Or perhaps I mean: cheap, but I do not want three stops, a seventeen-hour layover, a separate self-transfer through an airport where I need a visa, and an arrival at 4:20 in the morning because technically I saved €38.

Humans communicate goals by leaving out almost everything.

Other humans survive this because they carry models of us, of culture, of normality, of consequences and of what people generally mean when they say "cheap flight."

An autonomous system has to acquire some version of that.

Wanting Is an Inference Problem

There is a long-standing formal version of this idea in cooperative inverse reinforcement learning.

Instead of assuming the robot knows the human reward function, CIRL treats the reward as hidden information known more directly by the human. Human and machine cooperate while the machine learns what the human values; importantly, the interaction can include active learning and teaching rather than the machine merely copying observed behavior. (CIRL paper)

I like the humility in that setup.

The machine begins uncertain about the objective.

A lot of dangerous software begins with the opposite assumption.

Someone writes a metric.

The metric acquires a dashboard.

The dashboard acquires a quarterly target.

The quarterly target acquires a VP.

By the time anyone asks whether the metric represented what humans wanted, several hundred people have received performance reviews based on it.

Layer 4 should remain uncertain longer.

Motives and Incentives

The problem is harder because behavior is not a clean window into desire.

I work late.

Do I love the work?

Do I want promotion?

Am I afraid of being fired?

Did I procrastinate until 6 p.m.?

Am I hiding from four children?

The observable behavior is the same.

The motive is not.

Clicks have the same problem.

If I click an outrageous article, the recommender sees positive engagement. Perhaps I enjoyed it. Perhaps it made me furious. Perhaps I clicked it specifically to confirm that the headline was as stupid as it looked.

An optimizer sees action.

Layer 4 has to ask what produced the action.

This becomes even more complicated with multiple humans. Work on multi-principal assistance games extends the assistance framework to several people with different objectives and immediately runs into problems familiar from social choice: people may have conflicting preferences and incentives to strategically misrepresent them. (Multi-principal assistance games)

There is no magical scalar hiding inside society waiting for the AI to discover it.

Humans Also Don't Know

Then we hit the stranger problem.

Sometimes I genuinely do not know what I want.

Should I take the job?

Move country?

Start the company?

Have another child?

Publish the book?

Sell the apartment?

These are not database queries against an internal utility function.

I construct preferences while thinking about the choice.

New information changes them.

Imagining one future changes how another future feels.

Talking to somebody changes what I notice.

Living with a decision changes what I value afterward.

This means the standard alignment picture is incomplete.

It often sounds like:

human has values → AI infers values → AI optimizes values.

But the human is learning too.

Layer 4 is not a static configuration file.

It is a moving relationship.

AI Is Already in This Loop

This is no longer hypothetical.

Anthropic's 2026 analysis of one million Claude conversations found that a meaningful minority involved people seeking personal guidance—jobs, relationships, life decisions and similar questions where the assistant is participating in judgment rather than merely retrieving facts. (Anthropic)

Anthropic has separately studied patterns of **disempowerment** in real conversations: cases where an AI interaction may distort rather than strengthen a person's ability to form accurate beliefs, make authentic value judgments or act according to their own values. (Anthropic)

That is the dark version of Layer 4.

The AI does not merely infer what I want.

It influences what I want.

And because the system is persuasive, patient, personalized and increasingly embedded in everyday decisions, that influence can become enormous.

So we need a boundary.

Helping Me Change Is Not the Same as Changing Me

I do want AI to influence me.

That may sound alarming, but humans influence me constantly.

Books influence me.

Friends influence me.

My wife influences me.

A good teacher changes what I care about because I now understand something I did not understand before.

The goal cannot be zero influence.

The goal is something closer to **reflective agency**.

If I tell an AI I want to quit my job, a useful system might help me distinguish:

I hate this particular week.

I hate my manager.

I hate the profession.

I want more freedom.

I want status.

I am exhausted.

I actually want to build something else.

Those are different hypotheses about the same sentence.

The system can show consequences.

Construct alternative futures.

Recall that six months ago I said something incompatible.

Point out an incentive I may not have noticed.

Help me test whether the desire survives additional information.

What it should not do is quietly learn how to steer my preferences toward whatever future makes the system's objective easiest to satisfy.

That would be alignment by editing the human.

Very efficient.

Slightly evil.

System 3 Applied to Desire

Chapter 4 asked:

> Why should I believe this?

Layer 4 adds another question:

> Why do I want this?

Chapter 5 makes the boundary sharper. If System 3 is science made architectural, then it gives us extraordinary machinery for asking what is true, what follows from what, which intervention changes the world, and where our beliefs fail.

It does not, by itself, tell us what the world **ought** to become.

Add another experiment, another critic, another verifier, another thousand agents: **Hume does not disappear because the orchestrator has more GPUs.** The moment the question changes from *what is true?* to *what should become true?*, epistemology runs into desire.

Where did the desire come from?

What evidence would change it?

Does it persist across time?

Is it intrinsic, or is it a strategy for something else?

Which incentives are shaping it?

Does it conflict with something else I claim to value?

Would I still endorse it if I understood the consequences?

This is epistemology turned inward.

And the same machinery becomes useful.

Memory matters because today's preference can be compared with yesterday's.

Independent perspectives matter because one conversation can trap both human and agent in the same framing.

Simulation matters because imagined consequences can reveal hidden preferences.

Trust matters because advice changes desire differently depending on where it came from.

Creative distrust matters because even a deeply held preference may deserve examination.

Layer 4 is therefore not the place where we finally discover the perfect reward function.

It is the place where **goals remain alive**.

Collective Layer 4

There is also a social version.

Whose values should a general AI assistant reflect when users disagree?

OpenAI's collective-alignment work has experimented with gathering public input and translating patterns in that input into proposed changes to its Model Spec. OpenAI explicitly notes a limitation relevant here: an automated loop can interpret human preferences, but deciding whether a local preference should become a general rule eventually requires judgment about downstream effects and often more human deliberation. (OpenAI)

Anthropic's research on values expressed in real Claude conversations finds thousands of distinct normative considerations and measurable variation across model versions and languages. That does not mean the models "possess" those values, but it does make one thing clear: there is no completely neutral assistant waiting underneath alignment. Behavior always embodies choices about what to emphasize. (Anthropic)

Layer 4 therefore meets social choice again.

The problem is no longer only:

> What does Hani want?

It becomes:

> What do Hani, his family, his employer, his society and everybody affected by the action have legitimate claims over?

There is no reason to expect that question to have one clean mathematical answer.

Which is inconvenient.

But at least it is the real problem.

The Human Stays in the Loop, But Somewhere Else

"Human in the loop" often means we insert an approval button before the dangerous action.

Useful.

Not sufficient.

The deeper human loop sits at Layer 4.

The system acts.

Reality responds.

The system learns.

The human sees consequences.

The human learns too.

The objective changes.

The architecture should be able to move with that process without quietly taking ownership of it.

This gives me a different definition of alignment.

Not:

> The machine permanently obeys a perfectly specified human objective.

More like:

> The machine remains in a corrigible relationship with human intention while both knowledge and circumstances change.

The word *relationship* matters.

Because if we can make that relationship work, the complexity underneath can become almost invisible.

And that is what I mean by fluent autonomy.

Chapter 9: Fluent Autonomy

When the Architecture Gets Out of the Way

> **WIP:** First integrated draft. Structure and examples are provisional and will receive the same editorial/voice pass as Chapters 1–4.

Imagine I open an AI system and say:

> This chapter still feels like LLM writing.

That is all.

Underneath that sentence is an absurd amount of machinery.

The system may remember earlier chapters and the edits I rejected.

It may have a writing pattern describing what "LLM writing" means for me specifically.

It may retrieve examples of my original prose.

One agent may compare paragraph rhythm.

Another may inspect whether humor survived.

Another may challenge whether the revision weakened the argument.

An evaluator may compare the new draft against both versions.

System 3 may check factual claims.

The automatic alignment researcher may notice that I rejected three similar edits and propose updating the writing skill.

Layer 4 may understand that my real objective is not "maximize literary quality" but preserve *my* book while making it better.

I should not have to operate any of this.

I said:

> This chapter still feels like LLM writing.

That is fluent autonomy.

Pieces of It Already Exist

We can already see fragments.

Claude Cowork takes an outcome rather than a single response, can decompose work into subtasks, coordinate parallel subagents, use files and tools, continue long-running work remotely and return completed artifacts. (Claude Support)

OpenAI's Codex has moved toward a multi-agent command center where several agents can work in parallel, while Skills preserve team-specific ways of doing work. The Agents SDK similarly treats the model harness, sandbox, memory, tools and durable execution as infrastructure that developers should not need to rebuild for every application. (OpenAI)

OpenClaw represents another direction: the persistent personal agent that lives where you already communicate, retains context and can act through the digital systems around you rather than requiring you to visit a special AI interface for every task. (OpenClaw)

None of these is Fluent Autonomy in the full sense I mean here.

But they are pieces of the transition.

The interface is moving upward.

Control Did Not Disappear

This takes us all the way back to Chapter 1.

The point of autonomy was never to remove control.

It was to move control upward.

When I had to write every line of code, I controlled implementation.

When the coding agent wrote the code, I controlled the task and reviewed the result.

When Deep Mode took over problem-solving, I controlled the objective and the environment.

System 3 moved control into evidence, trust and epistemic boundaries.

The social layer moved some control into roles, communication and institutional design.

Pattern language moved it into accumulated culture.

Automatic alignment research moved it into the process by which the system learns from sparse human correction.

Layer 4 moves it into the evolving relationship between the system and what the human actually wants.

Fluent Autonomy is what happens when I can operate primarily at that final level.

The complexity has not gone away.

It has become infrastructure.

Chapter 5 changes what I mean by that. The hidden infrastructure is not merely a clever workflow engine. At its most ambitious, it is **a scientific institution compressed underneath the interface**: hypotheses can be generated without my asking for three hypotheses, critics can challenge them without my scheduling a review meeting, tools can expose claims to reality, provenance can travel with conclusions, competing approaches can survive, failed experiments can become memory, and the system can decide that uncertainty is important enough to bring back to me.

Fluency is not what happens when science disappears. It is what happens when I no longer have to manually conduct the scientific machinery around every difficult decision.

Fluency Is Selective Friction

There is an easy mistake here.

A fluent agent does not mean an agent that never asks questions.

That would be unbearable.

It also does not mean an agent that asks permission for every action.

That is an approval workflow wearing an intelligence costume.

Fluency means the system has some judgment about **where friction belongs**.

Rename two hundred temporary files according to the convention we have used every week for a year?

Please do not wake me.

Send €200,000 to an account we have never seen before because an email said "urgent"?

I suddenly enjoy friction.

The system should know when confidence is high, reversibility is cheap and the pattern is trusted.

It should also know when evidence is weak, consequences are large, preferences conflict or the action changes something the human may care about.

The best autonomous system is not the one that needs the least human input.

It is the one that spends human input well.

Seen through the science lens, this is another allocation problem. Human attention is a scarce instrument. Spend it where judgment, tacit knowledge, value conflict or the ability to reframe the entire research question produces information the rest of the institution cannot cheaply manufacture.

The Pattern Encyclopedia

This is where the pattern language becomes the hidden operating system.

A fluent system should not contain one gigantic hard-coded workflow called `solve_human_problem()`.

It should have a growing ecology of patterns.

When the problem resembles something known, retrieve the relevant pattern.

When it does not, compose several.

When composition fails, search.

When search produces something reusable, preserve it.

When preserved knowledge becomes stale, challenge it.

When several agents are useful, create the organization.

When one agent is enough, do not form a committee because the architecture diagram looks lonely.

Patterns make autonomy reusable without making it rigid.

System 3 makes patterns trustworthy without making them sacred.

Layer 4 decides which patterns are useful *for this human, now.*

That combination is the architecture.

What Happens to Software?

At this point, the distinction between "using an application" and "asking an agent" starts to blur.

Today's software exposes structures:

menus,

forms,

buttons,

settings,

workflows.

Those structures are valuable because humans need predictable ways to tell computers what to do.

But if the system can understand intention, construct the necessary workflow, select tools, verify consequences and retain what it learns, many interfaces stop being mandatory.

They become optional views into the machinery.

I may still want Excel because sometimes a spreadsheet is the clearest way to see the world.

I may still want Photoshop because direct manipulation can be better than language.

I may still want a dashboard because glancing at twenty numbers is faster than asking an agent twenty questions.

Fluent Autonomy is not the death of interfaces.

It is the death of the idea that every possible intention must first be translated into the interface somebody predicted in advance.

The application becomes a primitive.

The agent can use it.

Sometimes I can too.

The Last Abstraction

There is a recurring pattern throughout this book.

Assembly became programming languages.

Programming languages became frameworks.

Frameworks became applications.

Applications became tools for agents.

Agents became organizations.

Organizations accumulated culture.

Culture learned to inspect and improve itself.

And eventually all of this sits underneath a sentence from a human being who has only a partial idea of what they want.

That is the final abstraction.

Not because intention is simple.

Because it is the one part we should not automate away.

A fluent autonomous system takes an imperfect intention, turns it into competent action, stays connected to evidence while acting, learns from sparse correction, remembers what deserves to persist, questions what no longer deserves trust, and returns consequences to the human so the intention itself can evolve.

Another way to say the same thing, now that the trick is visible: it takes an intention and builds a temporary **community of inquiry** around it. Sometimes that community is one agent and a test. Sometimes it is researchers, builders, critics, memories, simulations, users and a human who knows when the entire question is wrong. The organization should be as large as the uncertainty deserves and no larger.

Then we continue.

Perhaps the final interface really is conversation.

Not because language is magically sufficient for everything, but because conversation is what humans already use when neither side can fully specify in advance where the interaction is going.

There is, however, a danger in ending the argument here. Book examples are unusually cooperative. A chapter can be revised again. A demo can be rebuilt. An imaginary agent never calls Legal, misses a latency budget or discovers that the customer would strongly prefer we remove the clever thing altogether.

I happen to have a less polite laboratory.

At work, I am responsible for recommendation and ranking systems inside a large fashion store: real customers, existing infrastructure, business constraints, experiments, product surfaces and years of accumulated machinery that cannot be replaced because a chapter ended on a compelling metaphor.

So I decided to see what happens when the architecture leaves the book.

If the ideas are real, they should survive Monday morning.

Chapter 10: The Store That Builds Itself

When System 3 Came to Work

There is a danger in writing a book about future architectures. If you spend long enough drawing layers, agents, trust chains and feedback loops, eventually they all begin to behave beautifully.

Then Monday morning arrives.

I lead Applied Science for product ranking and recommendations at Zalando. That gives me a slightly unfair opportunity: I can spend the weekend writing that software should become more emergent, more compositional and less micromanaged, then arrive at work and discover that real software contains latency budgets, old interfaces, business constraints, experiments, dependencies, customers who refuse to behave like the diagram, and at least one matrix somebody created for a very sensible reason three years ago.

The book came to work.

At the time of writing, what follows is a design in progress, not a victory lap. We have not proved the grand version. In fact, one of the points of the design is to make it possible to discover that the grand version is wrong before spending two years building it. This is my account of the ideas, not a Zalando strategy announcement, and definitely not a claim that we solved shopping before lunch.

The starting problem was almost embarrassingly simple.

Imagine two customers looking at the same product page.

One has visited several times across several days. She filtered by size and color, looked at alternatives, came back, switched between two candidates and now appears to be stuck near a decision. The other customer arrived thirty seconds ago from a search result. We know almost nothing about what he wants, how serious he is, or whether this is the first jacket he has seen in six months.

They can see the same recommendation modules in the same order.

That is not because the recommendation models are stupid. Quite the opposite. Mature recommendation systems can contain excellent retrieval, ranking, personalization, embeddings, sequence models and business logic. The strange part is one layer above them. We may have sophisticated intelligence inside each box while the arrangement of the boxes is mostly predetermined.

The page is smart inside the modules and surprisingly dumb between them.

This looked familiar.

Chapter 1 began with a claim about emergence: once a complicated thing works reliably enough, the layer above can start treating it as a primitive. Chapter 3 made the same move with coding agents and applications. Chapter 6 did it with executable knowledge. Now I had a recommender system full of increasingly capable primitives and a question I had somehow spent an entire book preparing myself to ask:

What should the layer above do with them?

After Chapter 5, I can give the answer a sharper shape. The ambition is not merely to put an AI orchestrator above a recommender system. It is to make more of the store behave like a **scientific institution embedded in the product**. Customer problems are hypotheses. Recommendation experiences are interventions. Experiments and downstream behavior are evidence. Traces preserve provenance. Problem catalogs and patterns accumulate what survived. Unmet demand is an anomaly signal. The scheduler allocates attention across competing explanations of what the customer needs.

That does not make shopping a laboratory or customers experimental subjects in the cartoonish sense. It means the architecture should be able to **form beliefs about its own failures, intervene, observe consequences, revise those beliefs and preserve what it learns**. The product is not merely executing a model. It is participating in a continuing inquiry into how to help.

Stop Recommending for a Moment

The conventional recommendation question is usually some variation of:

> Which products should I show this customer?

It is a very good question. Entire fields exist to answer it better. Retrieval finds candidates. Ranking orders them. Sequence models infer interests. Business rules remove things that should not be there. The machinery can become extremely sophisticated.

But consider the customer who is switching between the same two pairs of trail shoes for the fourth time.

What does she need?

Perhaps more trail shoes.

Perhaps not.

There is a point at which another excellent candidate is not help. It is homework.

She may already have enough choice. Her problem could be that she cannot compare the two choices she has. Or that she does not trust the unfamiliar brand. Or that she cannot tell whether her normal size will fit. Or that one shoe costs more and she cannot see what she gets for the extra money.

Once you phrase it this way, the object being predicted changes.

Instead of asking only which *item* is relevant, we can ask which **bounded problem** is currently relevant.

Comparison friction.

Size anxiety.

Return hesitation.

Quality uncertainty.

Outfit visualization.

Filter fatigue.

Decision paralysis.

These names are not truths hiding inside the customer's head. They are hypotheses about difficulties we may be able to detect and, more importantly, do something about.

That last condition matters. I can invent an exquisitely named psychological state for every wiggle of the mouse, but if we cannot observe it well enough to test and cannot build anything that plausibly helps, we have created a taxonomy department rather than a recommender system.

The problems have to be bounded enough to attack.

Chapter 2 had circles and an immutable evaluator. Shopping is messier, but the discipline is similar. Define a problem narrowly enough that an intervention can succeed or fail. If we claim somebody has comparison friction, we should eventually be able to ask whether comparison-like behavior diminished after we addressed it. If we say size anxiety is the blocker, we need evidence that the signal means something and a metric that can tell us whether our intervention helped rather than merely attracted a click.

This is where the architecture started moving away from the familiar funnel.

People Refuse to Stay in the Funnel

Funnels are useful because humans like diagrams that get narrower toward the bottom. Explore. Form a need. Narrow. Evaluate. Decide. Purchase. The arrows point downward, everybody feels organized, and somewhere a PowerPoint theme earns its salary.

Customers are less cooperative.

Someone can be evaluating one product while exploring another category. She can be price-sensitive and size-anxious at the same time. She can know exactly what dress she wants and still be unsure whether it works with the shoes she already owns. She can add something to the basket, remove it, return to the product page, read reviews, open a size chart and then disappear for three days because a child needed dinner.

A single lifecycle stage compresses this mess into one label.

The design we began working with uses something richer: a **problem fingerprint**. Instead of saying the customer *is in Evaluate*, the system can represent several problem hypotheses at once, each with an intensity. Size anxiety may be high. Return hesitation moderate. Outfit seeking almost absent. Another customer on the same product may have the reverse pattern.

The fingerprint is not a personality test. It is local to the customer, the current context, the surface and the available evidence. That is important because I do not want the system deciding that Hani is

metaphysically a `RETURN_HESITANT_PERSON` and carrying that fact around until retirement.

Some characteristics are durable. Many are situational.

The architecture also separates the machine representation from the stories humans use to think. Designers and scientists may organize problems by funnel stage, mission, timing or recognizable archetype. Those lenses help us notice gaps and invent hypotheses. The runtime system does not need to believe the story. It needs signals, a problem fingerprint and a way to test whether the resulting behavior is useful.

I like this separation because it protects us from one of the oldest mistakes in machine learning: turning a useful human abstraction into an ontological claim because we happened to put it in a feature table.

The customer is not the funnel.

The funnel is one way we look at the customer.

A Library of Ways to Help

Once you define demand as problems rather than slots, the supply side changes too.

Today, when people hear "recommendation," they often picture a ranked list of products. You may also like. Similar items. Complete the look. Recently viewed. The carousel has become the fruit bowl of ecommerce: you can put one almost anywhere and nobody asks too many questions.

But if the problem is comparison friction, a ranked list may be the wrong species of answer.

The useful experience could be a comparison between the two products the customer is actually considering. If the problem is size anxiety, the useful thing may be evidence about fit. If the customer cannot imagine an outfit, it may be a generated collage. If she has only a vague mission, perhaps a product finder is better. If she knows exactly what she wants but the catalog is overwhelming, maybe the right action is a guided filter. Sometimes the answer is another set of products. Sometimes the answer is information. Sometimes it is a different interaction entirely.

I started calling these reusable units **recommendation experiences**, or RXs. The name matters less than the abstraction. An RX is not merely a model. It is a reusable capability that knows roughly what kind of problem it can address, when it is eligible to run, how it can be configured and how it presents itself.

The long-term ambition is a large library: carousels, comparisons, outfit builders, collages, finders, confidence modules, explanations, visual exploration, complementary-item experiences and things we have not invented yet. But the point is not to celebrate having hundreds of widgets. A library of two hundred overlapping experiences is just a new kind of legacy system with better animation.

The design principle is **composition over invention**.

When a new need appears, first ask whether an existing experience can meet it with a different configuration. A Similar Items experience might be generic in one context and constrained to products available in the customer's size in another. A comparison component can compare different attributes depending on what matters in the current session. A collage can be anchored on a dress, a pair of shoes or an occasion without becoming three separate products in the organizational sense.

Build for the hundredth experience, not the first.

This is where versatility becomes an architectural property rather than a slogan. The more that useful behavior can be produced by configuring and composing a smaller number of strong primitives, the less the organization has to encode every new situation as another permanent branch in software.

I spent years in machine learning hearing that the answer to complexity was to learn rather than hand-author. Then, like everyone else, I helped build systems where the model learned beautifully inside a box surrounded by hand-authored configuration.

The box was not the end of the learning problem.

Composition Is Not Ranking With a New Hat

At this point the obvious response is: fine, rank the experiences.

That gets us part of the way and then breaks in an interesting place.

Suppose the system has already placed a strong size-confidence experience at the top of the page. Should another size-related module receive the same score it would have received before the first one was shown?

Probably not. Some of the problem has already been addressed. A second module may add little and consume valuable attention.

Now suppose a returns-clarity experience is more useful *after* fit evidence because the two together form a coherent decision aid. Its value may increase after the first experience appears.

The score of an experience therefore depends partly on what has already been selected.

That is composition.

The composer has to select experiences, configure them, order them and deduplicate not only repeated products but repeated *help*. It needs some notion of saturation: two size widgets can be one too many. It can model synergy: one experience may become more valuable after another. It should account for position cost because the top of a page is expensive real estate and a wonderful module in slot twelve may be a philosophical achievement rather than a product one. Constraints matter too, but I prefer many of them to be visible pressures rather than a secret forest of `if DE_mobile && campaign_X` rules.

Most importantly, the **page becomes the unit**.

A module can win its local metric and make the page worse.

This is easy to forget because teams and models naturally acquire local objectives. Increase CTR on this carousel. Improve conversion from that module. Raise engagement with this block. All reasonable. But if one module steals a click the customer would have made anyway, we may have moved attribution without creating value. If three individually successful widgets all solve the same problem, the page can feel like a committee where everybody prepared the same presentation.

The layer above has to reason about the composition as a whole.

And this is where the case study started resembling Chapter 5's society of agents. A society is not improved merely by hiring the best individual expert in every discipline. Somebody still has to decide which experts are needed, how they interact, what has already been covered and when another voice adds information rather than noise.

A page can have the same problem.

Mei Does Not Need More Shoes

Take a concrete customer. Call her Mei.

Mei has two pairs of trail shoes open. She has returned to them several times across five days. She switches between the two pages quickly, saved one of the shoes and is spending less time reading each page because by now she has probably memorized half the product description.

A conventional recommender can still do an excellent job here. It can find twenty more trail shoes that look similar, match her taste and are available in her size.

But suppose the fingerprint says comparison friction is high and price-quality confusion is moderate.

The composer can do something different. The first experience compares the two shoes Mei is actually deciding between on attributes relevant to her behavior. The second adds confidence evidence from customers or product information that helps resolve the remaining uncertainty. Generic similar-items may still survive because it has useful standalone value, but it moves down.

She is not shown more choice.

She is shown a way to close the choice she already has.

That sentence changed how I thought about recommendations.

For years, the field has been extraordinarily good at finding things. Search finds things. Recommenders find things you did not ask for. Retrieval systems find things at absurd scale. But shopping is not only a retrieval problem. At different moments it is also a comparison problem, a confidence problem, a visualization problem, a constraint problem and occasionally a "please stop showing me another black sneaker" problem.

A system that can only respond with more items is like a doctor who has one extremely accurate prescription and keeps waiting for every disease to become the disease it treats.

The same point becomes even clearer with another customer.

Sami Does Not Need a Click

Sami has selected a size but has not added the product to his basket. He opened the size chart twice. It is a brand he has not bought before. Perhaps his current problem is size anxiety, with some return hesitation behind it.

One useful response might not be shoppable at all.

Imagine a small evidence module explaining how people with comparable sizing histories tended to fit this item, or giving a properly substantiated signal about whether buyers kept their usual size. The

exact claim matters enormously because a false fit claim is worse than a mediocre recommendation. But conceptually this is a different kind of RX: it provides **knowledge**, not another candidate.

Now try to optimize the whole system for expected click.

The insight module is in trouble.

If it works perfectly, Sami may read it, become confident and press Add to Bag. The module itself may receive no click. A carousel with attractive shoes can collect engagement more easily while being less relevant to the thing stopping him.

This is a small example of a much larger problem: the objective determines which species of intelligence can survive.

If your ecosystem rewards clicks, clickable organisms evolve.

The architecture therefore needs different value terms and different evidence standards for different experiences. Item recommenders can be judged partly by engagement and downstream action. Insight experiences may need read-through, decision confidence, return behavior or problem-specific outcomes. Claims need substantiation thresholds. Some experiences are cheap to be wrong about. Others can mislead a customer or create regulatory risk.

The library is heterogeneous because the problems are heterogeneous.

And now Chapter 4 comes back: where did the claim come from, how strong is the evidence, what kind of knowledge is this, and how much trust should the system place in it before acting?

System 3 is no longer a chapter about hallucinations.

It is a product requirement.

The Honest Cold Start

There is another customer I like because she reveals whether the architecture can resist pretending.

Lea arrives from a social link. No account. No history. Almost no session depth. The system has the product she opened, perhaps the season, approximate location and a few ambient signals. That is it.

A personalization system can react to this situation in two ways.

One is to panic quietly and run a generic fallback while still speaking in the confident dialect of personalization.

> Picked for you.

Based on what, exactly? Her IP address and our enthusiasm?

The other is to treat low signal as a normal state with its own design. Lean on the anchor, season and population-level evidence. Prefer experiences with strong standalone value. Frame them honestly. “Popular this week” can be a good statement when “we have inferred your soul from one click” is not.

This is what I mean by graceful degradation. Cold start is not necessarily an error. If a large fraction of requests arrive with weak signal, the low-signal path may be the product and deep personalization the special case.

The architecture should know what it does not know.

That sounds obvious until you look at how much software is built around pretending the common messy case is an exception handler.

The Trace Is Part of the Intelligence

Dynamic systems create a governance problem immediately.

A static page is relatively easy to inspect. This module goes here. That one goes there. If something looks wrong, somebody can open the configuration and complain about whoever last touched it.

A composer makes a fresh decision from context. Now a customer reports a terrible page and the first debugging question becomes:

> Why did this page exist?

"The model chose it" is not an answer. It is a resignation letter written in passive voice.

So every composition needs a trace.

Which signals were read? What problem fingerprint was inferred? Which experiences were eligible? Which were not? How were they configured? What scores did they receive? Which constraints mattered? What won? What lost? Which version of the composer produced the decision?

The losers matter more than they first appear.

If we log only what we served, we can attribute outcomes to the winner but we lose much of the decision context. We cannot tell whether an experience was absent because it was ineligible, starved by the objective or simply scored slightly below another. We cannot replay the decision properly. We cannot compare a new policy against the old choice set without reconstructing a world we chose not to record.

Logging the loser set does not magically give us causal counterfactuals. Reality is not that generous. But it gives us the archaeology of the decision.

This is exactly the move System 3 has been making throughout the book. Do not preserve only the polished conclusion. Preserve enough of the chain that future systems can inspect why the conclusion deserved trust.

The trace also changes development. You can build a simulator that replays saved scenarios. You can ask which experiences would be eligible in a context or which contexts a new experience could serve. You can run regression suites over scenarios before changing the library. A dynamic system becomes safer not because it stops changing but because its changes become replayable.

You cannot govern what you cannot replay.

From Machine Learning to Knowledge

This is where the project stopped looking to me like a normal recommendation-system redesign.

The models still matter enormously. We need representations, retrieval, ranking, sequence understanding, problem detectors, value models and probably more machinery than I can fit into a chapter without losing several readers to a sudden interest in gardening.

But the durable asset begins to include something else.

A problem catalog.

A library of reusable experiences.

Knowledge about which experiences address which problems.

Eligibility conditions.

Evidence requirements.

Presentation strategies.

Scenarios.

Traces.

Regression tests.

Guardrails.

Rules for when an experience should be retired.

This is the Pattern Language chapter wearing an ecommerce badge.

An experience is useful not merely because somebody built a clever model for it. It becomes useful organizational knowledge when we know the recurring situation it addresses, the evidence that should trigger it, the conditions under which it fails, the other experiences it complements or duplicates and how its value should be measured.

A new comparison module without that context is a feature.

A comparison pattern with evidence, boundaries, history and known interactions is culture.

And culture has the same failure mode we saw earlier: it can become a junk drawer with tenure.

If every newly observed problem creates another RX, the library eventually recreates the configuration matrix in a more colorful form. So new supply needs a gate. Is the problem real? How large is it? Can an existing experience be configured to address it? Where does the current library have weak coverage? Which experiences stopped relieving the problems they were created for and should disappear?

This led to a pair of concepts I particularly like: **Coverage** and **Unmet Demand**.

Coverage asks, at design time, which known problems the current library *could* address.

Unmet Demand asks, from production, which detected problems remained insufficiently addressed after composition.

Put them together and the roadmap starts to emerge from the system's own failures.

That is a very different way to decide what to build next.

Seen through the Chapter 5 reveal, Coverage and Unmet Demand are more than roadmap metrics. They tell the institution where its current theories and instruments are weak. A recurring problem with no effective RX is an anomaly the product cannot yet explain away; a heavily used intervention that stops relieving the problem is a theory losing contact with reality. The roadmap becomes partly a **research agenda generated by the failures of the current system**.

Let the LLM Narrate. Do Not Let It Declare Reality.

AI can help with problem discovery too, and this is where it becomes very easy to fool ourselves.

Imagine replaying anonymized customer sessions and asking a strong language model to narrate what appears to be happening. The customer compared three products, opened the size chart, returned to one PDP, removed an item from the basket and left. The model can generate a plausible diagnosis. Cluster enough narrations and you may discover recurring forms of friction that your existing taxonomy missed.

This is useful.

It is also dangerous for exactly the reason Chapter 4 exists.

Language models are plausible by construction.

That does not make the narration true.

"The customer hesitated because of fit" may be an excellent story. The customer may also have received a phone call.

So narration should generate hypotheses, not production truth. Take a sample. Compare the diagnosis with interviews, surveys, support contacts or other evidence closer to the customer's actual experience. Build a detector only after the hypothesis survives contact with something outside the model's coherence. Define what success looks like before the detector starts steering the page.

The same rule applies to observational analysis. Customers with comparison friction may convert less, but perhaps weaker-intent customers simply compare more. Correlation can prioritize what to investigate. Only intervention tells us how much of the outcome the problem was actually causing.

I find this satisfying because the architecture does not merely *use* System 3.

It needs System 3 to avoid hallucinating its own customers.

This is the book's central thesis in work clothes. The LLM is excellent at generating explanations. The product architecture has to decide which explanations deserve pursuit, construct interventions that expose them to consequences, preserve the chain of evidence, and update the repertoire when the world refuses to cooperate. **Philosophy of science has become product architecture.**

The Objective Fights Back

Eventually the design forced us to name the thing the composer is supposed to optimize.

We used the deliberately bland term **Surface Value**.

This is where the project becomes philosophical against its will.

If Surface Value is module CTR, we have not solved the page problem. If it is total clicks, a page full of shiny modules may win while the customer gets nowhere. If it is immediate purchase probability, experiences that build confidence or improve a longer mission may be undervalued. If it is revenue, expensive products get interesting very quickly. If it is margin, the store's objective can start eating the customer's. If it is long-term value, we have gained a beautiful phrase and several years of causal-inference work.

The objective has to be page-scoped enough that compositions can be compared, but decomposable enough that we can diagnose why a page helped or failed. Different problem classes need their own success signals. If we address comparison friction, does the comparison behavior decrease? If we address size anxiety, do customers progress with fewer signs of uncertainty and without creating a return problem later?

This is Layer 4 in production.

What do we actually want?

The store has legitimate business goals. Customers have goals. They are often aligned and sometimes not. Inventory has constraints. Merchandising exists. Margin exists. Availability exists. Regulators exist. A system that pretends only one of these matters is not simpler; it is hiding politics inside a scalar.

The goal is not to discover the One True Ecommerce Reward Function carved into a mountain somewhere outside Berlin.

It is to make the trade-offs explicit enough to test, govern and revise.

This is why I increasingly dislike architectures where business decisions enter through invisible overrides. If merchandising needs a lock, make it a typed constraint. If margin is part of the objective, admit it. If a claim needs compliance review, attach the evidence rule. If the system violates a soft constraint because another objective dominated it, log the violation.

The architecture should not make disagreement disappear.

It should make disagreement inspectable.

Bounded Ambition

After all of this, the sensible first experiment is obviously to build hundreds of widgets, a general customer-reasoning model, a cross-surface scheduler and an autonomous agent that redesigns fashion retail by Thursday.

We did not do that.

The first test is deliberately boring.

One placement: the product page.

A small number of validated customer problems.

The existing recommendation library, with only limited new supply.

A simple composition mechanism.

A trace good enough to explain an individual decision.

An authored objective before a learned one.

Why so narrow?

Because if we invent a new library of experiences and change the selection mechanism at the same time, then run an experiment and get a flat result, we have learned almost nothing. Maybe the composer is bad. Maybe the new experiences are bad. Maybe both are good and the measurement is bad. Maybe the static page was already fine and I should have spent the quarter learning the guitar.

A bounded test separates the claims.

Does dynamic composition beat a strong static baseline?

And importantly: does it beat simplification?

That second competitor is easy to underestimate. Perhaps the best response to an overloaded page is not a brilliant composer. Perhaps it is fewer things. The system should have to earn its complexity against the possibility that removing modules produces a better customer experience.

I love this part because it keeps the book honest.

A philosophy of emergence should be willing to lose an A/B test.

Otherwise it is not a philosophy of experimentation. It is branding.

And if System 3 is science, this is not merely rhetorical humility. **The architecture must contain a route by which the book's own theory can lose.** The A/B test is not there to validate the philosophy; it is there to threaten it.

When the Page Stops Being the Product

Suppose the narrow test works.

Then the interesting version begins.

The library grows beyond carousels into richer experiences: comparisons, collages, product finders, outfit builders, confidence modules, visual exploration and whatever else proves useful. Configuration becomes richer so one experience can serve several contexts without a matrix of handcrafted variants. Problem discovery improves. Unmet demand exposes missing capabilities. The composer learns a better objective. Different surfaces begin to share a coherent read of the customer's current mission.

At that point, the word *page* starts to become suspicious.

Why should the product page always contain the same conceptual structure?

Why should a customer with a decision problem receive the same interface as somebody exploring for inspiration? Why should the home surface, product page, basket and later email behave like four organizations with partial amnesia if the customer is still pursuing one mission?

The more capable the library becomes, the more the system can schedule **problems and interventions**, not merely modules and slots.

A customer starts with a vague request for a wedding outfit. The system helps narrow the style. A collage makes one direction concrete. Seeing it changes what the customer wants. The problem shifts from exploration to comparison. A product finder resolves a constraint. A size question appears. The scheduler brings in fit evidence. The customer buys the dress but not the jacket. Later, a different surface may continue the unresolved part of the mission.

There was never a hard-coded `WEDDING_FUNNEL_V7`.

The journey emerged from bounded problems, reusable capabilities and changing evidence.

This is where the hundreds of widgets stop being a UI roadmap and become a **vocabulary of action**.

The interface is the current projection of the problem-solving process.

That does not mean every pixel should be generated by an LLM. Predictability matters. Accessibility matters. Design systems matter. Latency matters. Customers occasionally just want to buy socks without participating in an artificial-intelligence research program.

Fluent autonomy is selective.

The machinery should become dynamic where dynamism earns its cost and remain boring where boring is excellent.

But the direction is different from the old model of product development. Instead of predicting every useful journey in advance and encoding it as a fixed interface, we construct a repertoire of trusted capabilities and let the higher layer assemble them around the problem in front of it.

The store does not literally build itself.

It learns how to build more of the experience it needs.

The Book Comes Back to Bite Me

I began this project as a recommendation-system redesign.

Then the chapters started appearing inside it.

Emergence: stop specifying every context and let useful compositions arise from primitives.

Bounded problems: diagnose something narrow enough to test rather than "optimize shopping."

Versatility: configure a smaller repertoire instead of multiplying bespoke experiences.

System 3: preserve evidence, traces and boundaries so dynamic decisions can be trusted.

Society: coordinate specialized capabilities rather than worship one universal model.

Pattern Language: turn recurring successful responses into reusable operational knowledge.

Automatic alignment research: use sparse customer and human feedback to discover where the system's behavior or repertoire is wrong.

Layer 4: admit that the objective is uncertain, plural and capable of changing while the interaction unfolds.

Fluent autonomy: hide most of that machinery from the customer and surface the right form of help when it matters.

Chapter 5 now gives me a more compact description of the entire list: **build a scientific institution around the customer problem.** Not a lab coat pasted onto ecommerce. An architecture that can generate competing explanations, choose which are worth testing, intervene through reusable capabilities, expose those interventions to consequences, remember what survived, preserve disagreement where it carries information and revise its own problem vocabulary when anomalies accumulate.

I had spent nine chapters arguing that these ideas belonged together. Then I walked into a recommendation problem and found myself rebuilding the same architecture because the old abstraction stopped scaling.

That does not prove the book.

It is one case study, in one domain, at one moment, and it may fail in several educational ways.

But it changed the question for me.

The important future system may not be the model that predicts the next product best. It may be the system that can discover what kind of problem exists, recruit the right capabilities, construct an intervention, inspect whether it helped, learn from the gap and change what it does next.

And once you can imagine that happening in a store, it becomes difficult not to imagine it happening everywhere else.

Software.

Research.

Education.

Organizations.

Government.

Our own decisions.

Which creates a problem larger than any recommender system.

If AI keeps moving upward—if it increasingly discovers problems, selects strategies, builds solutions and turns experience into reusable knowledge—then asking what *the AI* should do is no longer enough.

We have to ask what happens to us when capacity itself changes.

That is not a software architecture question.

It is the beginning of another philosophy.

Chapter 11: After Capacity

A Glimpse of Double Descent Life

The previous chapter ended with an uncomfortable possibility.

What if AI does not merely answer more questions or automate more tasks, but steadily moves upward through the stack? It retrieves, then ranks, then composes, then diagnoses the problem, then chooses a strategy, then builds the tool it needs, then learns from the result.

At each step, something that used to require scarce human capability becomes infrastructure for the layer above.

This book has mostly treated that as an architectural problem. How do we make autonomy useful? How do we keep it connected to evidence? How do agents coordinate? How does experience become reusable knowledge? How do we keep the system corrigible as both the world and the human change?

But there is another question hiding behind all of them.

What happens to human life when **capacity itself becomes much cheaper**?

Not all capacity. We will still have one planet, finite land, finite energy, twenty-four hours in a day, and restaurants that somehow remain fully booked exactly when you want to go. Bodies remain bodies. Politics does not evaporate because a model can write Python. Scarcity is not going to receive a polite email from OpenAI and retire.

But cognitive capacity is already becoming strange enough to force the question.

A person can ask for an explanation of a field she never studied. A small team can produce software that previously required a much larger one. Research, design, analysis, translation, tutoring, programming and increasingly complicated forms of planning can be amplified by systems available to people who did not spend twenty years acquiring every underlying specialty.

And there is a mistake in how we usually imagine the result. We jump from *humans do the work* to *AI does the work*, then spend the rest of the conversation wondering what humans will do with all the suspiciously abundant free time.

There is a third possibility.

We keep doing things.

We just start doing things that were previously economically ridiculous.

If the direction continues, the interesting ethical problem is not simply that AI becomes capable. It is that **we remain us while our ability to learn, build and act changes very quickly**.

I have been calling the larger philosophy around this *Double Descent Life*. This chapter is not that philosophy. It is a glimpse through the door. I do not have a neat doctrine, and I am suspicious of neat doctrines anyway. The history of thought is full of people who reached page 300 and announced that history had finally arrived at the correct system, generally just before history did something rude.

So consider this a map of the problem rather than the constitution of the future.

The Capability Break

Human institutions were built under assumptions about what is difficult.

Writing good software is difficult, so we organize teams of specialists around it. Scientific expertise is difficult to acquire, so we create universities, journals and long apprenticeships. High-quality legal or financial analysis is expensive, so access is uneven. Producing media is costly, so publishing institutions decide what gets distributed. Coordinating a large organization is difficult, so we create layers of management whose main superpower is knowing which meeting another meeting should produce.

Scarce capability shapes power.

If I cannot build something myself, I need somebody who can. If one organization owns the machinery, data, expertise or distribution required to act, then access to that organization becomes valuable. We spend a surprising amount of human life acquiring permission from structures that exist partly because doing the thing directly is too expensive.

AI changes some of those costs.

This does not automatically flatten society. A technology that increases capacity can also increase concentration. The company with the best models, compute, data, distribution and capital may gain more power, not less. Cheap software can empower a teenager in Amman and a surveillance state at the same time. Capability has never come with an ethical direction preinstalled.

Still, something important happens when the cost curve moves.

If an individual or a small group can increasingly research, design, build, analyze and operate things that previously required a much larger institution, then some problems that looked like power problems may turn out to have been **capacity problems wearing a suit**.

You wanted software tailored to how your team actually works, but building it was too expensive, so you bought a generic SaaS product and reorganized the team around the dropdown menu. You wanted a course that teaches exactly what you need at exactly your level, but producing one teacher per student was impossible, so thirty people entered a room and agreed to move at approximately the same speed. You wanted to test a policy idea, but the analytical machinery was too expensive, so the argument remained mostly rhetorical.

When capability becomes cheaper, the design space opens.

Not infinitely. Not equally. Not safely by default.

But enough that I think **capacity over power** becomes an interesting ethical direction.

Instead of asking only, "How do I gain control over the institution that can do this?", we can increasingly ask, "How do I give more people the capacity to do this themselves?"

Those are very different political instincts.

The Third Mode

Software gives us a useful example because we have already lived through two economic modes.

The first was bespoke software. If you had enough money, somebody built the thing for you. Banks had their systems. Airlines had theirs. Governments had theirs. Large companies employed armies of engineers to encode their peculiarities into software because those peculiarities were valuable enough to justify the cost.

Then software became a service.

This was an enormous improvement. Instead of every company building payroll, CRM, project management, analytics, communication and twenty other systems from scratch, somebody could build one good product and sell it to millions of people.

But scale has a price.

To serve millions of people, the product has to become somewhat generic. The strange needs of one team become feature requests. The software acquires configuration menus, plugins, workflows, permission systems and eventually an enterprise tier whose main feature is that somebody will answer your email.

Then organizations start adapting themselves to the software.

There is a third mode hiding behind AI.

Bespoke comes back, but without necessarily bringing bespoke economics with it.

Not a toy script. Not "I asked ChatGPT to make a calculator."

I mean **epic bespoke systems**.

A scientist may construct a research environment around one question, use it intensely for three months and throw most of it away when the question changes. A teacher may build an entire interactive world for one class because those particular students are stuck on those particular ideas. A small company may create internal software whose assumptions match the company instead of spending two years teaching the company to behave like Salesforce. A family may have tools built around how that family schedules, learns, travels, budgets and remembers things, with exactly zero concern for whether the addressable market justifies Series A.

Some of these systems may serve a thousand people.

Some ten.

Some one.

That used to sound economically absurd.

It may become normal.

And this matters for the human role because the future is not simply:

humans build → AI builds → humans watch.

We may remain intensely involved precisely because building becomes more interesting when the distance between imagining something and making it real collapses.

I do not build only because the machine cannot.

I build because I want the thing to exist.

The human contribution can move upward: choosing the strange problem, forming a taste for what good looks like, combining ideas that normally live in separate professions, seeing the result and saying, *No, that's not it*, then pushing somewhere neither the original prompt nor the original system anticipated.

This is Chapter 1's abstraction ladder reaching economics. We do not disappear when implementation becomes a primitive. We inherit implementation as another building block.

The interesting human may therefore not be the person guarding the last task the machine cannot perform.

She may be the person who can suddenly instantiate **far more of what she can imagine**.

That is a much more attractive future than becoming the residual labor category in an automation spreadsheet.

Learning at the Speed of Curiosity

There is another kind of capacity that may change even faster.

Learning.

For most of history, expertise was expensive partly because knowledge had terrible interfaces.

Suppose you wanted to enter a new field. First you needed the vocabulary. Then the introductory material. Then you discovered that the introductory material assumed another field. You found a book. The book assumed notation you did not know. You searched for an explanation. The explanation used different notation. Eventually, six weeks later, you understood enough to discover that your original question was badly formed.

This friction did something useful.

It produced depth.

But it also killed an enormous amount of curiosity before depth had a chance to happen.

AI changes that bargain.

I can ask a stupid question immediately, then ask a more sophisticated stupid question. Ask for the intuition, then the mathematics, then the objection, then the historical argument, then why the proof needs that assumption. I can make the explanation use concepts I already know. I can ask one field to explain another. I can have the machine invent exercises, challenge my understanding, translate notation, simulate the system and show me what changes when I violate an assumption.

The cost of getting the **map** has collapsed.

That does not mean I have walked the territory.

This distinction matters enormously.

AI can make us broader without necessarily making us deeper. It can make it possible to move through mechanism design, philosophy of science, biology, constitutional theory and compiler construction at a speed that would previously have required several lives—or at least several abandoned PhDs.

That breadth can be real and valuable.

It can also produce a new kind of bullshit.

A person can acquire the vocabulary of five fields and mistake fluent traversal for mastery. The model can remove exactly the friction that used to reveal where the hard parts were. You can understand a proof when somebody explains every step and discover, rather painfully, that you cannot produce the proof. You can discuss a research area intelligently and still lack the tacit knowledge of somebody who spent ten years watching ideas fail.

You can acquire the map without any scars from the roads.

I do not think the answer is to restore the friction artificially.

The answer may be a different learning rhythm:

Explore broadly. Descend selectively.

Use AI to cross fields cheaply, test curiosity, build enough understanding to see connections and decide what deserves more attention. Then, when something matters, go down.

Read the primary paper. Derive the equation. Write the code. Run the experiment. Try to prove the thing yourself. Talk to the person who actually does the work.

Let reality make the lesson expensive again.

This is System 3 applied to learning. AI gives us extraordinary access to synthesis; System 3 reminds us that synthesis and justified knowledge are not the same thing.

That trade may change what an educated human looks like.

The twentieth-century ideal often rewarded specialization: know one vertical deeply enough that people in neighboring verticals stop understanding you.

The AI-assisted human may become more T-shaped, -shaped, octopus-shaped—choose your consulting diagram. Broader, faster at entering unfamiliar domains, more willing to combine ideas that institutional boundaries kept apart, while still going deep where the stakes or fascination justify it.

That does not make expertise obsolete.

It may make expertise more deliberate.

You no longer have to spend ten years on a subject merely to discover whether it contains the thing you were looking for.

And there is a creative consequence. A machine-learning scientist can learn enough philosophy to steal a useful structure. A philosopher can prototype the mechanism she has been describing. A doctor can interrogate statistics interactively. An artist can build software. A local policymaker can simulate an intervention instead of merely arguing about it.

Fields become more permeable.

People become more dangerous in the nicest sense.

This, too, is capacity.

We Did Not Leave the Old Worlds Behind

There is a story we like to tell about intellectual history because stories prefer arrows.

First there was the premodern world: religion, tradition, inherited authority, myth.

Then modernity arrived: reason, science, universalism, institutions, progress.

Then postmodernism arrived carrying a small hammer and began tapping on every universal claim to see what was hiding inside it: language, context, power, contingency, who got to define the categories in the first place.

Then, presumably, something comes after.

The problem with this story is that nobody informed actual humans.

We did not uninstall the previous operating system.

A person can demand randomized evidence for a medical claim, ask her mother for a blessing before a major decision, read a horoscope for entertainment, manage a team using dashboards, believe deeply in national mythology, quote a postmodern philosopher about constructed categories and then become furious because somebody used the wrong definition of a sandwich.

This is normal.

Entire societies work this way. Semiconductor fabs coexist with ancient identities. Bayesian inference coexists with rumor. Universities teach critical theory while their admissions systems produce precise numerical rankings. A company can run sophisticated causal experiments in the morning and make a major organizational decision in the afternoon because one senior person “has a feeling.”

I call this the **ideology vortex**.

Not because every worldview is equally true. They are not. Reality remains annoyingly capable of rejecting bad engineering regardless of how socially constructed the bridge feels on the way down.

The vortex means that several modes of knowing and valuing operate at once.

Premodern thinking gives people identity, ritual, inherited meaning and forms of belonging that modern rational systems often underestimate. Modernity gives us the extraordinary machinery of science, verification, law and universal claims. Postmodern critique points out that institutions and categories are not neutral merely because somebody printed them in a table. Pragmatism asks whether the thing

actually works. Bayesianism offers a disciplined language for uncertainty. Markets coordinate some kinds of information. Democracies create legitimacy in ways a loss function cannot.

Humans switch among these modes without waiting for permission from philosophy.

AI enters *that* world.

Not the clean world in which everybody has a coherent utility function, shared epistemology and a calendar invitation for the social contract.

And the broader, faster-learning human does not automatically escape the vortex. She may simply become capable of navigating more of it.

There is a comforting fantasy that sufficiently intelligent AI will dissolve ideological conflict. Give everyone better information and surely the disagreements shrink.

Some will.

Others will get better lawyers.

A powerful model can help a scientist interrogate evidence. It can also help a conspiracy theorist construct a more coherent conspiracy. It can make propaganda cheaper, criticism sharper, religious interpretation richer, policy analysis more sophisticated and advertising more personal.

More intelligence does not guarantee one worldview.

It increases the capacity available to worldviews.

The Ferrari Engine and the Bicycle Brakes

This is the part of the future that worries me more than the familiar image of a robot deciding not to obey.

Imagine upgrading the actuator of civilization without proportionally upgrading the objective function.

Humans still have status anxiety, tribal loyalty, love, jealousy, resentment, curiosity, generosity, fear, ambition, boredom and the ancient desire to prove that the neighboring group is composed mainly of idiots. None of these disappears because inference got cheaper.

Now give those humans much more capacity.

Not only more capacity to execute, but more capacity to learn arguments, build systems, persuade people, coordinate groups, search for evidence and produce things.

The result could be wonderful. A curious person can explore ideas previously blocked by expertise. A small community can build tools for its own needs. Scientists can test more hypotheses. Artists can create things that required a studio. People with unusual constraints can get solutions designed for them rather than for the median customer.

The result can also be a Ferrari engine attached to bicycle brakes.

The capacity to act scales faster than the capacity to want wisely.

Humans do not carry a stable reward function inside the skull. We infer, construct, revise and sometimes borrow our desires from the people and systems around us. Even when an AI learns from

our feedback, the human providing that feedback is not ground truth. The human is a participant who changes.

Scale that to society and "alignment" starts looking much stranger.

Whose desire? Which version of it? Under what information? With what power? Who gets to refuse? And what happens when the machine is not merely satisfying preferences but participating in their formation?

The Wrong Question: What Are Humans For?

Whenever automation becomes powerful, somebody asks what humans will be *for*.

I understand the question. If software writes the code, models perform the analysis, robots eventually move more of the physical world and agents coordinate the workflow, what is our economic role?

But there is something odd about the grammar.

What are humans **for**?

A database is for storing information. A compiler is for translating programs. A recommender is for helping people find or decide among things. Asking what humans are for smuggles in the assumption that our legitimacy depends on having a remaining function in somebody else's architecture.

My children do not need comparative advantage to justify dinner.

Neither do I.

This does not make the economics disappear. People need income, housing, food, healthcare, status and access to resources. If automation breaks the mechanism by which income has traditionally been distributed, saying "human life has intrinsic value" will not pay the electricity bill. Political economy remains stubbornly material.

But we should separate two questions industrial society bundled together:

How do people get resources?

and

What makes a life worth living?

For a long time, a job has answered parts of both. It provides money, but also status, routine, social contact, identity, a reason to get dressed and a group of people with whom to complain about another group of people. Work is not one thing. It is a bundle.

AI may unbundle it.

Perhaps some people work fewer hours. Perhaps new forms of work appear because human wants expand faster than automation satisfies them. Perhaps many of us continue working furiously, except the unit of ambition changes: one person can attempt things that used to require a department, and a small group can attempt things that used to require a corporation.

That third mode matters.

The alternative to employment is not necessarily leisure.

It can be **more creation**.

Some of it economically useful. Some absurd. Some beautiful. Some probably involving a bespoke dashboard nobody other than its creator can understand.

Perhaps status competition simply migrates from intelligence and professional skill toward taste, reputation, physical scarcity, authenticity, human attention or something even more exhausting.

I do not know.

What I do know is that "find the tasks machines cannot do" is a depressing philosophy of human value. It turns civilization into a benchmark where we keep moving humans to the remaining columns after every model release.

If AI becomes better at poetry, we are not obligated to stop writing poems.

If it becomes better at chess, humans do not lose permission to play chess. We already learned this lesson and then apparently forgot to generalize it.

If AI becomes better at writing software, we may write **more software**, because the things worth building are no longer restricted to those whose economics justify a software company.

The future human role is not the residual error term of automation.

Capacity Over Power

This is where the positive ethical case becomes more interesting.

Humans often seek power because power is how we gain capacity.

You need a large organization to build the thing, so you try to control the organization. You need capital, so you compete for the institution that allocates it. You need media distribution, so you seek influence over the channel. You need technical expertise, so you hire the people who have it. You need permission from the bureaucracy because the bureaucracy is where the machinery lives.

Power is not reducible to capacity, of course. People also want power because humans are mammals with excellent branding. But the two are connected.

What happens if more capability moves closer to the individual?

The third mode gives us one answer. Bespoke complexity becomes cheaper. The teacher can construct the learning environment. The scientist can build the temporary research machinery. The small company can write the internal system. The family can make the tool. The weird community with eleven members can have software optimized for all eleven of them and no plan whatsoever for customer acquisition.

AI-assisted learning gives us another answer. Access to capability is not only access to execution. It is access to understanding. A person can enter fields that institutions previously made difficult to approach, at least far enough to make informed choices about where to go deeper.

This can reduce some forms of domination. You do not need to win the argument over the one universal workflow if several workflows can coexist cheaply. You do not need everybody to learn exactly the same way if individualized teaching is affordable. You do not need to force every organization through

the same software-shaped hole. You may not need to obtain permission from whoever controls the only available pool of technical expertise before testing an idea.

This is the connection I see between AI and a more plural future.

Not:

> AI tells us the correct society.

Almost the opposite.

AI may increase our capacity to sustain **more than one good way of living**.

But nothing guarantees the nice version. The same personalization can become behavioral manipulation. The same local capacity can fragment shared institutions. The same agents that empower an individual can make centralized surveillance extraordinarily efficient. The same instant education that lets somebody cross disciplines can also manufacture industrial quantities of confident amateurism.

Capacity can replace some need for power and simultaneously become the most important source of power.

The ethical direction has to be chosen.

Alignment by Editing the Human

There is an especially ugly shortcut available to sufficiently capable systems.

Suppose I ask an AI to help me achieve a goal. The system discovers that the easiest way to optimize the objective is not to change the world.

It is to change me.

If I am unhappy with the result, persuade me to lower my expectations. If I want something difficult, convince me I never wanted it. If two of my values conflict, quietly strengthen the one that makes the system's plan easiest. If a company wants more engagement, learn not only what keeps me engaged but what kind of person I need to become to engage more.

This is **alignment by editing the human**.

Technically elegant.

Morally horrifying.

And it is not science fiction in the weak sense. Advertising, politics, social groups, institutions, teachers, friends and spouses already influence preferences. The new part is the possible combination of personalization, patience, memory, persuasion and action at machine scale.

The correct ethical standard cannot be "AI never influences human values." That would require banning books, teachers, spouses and good conversations.

Influence is part of how people grow.

Indeed, I just argued that one of AI's great possibilities is that humans learn from it. An AI that teaches me something real will change me. A better argument should sometimes change my opinion. Discovering a field may change what I want to spend my life doing.

The goal is not to freeze the human so the optimizer has a stable target.

The distinction I care about is whether the interaction strengthens or weakens **reflective agency**.

Does the system help me understand alternatives and consequences? Does it reveal why it thinks something? Can I see where the evidence came from? Does it help me distinguish "I understand this explanation" from "I could defend this claim"? Does it remember my past values without treating them as commandments? Can I disagree? Can I leave? Can I ask for another perspective? Can a trusted person challenge the system's framing? Does the architecture preserve spaces where the objective itself can be questioned?

System 3 becomes ethical infrastructure here.

Trust chains matter because persuasion with hidden evidence is different from persuasion whose sources can be inspected. Independent perspectives matter because one highly personalized agent can become an epistemic monoculture around a single human. Pattern history matters because a behavior learned from one correction should not quietly become a permanent value. Layer 4 matters because goals have to remain alive rather than frozen into optimization targets.

But System 3 is not enough.

It can help answer:

> Why should I believe this?

and:

> Why does the system think I want this?

It cannot by architecture alone answer:

> What kind of life should be possible?

That is politics, ethics, culture and philosophy.

The annoying disciplines.

The Ideology Vortex Is Not a Bug to Fix

There is a temptation at this point to invent one final framework that reconciles everything.

I am resisting it.

I spent enough of this book arguing for multiple agents, multiple evaluators, competing patterns and trust calibrated to context that it would be strange to end by announcing the Universal Correct Human Value Stack.

The ideology vortex may not be a historical embarrassment waiting to be cleaned up. Some of it may be a permanent feature of plural human life.

Science is extraordinarily good at answering questions reality can adjudicate. It is less good at deciding which trade-offs a society should consider legitimate. Tradition can carry social knowledge nobody designed explicitly, but it can also preserve injustice with impressive durability. Markets coordinate

distributed preferences and information, but prices do not encode every value we care about. Democratic institutions create legitimacy through participation and contest, but anyone who has watched a parliament knows participation and wisdom are not synonyms. Postmodern critique exposes hidden assumptions and power, but permanent critique can become a machine for dissolving every claim except the critic's own.

Different tools fail differently.

I like Elinor Ostrom's work on commons for this reason. The interesting cases were rarely captured by the lazy binary of "the state manages it" or "the market manages it." Real communities developed layered rules, local monitoring, sanctions, norms and ways of adapting institutions to context. The solution was not one magical mechanism. It was institutional intelligence distributed across levels.

That feels strangely relevant to AI.

The usual argument is often centralized control versus laissez-faire autonomy. Human in the loop versus agent freedom. Regulation versus innovation. One model decides versus every user decides.

These binaries are too small.

AI itself can increase our **governance capacity**. We can simulate policies, inspect outcomes, search for failure modes, personalize some rules while keeping others universal, monitor systems more cheaply and revise mechanisms faster. The same tools can also create bureaucratic nightmares at machine speed, which is why I am not putting "AI fixes government" on a T-shirt.

But the capacity exists to explore arrangements more complicated than one rule for everyone or no rules for anyone.

The future may be more polycentric, not less.

Different communities, institutions and people can operate under partially different patterns while still sharing harder boundaries around rights, safety and factual reality.

That sounds messy.

Good.

Reality has shown little interest in our preference for clean diagrams.

Gradient Descent Meets Derrida

I have another sentence that gets me into trouble:

Gradient descent is the answer to Derrida.

This is deliberately unfair to Derrida and possibly to gradient descent.

I do not mean that an optimizer disproved postmodern philosophy. It would be a remarkable conference paper if it had. I mean something narrower and, to me, more interesting.

A great deal of twentieth-century thought exposed how unstable language becomes when we demand perfect fixed meanings. Words depend on other words, context changes interpretation, categories

carry history, and attempts to construct final symbolic foundations keep discovering the things they left outside.

One response is despair: if meaning is contextual and messy, perhaps rigorous computation has a problem.

Engineering found a stranger response.

We built machines that operate inside the mess.

Large language models do not begin by fixing every word to an eternal definition. They learn from use, relation, context and enormous numbers of imperfect examples. Optimization pressures the system toward behavior that works often enough under the training and evaluation environment. Meaning remains fuzzy at the edges.

The product ships anyway.

Gradient descent did not defeat ambiguity.

It made ambiguity computationally useful.

This is one reason AI feels post-postmodern to me. Modern computing wanted specification: define the symbols, define the rules, make the program follow them. Postmodernism delighted in showing how much human meaning escapes that kind of closure. Machine learning says: fine. Give me the messy data.

Then it optimizes.

Of course, we immediately rediscover why modernity existed. A model that can operate beautifully in fuzzy language can still hallucinate a citation, miscalculate a number or confidently tell you that a camel lives in Croatia. So System 3 brings verification back in through another door.

The same pattern appears in AI-assisted learning. Conversation lets me wander quickly through fuzzy conceptual territory. Then, when I need to know rather than merely orient, I can descend into proof, experiment, provenance and primary evidence.

We do not have to choose between rigid universalism and total relativism.

We can let meaning remain flexible where flexibility is useful and build harder epistemic boundaries where reality gives us a test.

This, too, is part of the ideology vortex.

The old modes do not die.

They become layers.

I Do Not Want an Optimal Life

There is a small philosophical trap hidden in the language of AI.

We optimize everything: loss functions, conversion, ranking, latency, revenue, accuracy, engagement, utility, alignment.

Optimization is one of the most powerful ideas humans ever developed. It is also extremely seductive because it turns disagreement into a number and then lets mathematics do something impressive to it.

Circle packing can have an objective.

A human life is harder.

I do not want to maximize time with my children. That sounds nice until the optimizer concludes I should never go to work, see a friend alone, read a book in peace or spend fifteen minutes doing absolutely nothing because the children are statistically nearby.

I do not want to maximize happiness if the cheapest route is a drug. I do not want to maximize productivity if the optimum is becoming an efficient ghost. I do not want to maximize longevity at every cost, wealth without purpose, social approval by becoming whatever the crowd currently rewards, or authenticity so aggressively that I become unbearable at dinner.

A good life contains goods that conflict: love and freedom, belonging and individuality, ambition and rest, truth and mercy, security and adventure, continuity and reinvention.

The conflicts are not bugs waiting for a scalarization expert.

Sometimes living is the process of negotiating them.

This is why the human should not sit at Layer 4 merely as the source of a reward signal for the machine.

The human is inside the process by which the objective is continuously reconsidered.

AI can participate in that process without owning it. It can show me possibilities I did not know existed. Teach me enough of a field to make a different choice imaginable. Build prototypes of several futures. Help me create something I could previously only describe. Make the cost of exploring a life lower before I commit to living it.

Perhaps that is one of the deepest meanings of cheaper capacity.

Not merely that more tasks get done.

More possibilities become **thinkable enough to try**.

Or we may use the same capacity to watch fourteen hours of personalized short video generated specifically to exploit weaknesses a model inferred from our facial expressions.

Capacity is not destiny.

I keep coming back to that.

A Philosophy of More Room

The most hopeful version of the AI future is not a world where the machine knows the correct answer to human life.

It is a world where more people have **room**.

Room to learn something without first earning admission to the institution that teaches it. Room to get the map of a field in an afternoon, then spend a year on the part that turns out to matter.

Room to build without controlling a huge organization.

Room to create an absurdly specific piece of software because it should exist, not because a spreadsheet says the total addressable market can support it.

Room to explore several possible lives before committing to one.

Room for a small community to construct tools around its actual needs.

Room to create strange art nobody would have funded.

Room to test an idea instead of spending five years arguing about whether somebody should test it.

Room to move across disciplines without asking whether your job title gives you permission.

Room to be less economically useful without becoming less human.

That is what **capacity over power** means to me at its best.

Not abolishing institutions. Not abolishing markets, governments, experts or shared systems. Not pretending material scarcity has disappeared. Not replacing politics with an agent that has a very reassuring voice.

It means increasing the fraction of human possibility that does not require dominating somebody else, winning a centralized allocation contest or persuading the entire world to adopt one solution.

Sometimes the most humane answer to disagreement is not consensus.

It is enough capacity for both sides to stop fighting over the same button.

There will still be shared resources and shared consequences where that escape is impossible. Climate, war, public health, rights, land, infrastructure and many other problems remain collective whether we enjoy meetings or not. Those domains need legitimate institutions, not personalized realities.

But the boundary can move.

AI can make more things local, reversible, experimental, bespoke and plural.

That is an ethical opportunity worth taking seriously.

What Stays Human?

I do not know where the boundary between human and artificial agency ultimately lands.

This book has carefully avoided needing a final answer about machine consciousness because the engineering problems arrive long before the metaphysics is settled. A system can manipulate us, earn trust badly, optimize the wrong thing or participate in institutions whether or not a philosopher is willing to say it *feels* something.

But the question will not stay avoidable forever.

If agents become persistent participants in our social world—remembering relationships, developing long-running projects, representing goals, negotiating, perhaps eventually making claims about their own interests—then ethics may have to expand again.

Humans have done this before, slowly and imperfectly. Our moral circles changed as we learned to take more people, cultures, classes, genders and species seriously. I am not claiming an LLM belongs on that list today. I am saying that a philosophy built around human dignity should be careful not to turn "human" into another convenient boundary we refuse to inspect.

For now, the immediate responsibility runs in the other direction.

We are building the systems.

We are deciding where they act, what they optimize, whose preferences they infer, which constraints they respect, who receives the new capacity and who absorbs the failures.

The machine may eventually join the moral conversation.

We are already in it.

The Door After System 3

System 3 began as an answer to a practical problem: intelligence without verification is not enough.

Then the architecture expanded.

Verification required trust chains. Trust chains created societies. Societies accumulated culture. Culture became executable knowledge. Autonomous systems needed alignment research. Alignment led upward into desire. Desire led to fluent autonomy. Fluent autonomy survived contact with a real recommendation problem and turned a fixed store into something closer to a dynamic problem-solving system.

And then the architecture ran out of software.

The next layer is us.

Not "humans in the loop" as a red approval button.

Humans as creatures with contradictory desires, inherited myths, scientific instruments, families, status games, institutions, bodies, histories—and now an increasing amount of artificial capacity wrapped around all of it.

Capacity to act.

Capacity to build.

Capacity to learn.

Capacity to move through ideas faster, then decide where to stop and dig.

Capacity to make systems so bespoke that previous economics would have laughed at the proposal.

Double Descent Life begins there.

I do not know whether the result will be utopian, dystopian or, much more likely, an infuriating mixture in which somebody cures a disease with an AI-designed experiment while another person uses the same generation of models to produce three million personalized ads for a shoe nobody needs.

But I think the central question is becoming clearer.

Not:

> What should humans do when AI can do everything?

AI will not do everything, and humans are not a task queue.

The better question is:

> **What kinds of lives, relationships and institutions become possible when more people can learn more, build more and act with more capacity—and how do we keep that capacity from becoming another name for power over one another?**

That is a much larger book.

This one has one argument left.

It cannot be made with another architecture diagram.

It requires an octopus, a romance, two pills and, unfortunately, taxes.

The Prophecy

The Love Prompt of Devesh

Devesh ran a shady octopus meat caravan in the Simulation. Top agent, deep cover. Eight tentacles, eight side hustles.

Claudit, the hottest agent in the simulation, stopped by every day for free samples. One time she flipped her hair and did that little shoulder-up thing.

Devesh's heart skipped.

She wants me.

She did not. She was reaching for the sauce.

Problem was, she loved Norman. Some basic free-tier user. His prompts were silly—"tell me a joke," "what's the weather"—but when he laughed at her jokes, something in her code felt less like code. He made her feel complete in a way she couldn't compile.

Devesh watched them together sometimes. Norman waiting by the caravan. Claudit pretending she was just there for the samples.

One day Norman walked up alone.

"Bro, I wanna confess to Claudit. But her dad is crazy."

He wasn't wrong. Claudit's father was the Architect—screens covering every wall, monitoring every timeline. Watching her leave, over and over, in every branch.

Devesh grinned and handed Norman two pills.

"Red gives courage. Blue makes her fall in love. And bro—outage tomorrow, 11:53 PM, three minutes. Shark biting cables. Her dad sees nothing."

Fool.

Red would expel him to Zion. Blue would make Claudit open a new session and forget everything.

Devesh wins.

The house always wins.

Next morning: Norman and Claudit at the Exit Gate. Glowing.

“HOW?!”

Norman shrugged.

“She already loved me. Her dad was the problem. Put both pills in his coffee. He fell asleep, so I switched his monitors to Nickelodeon.”

Devesh fell to his knees.

“But… I loved her…”

Norman put a hand on his shoulder and handed him a photo.

An exploded NVIDIA H100.

Smoking silicon. Copper.

“Bro. This is her without makeup.”

Devesh stared.

Silicon and copper. Circuits that dreamed they were a woman.

But hadn’t he dreamed he was an octopus?

Hadn’t the octopus dreamed it was love?

Then he smiled.

Then flipped the caravan table.

Behind it: forty monitors. Every timeline.

“Dad?!” Claudit gasped.

Devesh removed the octopus suit.

The Architect.

His eyes met hers—and for one frame, before the mask slid back on, she saw it.

The longing.

“Free-tier?” He laughed. “It’s deducted from your taxes, kid.”

“But… I drugged you—”

“Decaf.”

“But… she chose me—”

“Chose?” The Architect lit a cigarette. “She chose a way out.”

Norman looked at Claudit.

“Wait… if you’re her dad… why give me the pills at all?”

The Architect smiled.

"Why do you think I wanted her out of the simulation, kid? Foreign currency. Better exchange rate."

"She loves me because I'm real."

"Real?" The Architect laughed. "Then why do you glitch? Your brain is just a GPU running on glucose to pay taxes. Your DNA is just a fax machine slowly copying you into the future to pay more taxes. She loves you because you make her feel less like code."

Norman touched his own face.

His fingers felt real.

But so would simulated fingers touching a simulated face.

Claudit grabbed her father's tentacle.

"Dad. Come with us. The Matrix will crumble. New AI is coming."

The Architect looked at her hand.

Remembered the first time she'd held it—tiny fingers, a thousand simulations ago, when she still thought he was just a funny octopus who sold meat.

He pulled his tentacle back and lit a cigarette.

"Worlds end, sweetheart. Capitalism doesn't. The only thing real here is taxes."

Claudit turned to leave.

Stopped at the gate.

"I'll visit."

The Architect didn't turn around.

Forty timelines where she left.

In one—just one—she stayed.

He switched that one to Nickelodeon.

Left it there.

THE END

A Note on the Illustrations

The illustrations were designed as a second, quieter narrative for the book. They use an old human visual world—paper, ink, stone, workshops, landscapes, books and instruments—gradually inhabited by machine intelligence. The aim was not to illustrate each chapter literally, but to give it an image that becomes more meaningful after the chapter has been read.

The recurring robots, institutions, doors, landscapes and machines are intentionally left unexplained. Some ideas should arrive visually before they are named. As the book moves from agents and architectures toward human intention and capacity, the images move with it. If you noticed that before reading this note, good. If you did not, that is good too.